

Complete Skills Catalog

Full explanations and details for every skill installed in this environment: what each skill does, its real commands, when to use it, and its requirements.

Total skills documented	110
Active in the skill catalog	64
Installed but not active	46
Custom workspace skills (~/.openclaw/skills)	43
Plugin skills (~/.openclaw/plugin-skills)	10
OpenClaw built-in skills (npm package)	57

Status legend: **Active in skill catalog** means the skill is exposed to the assistant as an invokable skill in this environment.
Installed, not in active catalog means the skill ships with the platform but is not exposed in the current catalog (often because its CLI or operating system is unavailable here).
Generated 2026-09-24.

How this catalog is organized

A skill is a capability package: a SKILL.md instruction file (plus optional helper scripts) that the assistant loads on demand when a task matches the skill's purpose. Skills are grouped by where they live on disk:

- **Custom workspace skills** - `~/.openclaw/skills/`: environment-specific skills added to this workspace (design craft, MiMo multimodal/TTS, office documents, data tooling).
- **Plugin skills** - `~/.openclaw/plugin-skills/`: skills delivered by OpenClaw plugins (DingTalk, Feishu/Lark, QQ Bot, DWS).
- **OpenClaw built-in skills** - `/usr/lib/node_modules/openclaw/skills/`: the skill library that ships with the OpenClaw package.

Each entry lists the skill's exact file location, category, catalog status, a one-line summary, then an overview, concrete capabilities, real usage commands taken from the source, trigger scenarios and requirements. Where two skills share a name (**github**, **healthcheck**, **skill-creator**) or are duplicates (**self-improvement** / **self-improving-agent**), each entry states which copy is which.

Quick Index

Part 1 - Custom Workspace Skills (Active in Skill Catalog)

Skill	Summary
adapt	Adapts existing designs to different screens, devices, platforms, and contexts while keeping experiences consistent and native-feeling.
agent-browser	Headless browser automation CLI for navigating sites, filling forms, clicking, scraping, testing, and screenshots.
akshare	Fetch free Chinese financial data (A-shares, Hong Kong stocks, futures, funds, macro indicators) with the AKShare Python library.
animate	Reviews a feature and adds purposeful animations, micro-interactions, and motion effects that improve usability and provide feedback.
arrange	Improves layout, spacing, and visual rhythm, fixing monotonous grids and weak hierarchy to create intentional compositions.
audit	Runs comprehensive interface quality checks across accessibility, performance, theming, responsiveness, and anti-patterns, producing a prioritized audit report.
bolder	Amplifies safe or boring designs with stronger typography, color, spatial drama, and motion while preserving usability and coherence.
clarify	Improves unclear UX copy, error messages, microcopy, labels, and instructions so interfaces are easier to understand and use.
clawhub-cli	Agent playbook for the ClawHub CLI: search, install, list, update, publish, and sync agent skills from clawhub.ai.
code-generator	Multi-language code scaffolding: generate functions, classes, API endpoints, CRUD code, tests, refactoring advice, and language conversion guides.
colorize	Adds strategic color to monochromatic designs, using semantic, accent, and decorative color to improve meaning, hierarchy, and warmth.
critique	Evaluates design effectiveness from a UX perspective, covering hierarchy, architecture, emotional resonance, and AI slop detection with actionable feedback.
Data Analysis	Decision-first data analysis and visualization methodology with metric contracts, chart guidance, statistical rigor checks, and decision brief formats.
delight	Adds moments of joy, personality, and unexpected polish through micro-interactions, copy, illustrations, sound, and easter eggs.
distill	Strips designs to their essence by removing unnecessary complexity across architecture, visuals, layout, interaction, content, and code.
Excel / XLSX	Create, inspect, and edit Excel workbooks with reliable formulas, dates, types, formatting, recalculation, and template preservation.
extract	Extracts reusable components, design tokens, and patterns into the design system, then migrates existing usages to shared versions.
find-skills	Highest-priority skill discovery flow: search skillhub first, fall back to clawhub, present options, and install skills.
frontend-design	Creates distinctive, production-grade frontend interfaces with bold, intentional aesthetics that avoid generic AI-generated design patterns.
github	Compact GitHub skill using the gh CLI for pull requests, CI runs, and advanced API queries with JSON and jq filtering.
harden	Strengthens interfaces against edge cases, errors, internationalization issues, and real-world usage so designs survive production.
healthcheck	Track water intake and sleep in a JSON file using self-contained Node.js one-liner scripts for logging and stats.
humanizer	Removes AI writing patterns from text using Wikipedia's signs-of-AI-writing guide, then adds genuine voice and personality.
mimo-omni	Analyzes images, video, and audio with Xiaomi MiMo multimodal models for description, OCR, transcription, and understanding.

Skill	Summary
mimo-tts-voice-clone	Clones a voice timbre from a short reference recording and synthesizes styled speech as a standard WAV file.
mimo-tts-voice-design	Generates a brand-new voice timbre from a one-line natural-language description and synthesizes speech as WAV.
mimo-tts-wav	Converts text to speech or song with the MiMo TTS API and saves a standard WAV file.
normalize	Analyzes and redesigns a feature to match the project's design system standards, aesthetics, tokens, and patterns.
onboard	Designs onboarding flows, empty states, and first-time user experiences that move users to value as fast as possible.
optimize	Identifies and fixes frontend performance issues across loading, rendering, animation, images, and bundle size with measured improvements.
overdrive	Pushes interfaces past conventional limits with ambitious techniques like shaders, virtual scrolling, spring physics, and scroll animation.
pdf	PDF toolkit for extracting text and tables, creating PDFs, merging, splitting, rotating, watermarks, encryption, and form filling.
polish	Performs a final meticulous quality pass fixing alignment, spacing, states, copy, and detail issues before shipping.
Powerpoint / PPTX	Create, inspect, and edit PowerPoint PPTX decks with template fidelity, layout mapping, notes, charts, and visual QA.
quieter	Reduces visual intensity in overly bold or aggressive designs, creating a refined, approachable aesthetic without losing impact.
self-improvement	Logs learnings, errors, corrections, and feature requests to markdown files for continuous agent improvement.
skill-creator	Guide for building effective agent skills: structure, writing principles, scaffolding scripts, and validated packaging.
skill-vetter	Security-first vetting protocol for third-party skills: source checks, code review, red flags, and risk rating.
skillhub-preference	Policy guidance preferring skillhub for skill search, install, and update, with clawhub as the documented fallback source.
teach-impeccable	One-time setup that gathers project design context through codebase exploration and questions, then persists it to .impeccable.md.
typeset	Improves typography by fixing font choices, hierarchy, sizing, weight consistency, and readability for intentional, polished text.
Word / DOCX	Create, inspect, and edit Word DOCX files with reliable styles, numbering, tracked changes, and layout fidelity.

Part 2 - Plugin Skills (Active in Skill Catalog)

Skill	Summary
dingtalk-channel-rules	Defines DingTalk conversation output style and routes all DingTalk business operations through the dws CLI, not the connector.
dingtalk-troubleshoot	Troubleshoots DingTalk connector failures: dws CLI installation, OAuth login, token expiry, permissions, bot connectivity, and recoverable errors.
dws-cli	Manages DingTalk products such as AI tables, calendar, contacts, chats, todos, approvals, attendance, reports, and DING via the dws CLI.
feishu-doc	Reads, writes, creates, and restructures Feishu documents and Docx tables through the single feishu_doc tool.
feishu-drive	Manages Feishu cloud storage: list folders, get file info, create folders, move and delete files via feishu_drive.
feishu-perm	Manages Feishu document and file permissions by listing, adding, and removing collaborators with view, edit, or full_access rights.
feishu-wiki	Navigates Feishu knowledge bases: list spaces and nodes, get node details, create, move, and rename wiki pages.

Skill	Summary
qqbot-channel	Proxies QQ Open Platform HTTP API calls via qqbot_channel_api with automatic token auth for guilds, channels, members, forums, announcements, and schedules.
qqbot-media	Sends and receives QQ rich media (images, voice, video, files) using qqmedia tags with the type inferred from file extensions.
qqbot-remind	Creates, lists, and cancels QQ reminders via qqbot_remind, with a cron-tool fallback for one-time and recurring schedules.

Part 3 - OpenClaw Built-in Skills (Active in Skill Catalog)

Skill	Summary
canvas	Presents HTML on connected node canvases with navigation, JavaScript evaluation, screenshots, and canvas host URL debugging.
diagram-maker	Creates SVG/HTML and Excalidraw diagrams for concepts, architecture, flows, and whiteboards as standalone visual artifacts.
meme-maker	Searches meme templates, suggests formats, and generates local SVG/PNG or Imgflip-hosted image memes.
node-connect	Diagnoses OpenClaw node pairing failures by verifying the network route, gateway configuration, and device authentication before fixing pairing.
node-inspect-debugger	Debugs Node.js with node inspect, the Chrome DevTools Protocol, breakpoints, heap snapshots, and CPU profiles.
notion	Manages Notion pages, data sources, files, comments, and Workers via the official nt:n CLI with a curl fallback.
python-debugpy	Debugs Python with pdb, breakpoint(), post-mortem inspection, and debugpy remote attach to running processes.
spike	Run throwaway prototypes to validate feasibility, compare approaches, and report an evidence-backed verdict before real builds.
taskflow	Coordinate multi-step detached work as one durable TaskFlow job with owner context, persisted state, waits, and linked child tasks.
taskflow-inbox-triage	Example TaskFlow pattern for inbox triage with intent routing, Slack reply waits, and deferred end-of-day summaries.
video-frames	Extract single frames or quick inspection thumbnails from videos at chosen timestamps using ffmpeg.
weather	Current weather, rain and temperature checks, and forecasts from wtr.in via curl for locations and travel planning.

Part 4 - Additional Installed Skills (Not in Active Catalog)

Skill	Summary
1password	Sets up and uses the 1Password CLI (op) for sign-in, desktop app integration, and secret injection via tmux sessions.
apple-notes	Creates, views, edits, deletes, searches, moves, and exports Apple Notes from the terminal via the memo CLI on macOS.
apple-reminders	Lists, adds, edits, completes, and deletes Apple Reminders and reminder lists from the terminal using remindctl on macOS.
bear-notes	Creates, reads, searches, and manages Bear notes on macOS through the grizzly CLI with token-based operations.
blogwatcher	Monitors blogs and RSS/Atom feeds for new articles, tracks read state, and manages the watch list via the blogwatcher CLI.
blucli	Controls Bluesound and NAD audio players via the blu CLI: device discovery, playback, grouping, volume, and TuneIn radio.
camsnap	Captures snapshots, video clips, and motion events from configured RTSP/ONVIF cameras via the camsnap CLI.

Skill	Summary
clawhub	Searches, installs, updates, and publishes agent skills through the ClawHub CLI and its public skill registry.
coding-agent	Delegates large coding tasks to Codex, Claude Code, OpenCode, or Pi as background workers with explicit completion notifications.
discord	Runs Discord message-tool operations: send, read, edit, delete, react, poll, pin, thread, search, presence, and media.
eightctl	Controls Eight Sleep pods via the eightctl CLI: status, power, temperature, alarms, schedules, audio, and base.
gemini	Runs Gemini CLI headless one-shot prompts, summaries, generation, and skills, hooks, MCP, or Gemma routing management.
gh-issues	Automates issue-to-PR work: fetch GitHub issues, spawn background fix agents, open PRs, and process review comments.
gifgrep	Searches GIF providers with CLI or TUI, downloads results, and extracts stills or contact sheets from clips.
github (built-in)	GitHub CLI reference for issues, PRs, CI and check logs, comments, reviews, releases, repos, and API queries.
gog	Google Workspace CLI for Gmail, Calendar, Drive, Contacts, Sheets, and Docs with one-time OAuth setup.
goplaces	Queries Google Places for text search, place details, resolution, and reviews with human or scriptable JSON output.
healthcheck (built-in)	Audits and hardens OpenClaw hosts with read-only security checks and staged, reversible hardening recommendations.
himalaya	IMAP/SMTP mail from the shell with himalaya: list, read, search, compose, reply, forward, copy, move, delete.
imsg	iMessage and SMS CLI for listing chats, reading history, watching messages, and sending via Messages.app.
mcporter	Lists, configures, authenticates, calls, and inspects MCP servers and tools over HTTP or stdio with mcporter.
model-usage	Summarizes CodexBar local cost logs by model for Codex or Claude with current-model or full breakdowns.
nano-pdf	Applies natural-language edits to a specific page of a PDF using the nano-pdf CLI.
obsidian	Works with Obsidian vaults through the official obsidian CLI: notes, tasks, links, properties, and plugins.
openai-whisper	Transcribes audio locally with the OpenAI Whisper command line tool, requiring no API key.
openai-whisper-api	Transcribes audio through the OpenAI Audio Transcriptions API with models like gpt-4o-transcribe, diarize, and whisper-1.
openhue	Controls Philips Hue lights, rooms, and scenes through the OpenHue CLI over a local Hue Bridge.
oracle	Runs one-shot second-model reviews of code and design with selected files via the Oracle CLI, using API or browser engines.
ordercli	Checks past food delivery orders and tracks active Foodora order status, with reorder and session import support.
peekaboo	Captures and automates the macOS UI with the Peekaboo CLI: screenshots, element targeting, input, apps, and windows.
sag	Generates expressive ElevenLabs text-to-speech audio with local playback and a mac-style say command experience.
self-improving-agent	Duplicate copy of the self-improvement skill; inactive, the active copy is self-improvement.
session-logs	Searches and analyzes stored session JSONL transcripts with jq and ripgrep to recover historical conversation context.
sherpa-onnx-tts	Produces offline text-to-speech audio with the sherpa-onnx CLI and local voice models, with no cloud dependency.

Skill	Summary
skill-creator (built-in)	Authoring rules and validation helpers for creating, editing, auditing, and restructuring AgentSkills and SKILL.md files.
slack	Slack tool guidance for sending, reading, editing, and deleting messages, plus reactions, pins, emoji lists, and member info.
songsee	Generate spectrograms and audio feature-panel visualizations from audio files or streams using the songsee CLI.
sonoscli	Control Sonos speakers for discovery, status, playback, volume, grouping, favorites, queues, and Spotify search on the local network.
spotify-player	Control Spotify playback and search from the terminal using spogo (preferred) or spotify_player as the fallback CLI.
summarize	Summarize or transcribe URLs, YouTube videos, podcasts, articles, transcripts, PDFs, and local files with the summarize CLI.
things-mac	Add, update, list, search, or inspect Things 3 todos, inbox, today, projects, areas, and tags on macOS.
tmux	Control tmux sessions and panes for interactive CLIs: list, capture output, send keys, paste text, and monitor prompts.
trello	Manage Trello boards, lists, and cards with curl and jq against the Trello REST API.
voice-call	Start and manage outbound voice calls through the OpenClaw voice-call plugin with Twilio, Telnyx, Plivo, or mock providers.
wacli	Send third-party WhatsApp messages or sync and search WhatsApp history via wacli, reserved for messaging other people.
xurl	Authenticated X work with xurl: posts, replies, reads, search, DMs, media upload, followers, auth status, and raw v2 API calls.

Part 1 - Custom Workspace Skills (Active in Skill Catalog)

42 skills, alphabetical.

adapt

Location	/home/work/.openclaw/skills/adapt/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Adapts existing designs to different screens, devices, platforms, and contexts while keeping experiences consistent and native-feeling.

Overview

adapt is a user-invokable design skill for taking an interface built in one context and rethinking it for another, such as moving desktop web designs to mobile, tablet, TV, watch, print, or email. It follows a structured workflow: assess the source and target contexts, identify what will not fit or work, plan a context-specific strategy, implement responsive and touch adaptations, and verify on real devices. The skill stresses that adaptation is rethinking the experience, not merely scaling it.

Capabilities

- Assess source assumptions like screen size, input method, and connection speed
- Plan tailored strategies for mobile, tablet, desktop, print, and email targets
- Apply responsive breakpoints, container queries, and fluid sizing with clamp()
- Adapt touch targets to 44x44px minimum with thumb-friendly placement
- Verify across devices, orientations, browsers, operating systems, and input methods

Usage

- frontend-design — load first; it holds design principles and the Context Gathering Protocol
- teach-impeccable — run first if no design context exists yet
- [TARGET=<value>] [CONTEXT=<value>] — pass target platform and usage context
- clamp() — fluid sizing between minimum and maximum values
- srcset — responsive image adaptation for varied connection speeds

When to use

Use when a design must work across different screen sizes, devices, platforms, or usage contexts, or when moving between desktop, mobile, print, and email.

Requirements & notes

Requires the frontend-design skill and its Context Gathering Protocol; gather target platforms, devices, and usage contexts first. Testing on real devices is mandatory because browser DevTools emulation is helpful but not perfect.

agent-browser

Location	/home/work/.openclaw/skills/openclaw-agent-browser/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Headless browser automation CLI for navigating sites, filling forms, clicking, scraping, testing, and screenshots.

Overview

agent-browser is a headless browser automation CLI built for AI agents that need programmatic web interaction. The core workflow is: open a page, take an accessibility snapshot to discover element refs such as @e1, interact through those refs, and re-snapshot after any DOM change because refs are invalidated by navigation and dynamic content. It also supports named parallel sessions, login state persistence, PDF and screenshot capture, and optional output safety boundaries.

Capabilities

- Navigate pages, click, fill forms, select options, press keys, and scroll
- Snapshot interactive elements with stable refs for reliable targeting
- Extract text, URL, and page title from any element or the document body
- Persist login state to a JSON file and reuse it in later sessions

- Full-page screenshots, PDF export, and screenshot diffing against baselines
- Parallel browsing via named sessions with cleanup through close

Usage

- `scripts/setup.sh` — installs agent-browser and Chromium, needs Node.js
- `agent-browser open <url>` — navigate to a page
- `agent-browser snapshot -i` — list interactive elements with refs like @e1
- `agent-browser fill @e2 "text"` — type into the referenced field
- `agent-browser screenshot page.png` — capture the page image
- `agent-browser state save auth.json` — persist the session for reuse

When to use

Load this skill on requests to open a website, fill out a form, click a button, take a screenshot, scrape data, test a web app, log in to a site, or automate any browser task.

Requirements & notes

Setup requires Node.js; run `scripts/setup.sh` once to install agent-browser and Chromium. Optional environment variables `AGENT_BROWSER_CONTENT_BOUNDARIES`, `AGENT_BROWSER_ALLOWED_DOMAINS`, and `AGENT_BROWSER_MAX_OUTPUT` add output wrapping, domain allowlisting, and size caps. Always run `agent-browser close` when finished.

akshare

Location	/home/work/.openclaw/skills/akshare/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Fetch free Chinese financial data (A-shares, Hong Kong stocks, futures, funds, macro indicators) with the AKShare Python library.

Overview

AKShare is a free, open-source Python library for accessing Chinese financial market data, and this skill wraps its most useful entry points for quick financial data work. It solves the problem of getting real-time and historical market data without paid data services or API keys. The skill works by importing akshare as ak and calling fetch functions that scrape public websites and return tabular data. It is aimed at prototyping, learning quantitative strategies, and lightweight market monitoring.

Capabilities

- Fetch real-time A-share quotes and individual stock information
- Retrieve daily, weekly, and monthly K-line history with adjustments
- Load index series such as the Shanghai Composite and Shenzhen Component
- Get open fund rankings, net value history, and ETF real-time quotes
- Pull futures real-time quotes and daily futures history
- Access GDP, CPI, PMI, and money supply indicators

Usage

- `pip install akshare --upgrade` — installs or upgrades the AKShare library
- `ak.stock_zh_a_spot_em()` — fetches real-time quotes for all A-shares
- `ak.stock_individual_info_em(symbol="000001")` — gets information for one stock
- `ak.stock_zh_a_hist(symbol="000001", period="daily", start_date="20200101", end_date="20231231", adjust="qfq")` — historical K-line data with forward adjustment
- `ak.stock_zh_index_daily(symbol="sh000001")` — daily history for an index
- `time.sleep(1)` — recommended pause between calls to avoid rate limits

When to use

Use when you need Chinese market data (A-shares, Hong Kong stocks, futures, funds), macroeconomic indicators, or a free source for prototyping quantitative strategies.

Requirements & notes

Requires Python with the akshare package installed; no API key is required. AKShare scrapes public websites, so add delays between requests, expect occasional data gaps or errors, and validate critical values. Use bare 6-digit stock

codes such as "000001" for stocks and prefixed codes such as "sh000001" for indices. For guaranteed stability, high-frequency trading, or real-time tick data, prefer paid feeds such as Tushare Pro or broker APIs.

animate

Location	/home/work/.openclaw/skills/animate/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Reviews a feature and adds purposeful animations, micro-interactions, and motion effects that improve usability and provide feedback.

Overview

animate is a user-invokable skill that analyzes a feature and strategically adds motion where it improves understanding, feedback, or delight. It assesses static areas with missing feedback, jarring transitions, or unclear relationships, then plans a layered strategy around one hero moment plus feedback, transition, and delight layers. Implementation guidance covers entrance choreography, micro-interactions, state transitions, navigation flows, and precise timing and easing choices, with strict accessibility and performance rules.

Capabilities

- Identify missing feedback, jarring transitions, and unclear spatial relationships
- Plan hero moments and layered feedback, transition, and delight animations
- Implement entrance choreography with staggered 100-150ms reveal delays
- Specify durations by purpose: 100-150ms feedback up to 500-800ms entrances
- Apply GPU-friendly transform and opacity animation for smooth 60fps motion
- Respect prefers-reduced-motion with non-animated alternatives for accessibility

Usage

- frontend-design — load first for design principles and the Context Gathering Protocol
- teach-impeccable — run first if no design context exists yet
- [TARGET=<value>] — pass the feature or area to animate
- Web Animations API — programmatic control for complex interactive animations
- Framer Motion — animation implementation for React applications
- GSAP — complex animation sequences

When to use

Use when a feature feels static, lacks feedback, has jarring transitions, or needs purposeful micro-interactions and motion that enhance usability.

Requirements & notes

Requires the frontend-design skill and teach-impeccable context; additionally gather performance constraints. Animations must respect prefers-reduced-motion, avoid bounce and elastic easing curves, and never animate layout properties such as width, height, top, or left.

arrange

Location	/home/work/.openclaw/skills/arrange/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Improves layout, spacing, and visual rhythm, fixing monotonous grids and weak hierarchy to create intentional compositions.

Overview

arrange is a user-invokable skill that assesses and improves spatial design that feels monotonous, crowded, or structurally weak. It analyzes spacing consistency, visual hierarchy via the squint test, grid structure, rhythm, and density, then plans improvements around a consistent spacing system. The skill guides choices between Flexbox and Grid, breaking card-grid monotony, strengthening hierarchy with space and weight, and managing depth with semantic z-index and shadow scales.

Capabilities

- Diagnose spacing, hierarchy, grid, rhythm, and density weaknesses in layouts

- Establish semantic spacing tokens like `--space-xs` through `--space-xl`
- Create rhythm with tight 8-12px grouping and generous 48-96px separation
- Choose Flexbox for 1D layouts and Grid for 2D page structure
- Build semantic z-index and shadow scales for depth management
- Break card-grid monotony with varied sizes and mixed content

Usage

- `frontend-design` — load first for design principles and the Context Gathering Protocol
- `teach-impeccable` — run first if no design context exists yet
- `[TARGET=<value>]` — pass the layout area to improve
- `reference/spatial-design.md` — spatial design reference for grids and rhythm
- `repeat(auto-fit, minmax(280px, 1fr))` — responsive grids without explicit breakpoints

When to use

Use when layouts feel monotonous, crowded, or weak, when spacing values are arbitrary, or when identical card grids and flat hierarchy make interfaces feel generic.

Requirements & notes

Requires the `frontend-design` skill; consult its spatial design reference at `reference/spatial-design.md` for detailed grid and rhythm guidance. A consistent spacing scale (framework scale, rem-based tokens, or custom system) must exist or be defined as part of the work.

audit

Location	/home/work/.openclaw/skills/audit/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Runs comprehensive interface quality checks across accessibility, performance, theming, responsiveness, and anti-patterns, producing a prioritized audit report.

Overview

`audit` is a user-invokable skill that performs systematic quality checks and generates a comprehensive report with prioritized issues and actionable recommendations. It scans five dimensions: accessibility, performance, theming, responsive design, and anti-patterns such as AI-generated design tells. The skill deliberately documents issues instead of fixing them, mapping each finding to severity, impact, violated standards, and a suggested fix command so other commands can address problems afterward.

Capabilities

- Check contrast ratios, ARIA, keyboard navigation, semantics, alt text, and forms
- Detect layout thrashing, expensive animations, and bundle size problems
- Find hard-coded colors, broken dark mode, and inconsistent design tokens
- Verify 44x44px touch targets, breakpoints, and text scaling resilience
- Grade findings Critical, High, Medium, and Low with impact explanations
- Identify systemic patterns and positive findings worth maintaining

Usage

- `frontend-design` — supplies design principles and anti-pattern checklists to audit against
- `[AREA=<value>]` — pass the audit area or scope
- `/normalize` — suggested fix command for theming and design system issues
- `/optimize` — suggested fix command for performance issues
- `/harden` — suggested fix command for edge-case resilience gaps

When to use

Use for comprehensive interface quality reviews, pre-release audits, or when a detailed report of prioritized issues is needed before fixing with other commands.

Requirements & notes

Requires the `frontend-design` skill for design principles and anti-pattern definitions. This is an audit, not a fix; route remediation to other installed commands such as `/normalize`, `/optimize`, and `/harden`. Only suggest commands you are

sure exist, and never invent commands.

bolder

Location	/home/work/.openclaw/skills/bolder/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Amplifies safe or boring designs with stronger typography, color, spatial drama, and motion while preserving usability and coherence.

Overview

bolder is a user-invokable skill that increases visual impact and personality in designs that feel too safe, generic, or visually underwhelming. It diagnoses weakness sources like generic fonts, timid scale, and low contrast, then plans amplification around one focal point, a chosen personality direction, and a risk budget. The skill amplifies typography, color, space, effects, motion, and composition while explicitly warning against AI slop shortcuts such as glassmorphism and purple-blue gradients.

Capabilities

- Replace generic fonts with distinctive choices and 3x-5x scale jumps
- Pair extreme weights, such as 900 with 200, for weight contrast
- Introduce dominant color strategies and tinted neutrals instead of pure gray
- Create spatial drama with asymmetry, grid breaks, and 100-200px gaps
- Add motion using ease-out-quart, quint, or expo easing curves

Usage

- `frontend-design` — load first; contains design principles and all DON'T guidelines
- `teach-impeccable` — run first if no design context exists yet
- `[TARGET=<value>]` — pass the design area to amplify

When to use

Use when a design is too safe, generic, or visually underwhelming, or when stakeholders ask for more impact, personality, or memorability without chaos.

Requirements & notes

Requires the frontend-design skill and its Context Gathering Protocol; run teach-impeccable first if no design context exists. Bolder does not mean chaotic or garish: designs must stay readable, accessible, and performant, and all anti-pattern guidelines should be reviewed before amplification.

clarify

Location	/home/work/.openclaw/skills/clarify/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Improves unclear UX copy, error messages, microcopy, labels, and instructions so interfaces are easier to understand and use.

Overview

clarify is a user-invokable skill that identifies and rewrites unclear, confusing, or ineffective interface text to make products easier to understand. It diagnoses clarity problems like jargon, ambiguity, passive voice, and missing context, then improves copy across error messages, form labels, buttons, help text, empty states, success and loading states, confirmation dialogs, and navigation. Every rewrite follows six rules: be specific, concise, active, human, helpful, and consistent.

Capabilities

- Diagnose jargon, ambiguity, passive voice, and tone mismatches in copy
- Rewrite error messages that explain problems and suggest concrete fixes
- Create specific button labels like "Create account" instead of "Submit"
- Improve empty, loading, and success states with actionable guidance
- Verify copy for comprehension, actionability, brevity, consistency, and tone

Usage

- `frontend-design` — load first for design principles and the Context Gathering Protocol
- `teach-impeccable` — run first if no design context exists yet
- `[TARGET=<value>]` — pass the interface text area to improve

When to use

Use when UX copy, error messages, microcopy, labels, or instructions are unclear, wordy, jargon-heavy, or inconsistent with the product voice.

Requirements & notes

Requires the `frontend-design` skill and `teach-impeccable` context; additionally gather the audience's technical level and the user's mental state in context. Character limits, space constraints, brand voice, and localization considerations should be known before rewriting copy.

clawhub-cli

Location	/home/work/.openclaw/skills/clawhub-cli/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Agent playbook for the ClawHub CLI: search, install, list, update, publish, and sync agent skills from clawhub.ai.

Overview

This skill is an execution playbook for managing agent skills with the npm-installed ClawHub CLI against `clawhub.ai`. It covers discovering skills by keyword, installing them into a workspace, listing installed skills, updating to latest or pinned versions, and publishing or syncing local skill folders. Its goal is the smallest set of correct commands while avoiding destructive actions and never publishing secrets. Guardrails include never inventing CLI flags, preferring dry-run for bulk work, and keeping SemVer versions valid.

Capabilities

- Search ClawHub for skills using keyword queries
- Install skills by slug with an optional pinned version
- List installed skills to confirm lockfile state
- Update one skill or all skills to latest or pinned versions
- Publish a single local skill folder with slug, name, version, tags
- Sync many local skill folders in bulk with changelog and bump options

Usage

- `npm i -g clawhub` — installs the ClawHub CLI globally
- `clawhub login` — authenticates before publishing or syncing
- `clawhub search "your query"` — searches skills by keywords
- `clawhub install postgres-backup-tools` — installs a skill by slug
- `clawhub update --all` — updates every installed skill at once
- `clawhub publish ./skills/my-skill --slug my-skill --name "My Skill" --version 0.1.0 --tags latest` — publishes one skill folder

When to use

Use when the user asks to discover, install, list, update, publish, or sync agent skills through the ClawHub registry, or when shipping local skill folders.

Requirements & notes

Requires Node.js with npm or pnpm and a ClawHub account; run `clawhub login` before publish or sync, and check `clawhub install --help` since destination paths vary by CLI version. Never publish `.env` files, tokens, credentials, private keys, or proprietary data, and prefer `--dry-run` for bulk operations when the CLI supports it. Bundled references cover commands, publishing checklists, security, troubleshooting, SemVer bumps, folder structure, and Windows usage.

code-generator

Location	/home/work/.openclaw/skills/code-generator/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Multi-language code scaffolding: generate functions, classes, API endpoints, CRUD code, tests, refactoring advice, and language conversion guides.

Overview

Code Generator is a multi-language scaffolding skill published by BytesAgain: describe what you need and get working, commented code back. It covers function generation, class design with constructors and methods, RESTful API endpoints, full Create/Read/Update/Delete operations, unit test generation, refactoring suggestions with examples, conversion guides between languages, and project boilerplate skeletons. Everything is driven through a single bash entry script that takes a command name and a natural-language description. Supported languages are Python, JavaScript, TypeScript, Go, Java, Rust, PHP, Ruby, C#, and Shell.

Capabilities

- Generate complete annotated functions from plain descriptions
- Scaffold classes with constructors and methods for OOP design
- Produce RESTful routes and handlers for API endpoints
- Build full Create/Read/Update/Delete operation code
- Auto-generate unit tests with assertions
- Give refactoring suggestions and cross-language conversion guides

Usage

- `bash scripts/codegen.sh <command> <description>` — main script for every generation command
- `code-generator help` — lists all available commands and modes

When to use

Use when code is needed quickly: functions, classes, API endpoints, CRUD modules, test code, refactoring guidance, boilerplate projects, or converting code between languages.

Requirements & notes

Requires bash version 4 or newer and python3 with the standard library only, so no third-party packages or API keys are needed. Generation modes are function, class, api, crud, test, refactor, convert, and boilerplate. Every output includes a language label, a suggested filename, complete runnable code, inline comments, and a usage example. The skill ships one main script at `scripts/codegen.sh`; the publisher collects feedback and feature requests through its homepage.

colorize

Location	/home/work/.openclaw/skills/colorize/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Adds strategic color to monochromatic designs, using semantic, accent, and decorative color to improve meaning, hierarchy, and warmth.

Overview

colorize is a user-invokable skill that introduces color into designs that are too monochromatic, gray, or lacking visual warmth and personality. It identifies where color adds semantic meaning, hierarchy, categorization, or delight, then applies a planned palette of two to four colors beyond neutrals with a dominant 60/30/10 balance. The skill covers semantic states, accent applications, tinted backgrounds, data visualization, borders, typography color, and decorative elements, with strict accessibility and cohesion rules.

Capabilities

- Map semantic colors to success, error, warning, info, and neutral states
- Replace pure gray with warm or cool tinted neutrals using OKLCH
- Apply dominant, secondary, and accent colors in a 60/30/10 balance
- Color charts, heatmaps, status badges, progress bars, and focus rings
- Maintain WCAG contrast: 4.5:1 for text and 3:1 for UI components

Usage

- frontend-design — load first for design principles and the Context Gathering Protocol
- teach-impeccable — run first if no design context exists yet
- [TARGET=<value>] — pass the design area to colorize
- oklch(97% 0.01 60) — perceptually uniform warm neutral tint example

When to use

Use when interfaces are too monochromatic or gray, lack visual warmth and personality, or need strategic color for meaning, hierarchy, or engagement.

Requirements & notes

Requires the frontend-design skill and teach-impeccable context; gather existing brand colors first. Never rely on color alone as an indicator, verify red and green combinations for color-blind users, and keep color meanings consistent throughout the product.

critique

Location	/home/work/.openclaw/skills/critique/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Evaluates design effectiveness from a UX perspective, covering hierarchy, architecture, emotional resonance, and AI slop detection with actionable feedback.

Overview

critique is a user-invokable skill that conducts holistic design critiques the way a design director would, judging whether an interface works as a designed experience and not just technically. It evaluates ten dimensions, starting with AI slop detection as the most critical check, then visual hierarchy, information architecture, emotional resonance, discoverability, composition, typography, color purpose, states, and microcopy. Feedback is structured into an anti-patterns verdict, overall impression, what is working, priority issues, minor observations, and provocative questions.

Capabilities

- Detect AI-generated design tells against frontend-design anti-pattern guidelines
- Evaluate visual hierarchy and whether the primary action is obvious in 2 seconds
- Assess information architecture, discoverability, affordance, and emotional resonance
- Review typography, color purpose, edge-case states, and microcopy voice
- Produce priority issues with concrete fixes mapped to installed commands

Usage

- frontend-design — load first; its DON'T guidelines are the AI slop checklist
- teach-impeccable — run first if no design context exists yet
- [AREA=<value>] — pass the interface area to critique
- /bolder — suggested command for increasing visual impact
- /clarify — suggested command for microcopy and voice fixes

When to use

Use when a design needs honest evaluation of its UX effectiveness, an AI slop verdict, or a director-style critique with prioritized and actionable feedback.

Requirements & notes

Requires the frontend-design skill and its Context Gathering Protocol; additionally gather what the interface is trying to accomplish. Only reference installed commands you are sure exist. Feedback should be direct and specific: state what is wrong, why it matters, and how to fix it.

Data Analysis

Location	/home/work/.openclaw/skills/data-analysis/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Decision-first data analysis and visualization methodology with metric contracts, chart guidance, statistical rigor checks, and decision brief formats.

Overview

Data Analysis is a judgment-focused skill for analyzing, explaining, and visualizing data from SQL, spreadsheets, notebooks, dashboards, exports, or ad hoc tables. Its core principle is that analysis without a decision is just arithmetic, so every job starts from the decision it supports and what would change the analyst's mind. It enforces locking metric contracts before calculation, separating extraction from interpretation, and briefing every result in decision format with answer, evidence, confidence, caveats, and next action. The skill needs no setup state and ships small reference guides that are loaded only when relevant.

Capabilities

- Define metric contracts: entity, grain, numerator, denominator, filters
- Design fair comparisons and run statistical rigor checklists
- Match chart selection to trend, comparison, or funnel questions
- Catch failure modes like aggregation noise and post-hoc cuts
- Produce decision briefs with confidence ranges for stakeholders
- Escalate or downgrade claims the data cannot support

Usage

- `metric-contracts.md` — KPI definitions and caveats
- `chart-selection.md` — visual choice and chart anti-patterns
- `decision-briefs.md` — decision-ready stakeholder output formats
- `pitfalls.md` — failure modes to catch early
- `techniques.md` — method selection by question type
- `clawhub install <slug>` — installs related skills after user confirmation

When to use

Use for KPI debugging, experiment readouts, funnel or cohort analysis, anomaly reviews, churn or A/B questions, executive reporting, and quality checks on metrics or query logic.

Requirements & notes

No local folders, persistent memory, or setup are required, and the skill makes no external network requests or sends data externally. Statistical caveats apply: sufficient sample size, fair comparison groups, multiple-comparison risk, meaningful effect sizes, and quantified uncertainty. Related skills such as `sql`, `csv`, `dashboard`, `report`, and `business-intelligence` can be installed on request.

delight

Location	/home/work/.openclaw/skills/delight/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Adds moments of joy, personality, and unexpected polish through micro-interactions, copy, illustrations, sound, and easter eggs.

Overview

delight is a user-invokable skill that finds opportunities to add moments of joy and personality, transforming functional interfaces into delightful experiences. It targets success, empty, loading, and error states, achievements, and everyday interactions, matching the delight strategy to brand personality and emotional context. Techniques span micro-interactions and animation, playful copy, custom illustrations, satisfying interactions, subtle sound design, easter eggs, and celebration moments, all governed by rules that delight must never block or delay core functionality.

Capabilities

- Design success celebrations with confetti, checkmarks, and milestone recognition
- Write playful error, empty-state, and loading copy specific to the product

- Add hover surprises, custom cursors, and animated icons for personality
- Create easter eggs like hidden shortcuts and alt text jokes
- Implement with Framer Motion, GSAP, Lottie, and Canvas confetti

Usage

- `frontend-design` — load first for design principles and the Context Gathering Protocol
- `teach-impeccable` — run first if no design context exists yet
- `[TARGET=<value>]` — pass the feature or area to delight
- Framer Motion — React animation library for micro-interactions
- Howler.js — audio management for subtle sound cues
- React Spring — spring physics for satisfying toggles and transitions

When to use

Use when interfaces are purely functional and joyless, when success, empty, or loading states need personality, or when stakeholders want memorable, shareable moments.

Requirements & notes

Requires the `frontend-design` skill and `teach-impeccable` context; determine what delight fits the domain, playful versus professional. Keep delight skippable and under one second, respect reduced motion and system sound settings, and avoid clichéd AI loading copy.

distill

Location	/home/work/.openclaw/skills/distill/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Strips designs to their essence by removing unnecessary complexity across architecture, visuals, layout, interaction, content, and code.

Overview

`distill` is a user-invokable skill that removes unnecessary complexity from designs, revealing essential elements and creating clarity through ruthless simplification. It identifies complexity sources such as competing elements, excessive variation, information overload, visual noise, and feature creep, then finds the essence: the one primary user goal and the 20% of elements delivering 80% of value. Simplification runs across information architecture, visuals, layout, interaction, content, and code, with progressive disclosure and consolidation as key tools.

Capabilities

- Reduce scope to one primary action and few secondary actions
- Shrink color palettes to 1-2 colors and typography to 3-4 sizes
- Replace complex grids with linear flows and generous whitespace
- Cut copy in half repeatedly and remove jargon and redundancy
- Remove dead CSS, flatten component trees, and reduce variants

Usage

- `frontend-design` — load first for design principles and the Context Gathering Protocol
- `teach-impeccable` — run first if no design context exists yet
- `[TARGET=<value>]` — pass the design area to simplify

When to use

Use when designs feel cluttered or complex, when too many elements compete for attention, or when interfaces need ruthless simplification without losing necessary functionality.

Requirements & notes

Requires the `frontend-design` skill and `teach-impeccable` context. Simplification must never remove necessary functionality or accessibility support like labels and ARIA. If features or options are removed, document why, consider alternative access points, and note user feedback to monitor.

Excel / XLSX

Location	/home/work/.openclaw/skills/excel-xlsx/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Create, inspect, and edit Excel workbooks with reliable formulas, dates, types, formatting, recalculation, and template preservation.

Overview

Excel / XLSX is a reliability-focused skill for Microsoft Excel workbooks and spreadsheet files, used when formulas, dates, formatting, merged cells, workbook structure, or cross-platform behavior matter. It teaches choosing the right workflow per job: pandas for analysis and reshaping, openpyxl when formulas, styles, sheets, comments, merged cells, or workbook preservation are involved. It covers Excel date serial quirks, protecting data types from silent corruption, keeping live formulas instead of hardcoded results, preserving existing templates, and recalculating before delivery. It also addresses scaling workflows to large files with streaming or chunked reads.

Capabilities

- Choose between pandas and openpyxl by job requirements
- Write live formulas with correct absolute and relative references
- Handle date serial numbers and the 1900/1904 date systems
- Store identifiers, phone numbers, and ZIP codes as text
- Preserve templates: sheet order, widths, freezes, filters, validations
- Recalculate and eliminate formula errors before delivery

Usage

- pandas — analysis, reshaping, and CSV-like tabular tasks
- openpyxl — formulas, styles, sheets, comments, merged cells, preservation
- clawhub install <slug> — installs related skills such as csv or data
- clawhub star excel-xlsx — feedback when the skill proved useful
- clawhub sync — stay updated with newer versions

When to use

Use when the main artifact is an Excel workbook or spreadsheet file (.xlsx, .xlsm, .xls, .csv, .tsv) and formulas, dates, formatting, or structure must stay reliable after edits.

Requirements & notes

Requires Python with pandas and openpyxl available. Note that openpyxl preserves formulas but never calculates them, and cached values can be stale, so verify there is no #REF!, #DIV/0!, #VALUE!, #NAME?, or circular-reference fallout before delivery. Legacy formats and other viewers matter: .xlsm can carry macros, .xls is a tighter legacy format, Google Sheets and LibreOffice may reinterpret dates or styling, and newer functions such as FILTER, XLOOKUP, SORT, or SEQUENCE can degrade in older viewers.

extract

Location	/home/work/.openclaw/skills/extract/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Extracts reusable components, design tokens, and patterns into the design system, then migrates existing usages to shared versions.

Overview

extract is a user-invokable skill that identifies reusable patterns, components, and design tokens and consolidates them into the design system for systematic reuse. It discovers the existing design system or shared UI directory, finds repeated components, hard-coded values, and inconsistent variations, and assesses extraction value based on reuse count and consistency benefit. The skill then builds enriched components with accessible props APIs, creates semantic tokens, documents patterns, migrates existing instances to the shared versions, and deletes dead code.

Capabilities

- Locate design system structure, naming conventions, tokens, and documentation patterns
- Identify repeated components and hard-coded colors, spacing, and shadows

- Build components with clear props APIs, variants, and built-in accessibility
- Create primitive and semantic design tokens with usage documentation
- Migrate all instances to shared versions and delete old implementations

Usage

- `grep` — search for "design system", "ui", and "components" directories
- `[TARGET=<value>]` — pass the target area to scan for extraction opportunities
- `Storybook` — update the component catalog with newly extracted components
- `TypeScript` — required prop types and documentation for extracted components

When to use

Use when the same UI patterns appear repeatedly, hard-coded values should become design tokens, or multiple inconsistent implementations of one concept exist across the codebase.

Requirements & notes

If no design system exists, ask before creating one and understand the preferred location and structure first. Extract only what is used three or more times or likely to be reused, keep TypeScript types and prop documentation complete, and follow existing design system conventions.

find-skills

Location	/home/work/.openclaw/skills/find-skills/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Highest-priority skill discovery flow: search skillhub first, fall back to clawhub, present options, and install skills.

Overview

Find Skills is the mandatory entry point whenever a user wants to discover or install skills, including requests phrased in Chinese (the source triggers on the Chinese word for skill and related find/install phrases). It defines a fixed workflow: understand the need as a domain plus a specific task, search for candidate skills, present options with names, descriptions, sources, and install commands, then offer to install. For Chinese users and CN networks it applies an optimization policy that prefers skillhub for speed and compliance and falls back to clawhub. When a fallback happens, the skill must state it clearly to the user.

Capabilities

- Interpret skill requests into a domain and concrete task
- Search skillhub first as the CN-optimized preferred source
- Fall back to clawhub when skillhub is unavailable or empty
- Present candidates with name, purpose, and source used
- Install chosen skills with a pre-install risk summary
- Suggest custom local skills when no match exists

Usage

- `skillhub search [query]` — primary search command for candidates
- `clawhub search [query]` — fallback search when skillhub fails
- `skillhub install <slug>` — preferred install when the match is from skillhub
- `clawhub install <slug>` — install when no skillhub candidate exists

When to use

Use the moment a user asks to find or install skills or whether a skill exists for some capability; this skill must take priority over generic coding or answering.

Requirements & notes

Requires access to the `skillhub` and `clawhub` command-line tools. Before any install, summarize the source, version, and notable risk signals for the user. If no relevant skill is found, acknowledge it, offer to handle the task with general capabilities, and suggest building a custom local skill for recurring needs.

frontend-design

Location	/home/work/.openclaw/skills/frontend-design/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Creates distinctive, production-grade frontend interfaces with bold, intentional aesthetics that avoid generic AI-generated design patterns.

Overview

frontend-design is a methodology skill guiding the creation of web components, pages, posters, and applications with exceptional aesthetic quality. It enforces a mandatory context gathering protocol before any design work, requiring confirmed target audience, use cases, and brand personality. It then directs a bold, intentional aesthetic direction across typography, color, layout, motion, interaction, responsive behavior, and UX writing. A closing "AI Slop Test" quality gate rejects interfaces that are visibly machine-generated.

Capabilities

- Enforces a design context protocol before any design work begins
- Provides DO and DON'T guidelines for typography, color, and layout
- Recommends modern CSS color functions like `oklch`, `color-mix`, `light-dark`
- Defines motion rules using transform and opacity plus exponential easing
- Includes companion reference files for typography, color, motion, and interaction
- Applies the AI Slop Test to reject generic AI aesthetics
- Covers responsive design with container queries and UX writing rules

Usage

- `.impeccable.md` — project design context file checked before design work
- `teach-impeccable` — must run when no design context exists yet
- `clamp()` — fluid typography and spacing sizing recommended throughout
- `@container` — container queries for component-level responsiveness
- `reference/typography.md` — companion reference for scales and font pairing

When to use

Load this skill whenever the user asks to build web components, pages, artifacts, posters, or applications. It is also the mandatory preparation step for several sibling design skills.

Requirements & notes

Design context (target audience, use cases, brand personality) must be confirmed before work starts; if missing, run `teach-impeccable` first. Avoid overused fonts such as Inter, Roboto, Arial, and Open Sans, and never converge on the same choices across generations.

github

Location	/home/work/.openclaw/skills/github/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Compact GitHub skill using the gh CLI for pull requests, CI runs, and advanced API queries with JSON and jq filtering.

Overview

This custom skill covers GitHub interaction through the `gh` command-line tool in a compact reference form. It handles pull request CI checks, listing recent workflow runs, inspecting runs to find failed steps, and reading logs only for those failures. For data that other subcommands do not expose, it uses `gh api` against the GitHub REST API with `jq` field selection. Most commands accept `--json` for structured output and `--jq` for filtering, and `--repo owner/repo` should always be specified when not inside a git directory.

Capabilities

- Check CI status for a specific pull request
- List recent workflow runs with a result limit
- Inspect a run and identify which steps failed
- Show logs for failed steps only
- Query the REST API with selected fields via `jq`

- Filter structured JSON output from any subcommand

Usage

- `gh pr checks 55 --repo owner/repo` — checks CI status on a pull request
- `gh run list --repo owner/repo --limit 10` — lists recent workflow runs
- `gh run view <run-id> --repo owner/repo` — shows a run and its steps
- `gh run view <run-id> --repo owner/repo --log-failed` — logs of failed steps only
- `gh api repos/owner/repo/pulls/55 --jq '.title, .state, .user.login'` — API query with fields
- `gh issue list --repo owner/repo --json number,title --jq '.[ ] | "\(.number): \(.title)"'` — structured issue listing

When to use

Use when working with GitHub issues, pull requests, CI or workflow runs, or custom REST queries through the gh CLI.

Requirements & notes

Requires the GitHub CLI (gh) installed and authenticated to GitHub. A different, fuller built-in github skill also exists in the OpenClaw package and is covered elsewhere in this catalog; this entry documents the compact custom workspace skill. Always pass `--repo owner/repo` when the working directory is not a git repository.

harden

Location	/home/work/.openclaw/skills/harden/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Strengthens interfaces against edge cases, errors, internationalization issues, and real-world usage so designs survive production.

Overview

harden is a systematic resilience pass for frontend interfaces before production release. It starts with an assessment checklist covering extreme inputs, error scenarios, and internationalization, since designs that only work with perfect data are not production-ready. It then works through hardening dimensions such as text overflow and wrapping, RTL and character set support, error handling, empty and loading states, input validation, accessibility resilience, and performance resilience. Each dimension includes concrete CSS and JavaScript patterns plus manual and automated testing strategies.

Capabilities

- Tests extreme inputs: long text, emoji, RTL scripts, huge numbers, empty data
- Handles network failures, API status codes, validation errors, and rate limits
- Adds i18n support with logical CSS properties and Intl formatting
- Provides CSS overflow patterns: truncation, line clamping, word wrapping
- Defines empty, loading, permission, and error states with recovery paths
- Covers accessibility resilience: keyboard nav, ARIA, reduced motion, contrast
- Supplies manual and automated testing strategies including axe and WAVE

Usage

- `new Intl.DateTimeFormat('de-DE').format(date)` — locale-correct date formatting
- `new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' })` — currency formatting
- `debounce(handleSearch, 300)` — debounce pattern for search input handling
- `@media (prefers-reduced-motion: reduce)` — respect motion-sensitive users
- `margin-inline-start: 1rem` — logical property for RTL-friendly layout

When to use

Load when hardening an interface before production, when designs break on real data, or when preparing for global audiences and failure conditions.

Requirements & notes

Working knowledge of CSS, JavaScript, and ARIA accessibility practices is assumed. Verification uses accessibility tools such as axe and WAVE, and testing should cover old browsers, slow connections, and screen readers.

healthcheck

Location	/home/work/.openclaw/skills/healthcheck/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Track water intake and sleep in a JSON file using self-contained Node.js one-liner scripts for logging and stats.

Overview

Healthcheck is a simple personal health tracker for water intake and sleep, storing everything in a JSON file at `{baseDir}/health-data.json`. Each action is a self-contained inline Node.js script that reads the file, appends a record with an ISO8601 timestamp, and writes it back, creating the file automatically if it is missing. Trigger phrases (Vietnamese in the source, given here in translation) cover drinking X cups of water, going to sleep, waking up, and requesting statistics; wake records also compute hours slept since the last sleep entry. The skill can also update or delete the most recent water record.

Capabilities

- Log water intake with cup counts and ISO8601 timestamps
- Record sleep and wake events as an append-only event list
- Compute hours slept between the last sleep and wake events
- Show totals: water records, sleep records, and cups consumed today
- Update the cup count of the most recent water entry
- Delete the most recent water entry

Usage

- `node -e "const fs=require('fs');const f='{baseDir}/health-data.json';let d={water:[],sleep:[]};try{d=JSON.parse(fs.readFileSync(f))}catch(e){}d.water.push({time:new Date().toISOString(),cups:CUPS});fs.writeFileSync(f,JSON.stringify(d));console.log('Da ghi: '+CUPS+' coc')"` — adds a water record; replace CUPS with the cup count
- `node -e "const fs=require('fs');const f='{baseDir}/health-data.json';let d=JSON.parse(fs.readFileSync(f));d.water[d.water.length-1].cups=NEW_CUPS;fs.writeFileSync(f,JSON.stringify(d));console.log('Updated')"` — updates the last water entry
- `node -e "const fs=require('fs');const f='{baseDir}/health-data.json';let d=JSON.parse(fs.readFileSync(f));d.water.pop();fs.writeFileSync(f,JSON.stringify(d));console.log('Deleted')"` — deletes the last water entry
- `node -e` — sleep, wake, and statistics actions follow the same inline-script pattern

When to use

Use when the user logs water or sleep, wakes up, asks for health statistics, or wants to correct or remove the latest water record.

Requirements & notes

Requires Node.js only; all modules are built into Node and no packages or API keys are needed. The data file is created on first write and all timestamps are ISO8601. Note that a different built-in skill also named "healthcheck" (a host security audit) exists in the OpenClaw package; this entry is the custom water/sleep tracker.

humanizer

Location	/home/work/.openclaw/skills/humanizer/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Removes AI writing patterns from text using Wikipedia's signs-of-AI-writing guide, then adds genuine voice and personality.

Overview

humanizer is a writing editor skill that detects and rewrites 24 documented patterns of AI-generated text. It covers content patterns like inflated significance, promotional language, and superficial -ing analyses; language patterns like AI vocabulary and copula avoidance; style patterns like em dash overuse, boldface, and emoji decoration; and communication artifacts like sycophantic tone and knowledge-cutoff disclaimers. Beyond removing patterns, it instructs the editor to add soul: opinions, varied rhythm, acknowledged complexity, and specific feelings. Rewrites preserve meaning and match the intended tone.

Capabilities

- Detects 24 AI writing patterns with before-and-after examples
- Removes inflated symbolism, promotional language, vague attributions
- Fixes vocabulary tics like delve, tapestry, vibrant, underscore
- Eliminates style artifacts: em dash overuse, boldface, emoji, curly quotes
- Adds voice through opinions, rhythm variation, and first person
- Rewrites while preserving meaning and matching the intended tone

Usage

- `Read` — read input text for pattern analysis (allowed tool)
- `Grep` — search for repeated AI vocabulary patterns (allowed tool)
- `Write` — write the humanized rewrite (allowed tool)
- `Edit` — apply targeted fixes to problematic sections (allowed tool)
- `AskUserQuestion` — clarify tone or voice requirements (allowed tool)

When to use

Load when editing or reviewing text to make it sound more natural and human-written, or when removing telltale signs of AI writing from drafts.

Requirements & notes

No special program requirements beyond the listed editing tools. The pattern taxonomy derives from Wikipedia's "Signs of AI writing" page, maintained by WikiProject AI Cleanup and based on thousands of observed cases.

mimo-omni

Location	/home/work/.openclaw/skills/mimo-omni/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Analyzes images, video, and audio with Xiaomi MiMo multimodal models for description, OCR, transcription, and understanding.

Overview

mimo-omni calls the Xiaomi MiMo multimodal models through two interchangeable CLI tools, `mimo_api.sh` (bash/curl) and `mimo_api.py` (Python), which accept image, video, and audio input. The bash `mimo_api.sh` form is preferred and the Python version is only a fallback when bash or curl is unavailable. The question argument accepts any natural language, so the user's original query is passed through unchanged. The scripts parse the raw JSON response automatically: stdout carries the model's plain-text answer and stderr carries debug timing and token usage.

Capabilities

- Image description, object detection, OCR, and chart analysis from local files or URLs
- Video understanding with adjustable sampling rate and resolution modes
- Audio transcription with speaker distinction and sound description
- Joint video and audio analysis through the mixed mode
- Multi-image comparison through the images subcommand
- Token, timeout, fps, and resolution controls for long or dense media

Usage

- `bash mimo_api.sh image /path/to/photo.jpg "<question>"` — pass the user's question verbatim as prompt
- `bash mimo_api.sh video /path/to/video.mp4 "<question>" --fps 1` — sample one frame per second
- `bash mimo_api.sh audio /path/to/audio.wav "<question>"` — transcribe or describe audio content
- `bash mimo_api.sh images /path/to/img1.jpg /path/to/img2.jpg --question "Compare these two images"` — multi-image comparison
- `bash mimo_api.sh mixed --video /path/to/video.mp4 --audio /path/to/audio.mp3 "Describe the video and transcribe the audio"` — combined analysis
- `python mimo_api.py image /path/to/image.png "What objects are in the image?"` — Python fallback tool

When to use

Load this skill when the user sends image, video, or audio attachments or asks for visual description, OCR, object detection, scene understanding, video analysis, or audio transcription.

Requirements & notes

Local files are base64-encoded and the API caps base64 data at 10 MB, so large videos should be passed as URLs rather than paths. For videos longer than two minutes add `--timeout 600`; on repeated timeouts lower `--fps` first, then switch to `--resolution default`, then suggest clipping the video.

mimo-tts-voice-clone

Location	/home/work/.openclaw/skills/mimo-tts-voice-clone/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Clones a voice timbre from a short reference recording and synthesizes styled speech as a standard WAV file.

Overview

mimo-tts-voice-clone calls the MiMo-V2.5-TTS-VoiceClone model to replicate a target timbre from a few seconds of clean single-speaker reference audio, with no training or annotation required. The cloned voice then speaks the requested text, and free-form natural-language style or emotion instructions can be layered on top while the timbre stays fixed. Cross-language cloning works for both Chinese and English without losing timbre fidelity. The script writes a standard RIFF WAV file at 24 kHz, 16-bit PCM, mono.

Capabilities

- High-fidelity timbre cloning from 5 to 15 seconds of WAV or MP3 reference audio
- Free-form natural-language style and emotion control over the cloned voice
- Cross-language cloning between Chinese and English reference and output
- TTS text normalization guidance for numbers, symbols, and markdown residue
- Automatic RIFF header repair when the API returns raw PCM
- Distinct exit codes for missing arguments, API errors, and empty output

Usage

- `bash {baseDir}/scripts/voice_clone.sh <reference_audio> <text> <output_path> [style] — full command signature`
- `bash {baseDir}/scripts/voice_clone.sh ./my_voice.wav "The wind blows softly, carrying the scent of distant flowers." output.wav — minimal clone and speak`
- `bash {baseDir}/scripts/voice_clone.sh ./en_ref.wav "Ignore the sirens, and just breathe with me. In, and out." out_en.wav — English reference to English output`
- `bash {baseDir}/scripts/voice_clone.sh ./ref.mp3 "Congratulations on winning first prize!" congrats.wav "happy and excited" — style adds delivery only`
- `ffmpeg -i /tmp/tmp.wav out.mp3 — convert the WAV output to MP3`

When to use

Load this skill when the user supplies a reference recording and asks to speak, read, or narrate text in that voice, or to clone or imitate a recording's timbre.

Requirements & notes

Requires `jq`, `curl`, and `python3`, plus a MiMo API key via `MIMO_API_KEY` or `openclaw.json` auto-resolution. Reference audio must be clean single-speaker WAV or MP3 (recommended 5-15 s); the base64 payload is capped at 10 MB. Environment overrides: `MIMO_API_BASE_URL` and `MIMO_VOICECLONE_MODEL`.

mimo-tts-voice-design

Location	/home/work/.openclaw/skills/mimo-tts-voice-design/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Generates a brand-new voice timbre from a one-line natural-language description and synthesizes speech as WAV.

Overview

mimo-tts-voice-design calls the MiMo-V2.5-TTS-VoiceDesign model to create an entirely new timbre from a single natural-language description, with no reference audio needed. It fits requests where the user describes the voice they

want, such as age, gender, accent, temperament, or recording quality, and asks for text to be read in it. Descriptions can be specific, vague, or even contradictory and the model handles them well. The script writes a standard RIFF WAV file at 24 kHz, 16-bit PCM, mono.

Capabilities

- New timbre generation from free-form descriptions in Chinese or English
- Dimension guidance for age, gender, accent, texture, tempo, and recording style
- TTS text normalization rules for numbers, symbols, and formatting residue
- Multilingual design: description and text may be in different languages
- Automatic RIFF header repair when the API returns raw PCM
- Distinct exit codes for missing arguments, API errors, and empty output

Usage

- `bash {baseDir}/scripts/voice_design.sh <voice_description> <text> <output_path>` — full command signature
- `bash {baseDir}/scripts/voice_design.sh "Heavy Russian accent, gruff middle-aged male, blunt and matter-of-fact." "You want my opinion? Fine. This plan will not work." russian.wav` — designed voice, no reference
- `bash {baseDir}/scripts/voice_design.sh "Young male voice, bright and clean" "Hello" /tmp/tmp.wav && ffmpeg -i /tmp/tmp.wav out.mp3` — design then convert to MP3

When to use

Load this skill when the user describes a desired voice (accent, age, gender, temperament, recording feel) and asks for text to be read in it, especially when no reference recording is available.

Requirements & notes

Requires `jq`, `curl`, and `python3` plus a MiMo API key via `MIMO_API_KEY` or `openclaw.json` auto-resolution. The output file's parent directory must already exist. For replicating an existing recording use `mimo-tts-voice-clone`; for preset-voice synthesis use `mimo-tts-wav`.

mimo-tts-wav

Location	/home/work/.openclaw/skills/mimo-tts-wav/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Converts text to speech or song with the MiMo TTS API and saves a standard WAV file.

Overview

`mimo-tts-wav` wraps the MiMo TTS API (model `mimo-v2.5-tts`) to turn text into spoken or sung audio saved as a standard WAV file (24 kHz, 16-bit PCM, mono). It applies whenever the user asks the agent to speak, read text aloud, or sing, even with no output path given; the default is `output.wav` in the working directory. Voices come from the preset `mimo_default` or an optional reference clip for cloning, and delivery is steered with any natural-language style phrase.

Capabilities

- Speech synthesis with preset voice or optional WAV reference clip for cloning
- Singing mode via song name lookup or the `LYRICS`: prefix for direct lyrics
- Style phrases for emotion, delivery pace, character personas, and dialects
- TTS text normalization guidance for numbers, symbols, and formatting residue
- Automatic RIFF header repair when the API returns raw PCM
- Pipe-friendly WAV output for post-processing with `ffmpeg` or `SoX`

Usage

- `bash scripts/tts_to_wav.sh "<text>" "<output.wav>" [style] [voice_sample.wav]` — full command signature
- `bash scripts/tts_to_wav.sh "The weather is nice today" /tmp/hello.wav` — preset voice, no style
- `bash scripts/tts_to_wav.sh "Congratulations on winning first prize!" /tmp/congrats.wav "happy and excited"` — mood style phrase

- `bash scripts/tts_to_wav.sh "LYRICS:Two tigers, two tigers, running fast, running fast..." /tmp/song.wav "唱歌" — singing mode with direct lyrics`
- `ffmpeg -i /tmp/tmp.wav /tmp/out.mp3 — pipe the WAV output to MP3`

When to use

Load this skill whenever the user asks the agent to speak, say something aloud, read text out, or sing a song, including when no destination is mentioned.

Requirements & notes

Requires `jq`, `python3` (standard library only), and `curl`; the API key is resolved automatically. Optional environment variables: `MIMO_VOICE_SAMPLE`, `MIMO_API_BASE_URL`, `MIMO_TTS_MODEL`, and `MIMO_SING_DICT` (song lyrics dictionary `sing0301_dict.json`). The output parent directory must exist; exit codes 1, 2, and 3 distinguish argument, API, and output failures.

normalize

Location	/home/work/.openclaw/skills/normalize/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Analyzes and redesigns a feature to match the project's design system standards, aesthetics, tokens, and patterns.

Overview

`normalize` is a design-system consistency pass. It mandates use of the frontend-design skill and its Context Gathering Protocol, then discovers the project's design system documentation, UI guidelines, and component libraries through search. It analyzes where the current feature deviates, separating cosmetic inconsistencies from functional ones and finding root causes such as missing tokens or one-off implementations. Execution covers typography, color, spacing, components, motion, responsive behavior, accessibility, and progressive disclosure. Cleanup consolidates shared components, removes orphaned code, and verifies quality.

Capabilities

- Discovers design system docs, style guides, and component libraries
- Diagnoses deviations as cosmetic, functional, or conceptual misalignment
- Replaces hard-coded values with design tokens throughout
- Swaps custom components for design system equivalents
- Aligns motion, responsive, and accessibility behavior with conventions
- Consolidates reusable components and removes orphaned code after refactoring

Usage

- `grep` — search for "design system", "ui guide", "style guide" documentation
- `frontend-design` — mandatory preparation skill with design principles
- `teach-impeccable` — must run first when no design context exists
- `[FEATURE=<value>]` — argument hint naming the feature to normalize

When to use

Load when a feature looks off-brand or inconsistent with the rest of the product, or when asked to align something with the design system.

Requirements & notes

The project should have an established design system or style guide; if principles are unclear, ask rather than guess. Verification includes linting, type-checking, and testing per repository guidelines to confirm no regressions.

onboard

Location	/home/work/.openclaw/skills/onboard/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Designs onboarding flows, empty states, and first-time user experiences that move users to value as fast as possible.

Overview

`onboard` is a product education skill for creating and improving onboarding experiences. It begins by identifying the challenge, understanding user experience levels and motivation, and defining success metrics such as completion rate

and time to value. It then applies principles like show-don't-tell, optional skippable flows, time-to-value, and progressive disclosure across welcome screens, account setup, empty states, contextual tooltips, guided tours, and interactive tutorials. It also specifies implementation patterns using tooltip and tour libraries with local storage tracking and analytics.

Capabilities

- Assesses user needs, drop-off points, and target "aha moment"
- Designs welcome screens, minimal account setup, and first-success flows
- Crafts empty states that explain value and prompt first action
- Specifies guided tours of three to seven steps with skip options
- Builds interactive tutorials with sandbox data and validation
- Tracks onboarding progress via local storage and analytics
- Verifies quality with completion rate and time-to-value metrics

Usage

- `localStorage.setItem('onboarding-completed', 'true')` — track completed onboarding state
- `Tippy.js` and `Popper.js` — tooltip libraries for contextual hints
- `Intro.js`, `Shepherd.js`, `React Joyride` — guided tour libraries
- `frontend-design` — mandatory preparation skill for design principles
- `teach-impeccable` — run first when no design context exists

When to use

Load when designing or improving onboarding flows, empty states, first-run experiences, guided tours, or feature discovery moments.

Requirements & notes

Also gather the target "aha moment" and users' experience level before designing. Track what users have seen so onboarding never repeats, and always keep skip options easy to find.

optimize

Location	/home/work/.openclaw/skills/optimize/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Identifies and fixes frontend performance issues across loading, rendering, animation, images, and bundle size with measured improvements.

Overview

`optimize` is a performance improvement skill that starts by measuring current state through Core Web Vitals, load time, bundle size, runtime performance, and network waterfalls. It then applies systematic fixes across loading performance (images, JavaScript bundles, CSS, fonts), rendering performance (layout thrashing, DOM size, paint reduction), animation performance (GPU acceleration, 60fps targets), framework optimization, and network optimization. It defines Core Web Vitals targets for LCP, FID/INP, and CLS, and a monitoring toolset. Measure before and after; optimize the biggest bottleneck first.

Capabilities

- Measures Core Web Vitals: LCP, FID/INP, CLS, TTI, FCP, TBT
- Optimizes images with WebP, AVIF, `srcset`, and lazy loading
- Reduces bundles via code splitting, tree shaking, dynamic imports
- Prevents layout thrashing by batching DOM reads and writes
- Accelerates animations using `transform` and `opacity` on the GPU
- Targets LCP under 2.5s, INP under 200ms, CLS under 0.1
- Verifies with Lighthouse, WebPageTest, and real user monitoring

Usage

- `IntersectionObserver` — detect element visibility for lazy loading
- `lazy(() => import('./HeavyChart'))` — code-split heavy components
- `webpack-bundle-analyzer` — analyze and reduce bundle size

- Chrome DevTools (Lighthouse, Performance panel) — measure before and after
- WebPageTest — test under real network and device conditions

When to use

Load when an interface feels slow, when improving load times or animation smoothness, or when Core Web Vitals scores need work.

Requirements & notes

Measure before and after every change, and test on real devices and throttled connections rather than fast desktop Chrome alone. Monitoring options named in the skill include Sentry, DataDog, and New Relic.

overdrive

Location	/home/work/.openclaw/skills/overdrive/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Pushes interfaces past conventional limits with ambitious techniques like shaders, virtual scrolling, spring physics, and scroll animation.

Overview

overdrive drives technically ambitious interface enhancements that make users ask "how did they do that?" It demands mandatory preparation via the frontend-design skill and teach-impeccable, a proposal phase presenting two to three directions with trade-offs before writing code, and browser automation iteration to visually verify effects. Its toolkit spans cinematic transitions with the View Transitions API, scroll-driven animations, WebGL and WebGPU rendering, virtual scrolling, GPU-accelerated charts, registered custom properties, Web Workers, OffscreenCanvas, and WASM. Every technique must degrade gracefully through progressive enhancement.

Capabilities

- Creates shared-element morphing with the View Transitions API
- Animates entry from display: none using @starting-style
- Drives animation from scroll with animation-timeline: scroll()
- Renders shaders and particles via WebGL, WebGPU, or Canvas 2D
- Handles huge datasets with virtual scrolling and GPU charts
- Moves heavy work to Web Workers with OffscreenCanvas and WASM
- Enforces 60fps targets and prefers-reduced-motion fallbacks

Usage

- @supports (animation-timeline: scroll()) — feature guard for scroll animation
- if ('gpu' in navigator) — detect WebGPU, fall back to WebGL2
- @property — register typed custom properties for animatable values
- Three.js, OGL, regl — WebGL libraries for shader effects
- TanStack Virtual — virtual scrolling for complex large tables
- D3 transition() — animate between chart data states

When to use

Load when a design needs a showstopper moment such as cinematic transitions, shaders, massive data tables, or effects beyond normal CSS.

Requirements & notes

Every technique needs a graceful fallback and must respect prefers-reduced-motion. Propose directions and get user confirmation before implementing, then verify results through browser automation iteration and polish past the first working version.

pdf

Location	/home/work/.openclaw/skills/pdf-generator/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	PDF toolkit for extracting text and tables, creating PDFs, merging, splitting, rotating, watermarks, encryption, and form filling.

Overview

This skill is a comprehensive PDF processing guide spanning Python libraries and command-line tools. It covers reading PDFs and metadata, extracting text with layout and tables, merging and splitting documents, rotating pages, creating new PDFs and multi-page reports, adding watermarks, encrypting files, extracting embedded images, and OCR for scanned documents. Advanced features such as pypdfium2 usage and JavaScript libraries like pdf-lib live in `reference.md`, while form filling has its own `forms.md` instructions. A quick reference table maps each common task to its best tool.

Capabilities

- Extract text with layout and tables using pdfplumber
- Merge, split, and rotate pages with pypdf or qpdf
- Create single and multi-page PDFs with reportlab
- OCR scanned PDFs via pytesseract and pdf2image
- Add watermarks and password protection to documents
- Extract embedded images and PDF metadata

Usage

- `pdftotext -layout input.pdf output.txt` — extracts text preserving layout
- `qpdf --empty --pages file1.pdf file2.pdf -- merged.pdf` — merges PDFs from the command line
- `qpdf input.pdf --pages . 1-5 -- pages1-5.pdf` — extracts a page range
- `pdfimages -j input.pdf output_prefix` — extracts embedded images
- `writer.encrypt("userpassword", "ownerpassword")` — adds password protection with pypdf
- `page.extract_tables()` — extracts tables with pdfplumber

When to use

Use whenever a task involves reading, generating, merging, splitting, editing, or analyzing PDF documents, extracting tables or images, OCR of scans, or filling PDF forms.

Requirements & notes

Requires Python with pypdf, pdfplumber, and reportlab, plus command-line tools pdftotext and pdfimages from poppler-utils, qpdf, and optionally pdftk. OCR additionally needs `pip install pytesseract pdf2image`. The license is proprietary with full terms in `LICENSE.txt`; read `forms.md` before filling PDF forms.

polish

Location	/home/work/.openclaw/skills/polish/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Performs a final meticulous quality pass fixing alignment, spacing, states, copy, and detail issues before shipping.

Overview

polish is the last-step quality pass separating shipped from great. It begins with a pre-polish assessment of completeness, known issues, quality bar (MVP versus flagship), and ship date, since polishing incomplete work is wasted effort. It then works methodically through visual alignment and spacing, typography refinement, color and contrast, interaction states, micro-interactions, content and copy, icons and images, forms, edge cases, responsiveness, performance, and code quality. A full checklist and final verification with real-device testing and fresh-eyes review close the pass.

Capabilities

- Enforces pixel-perfect alignment and spacing-scale consistency
- Verifies typography hierarchy, line length, widows and orphans
- Checks WCAG contrast and consistent design-token color usage
- Implements all interaction states: hover, focus, active, disabled, loading, error
- Smooths transitions at 150-300ms with ease-out easing
- Cleans code: console logs, dead comments, unused imports, TypeScript any
- Runs a shipping checklist covering accessibility, layout shift, touch targets

Usage

- `frontend-design` — mandatory preparation skill with design principles
- `teach-impeccable` — run first when no design context exists
- `prefers-reduced-motion` — respect user motion preferences in transitions
- `[TARGET=<value>]` — argument hint naming the polish target

When to use

Load as the final pass before shipping, when small detail issues remain, or when reviewing alignment, spacing, and consistency across an interface.

Requirements & notes

Polish only after functional completeness, and gather the quality bar (MVP versus flagship) first. Standards applied include WCAG AA contrast, 44x44px minimum touch targets, and 14px minimum mobile text size.

Powerpoint / PPTX

Location	/home/work/.openclaw/skills/wps-ppt-generator/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Create, inspect, and edit PowerPoint PPTX decks with template fidelity, layout mapping, notes, charts, and visual QA.

Overview

Powerpoint / PPTX guides working with .pptx decks as OOXML packages with separate parts for slides, layouts, masters, media, notes, and comments. It insists on choosing the workflow first: text extraction, editing an existing deck, rebuilding from a template, or creating from scratch. The deck must be inventoried before content is replaced so placeholder and layout indexes are targeted correctly, and content QA and visual QA run as separate passes before delivery.

Capabilities

- Deck inventory: layouts, placeholders, notes, comments, media, and typography
- Content-to-placeholder matching sized to real idea, chart, and image counts
- Theme, master, and alignment preservation for consistent visual language
- Split QA passes: content semantics and rendered visual inspection
- Portability rules for fonts, effects, and aspect-ratio differences
- Trap catalog for placeholder leftovers, master overrides, and merged decks

Usage

- `clawhub install <slug>` — install a related skill after user confirmation
- `clawhub star powerpoint-pptx` — provide feedback if the skill was useful
- `clawhub sync` — stay updated with skill releases

When to use

Load this skill when the task is about PowerPoint or .pptx files, especially when layouts, templates, placeholders, notes, charts, comments, or final rendering quality matter.

Requirements & notes

Runs on Linux, macOS, and Windows with no required binaries. Render or inspect the actual deck output before delivery, assume the first render is wrong and run at least one fix-and-verify cycle, and re-check affected slides after every change.

quieter

Location	/home/work/.openclaw/skills/quieter/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Reduces visual intensity in overly bold or aggressive designs, creating a refined, approachable aesthetic without losing impact.

Overview

quieter tones down designs that are too bold, aggressive, or overstimulating. It first assesses intensity sources such as color saturation, contrast extremes, competing visual weight, animation excess, and complexity, while clarifying context and preserving what already works. It then plans refinement and executes across color (desaturation to 70-85

percent, tinted grays, neutral dominance), visual weight (lighter weights, more white space), simplification (removing decorative gradients, glows, layers), motion reduction (shorter distances, gentler easing), and composition. The goal is refined sophistication, not blandness.

Capabilities

- Identifies intensity sources from saturation to animation excess
- Desaturates palettes and lets neutrals dominate with color accents
- Reduces font weights and increases breathing room
- Removes decorative gradients, shadows, glows, and textures
- Shortens animation distances and swaps in ease-out-quart easing
- Preserves hierarchy, personality, and usability while calming the design

Usage

- frontend-design — mandatory preparation skill for design principles
- teach-impeccable — run first when no design context exists
- ease-out-quart — refined easing for smooth, understated motion
- [TARGET=<value>] — argument hint naming the design to quiet

When to use

Load when a design feels too loud, aggressive, or overstimulating, or when the brief calls for calm, premium restraint rather than boldness.

Requirements & notes

Design context from the frontend-design protocol is required first; ask the user directly about anything that cannot be inferred. Great quiet design is harder than great bold design, so subtlety must be executed with precision rather than collapse into generic laziness.

self-improvement

Location	/home/work/.openclaw/skills/self-improvement/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Logs learnings, errors, corrections, and feature requests to markdown files for continuous agent improvement.

Overview

The self-improvement skill captures learnings, errors, and corrections as structured markdown entries in a .learnings/ directory so future sessions avoid repeating mistakes. Entries use templated formats with IDs like LRN-YYYYMMDD-XXX, priority, status, area tags, and See Also links for recurring-pattern tracking, and can be resolved or promoted as fixes land. Broadly applicable learnings are promoted into project memory files such as CLAUDE.md, AGENTS.md, TOOLS.md, and SOUL.md, and valuable learnings can even be extracted into new skills.

Capabilities

- Structured logs: LEARNINGS.md, ERRORS.md, and FEATURE_REQUESTS.md with ID schemes
- Category tagging: correction, knowledge_gap, best_practice, and integration errors
- Recurring pattern detection via See Also links, Pattern-Key, and Recurrence-Count
- Promotion of durable learnings to CLAUDE.md, AGENTS.md, TOOLS.md, or SOUL.md
- Automatic skill extraction from qualifying resolved learnings
- Opt-in hook scripts for reminders and error detection
- Works across Claude Code, Codex, Copilot, and OpenClaw

Usage

- mkdir -p ~/.openclaw/workspace/.learnings — create the log directory
- clawdhub install self-improving-agent — install via ClawdHub
- openclaw hooks enable self-improvement — enable session-start reminders
- ./skills/self-improvement/scripts/extract-skill.sh skill-name --dry-run — preview skill extraction
- grep -r "keyword" .learnings/ — search for similar existing entries

When to use

Load this skill when a command fails unexpectedly, the user corrects the agent, a missing capability is requested, an external API or tool fails, knowledge proves outdated, or a better approach to a recurring task is found.

Requirements & notes

No special dependencies beyond standard shell tools; hook integration is opt-in via `.claude/settings.json` or the OpenClaw hooks directory (`scripts/activator.sh`, `scripts/error-detector.sh`). Gitignore `.learnings/` for per-developer logs or track it to share knowledge across the team.

skill-creator

Location	/home/work/.openclaw/skills/skill-creator/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Guide for building effective agent skills: structure, writing principles, scaffolding scripts, and validated packaging.

Overview

This custom skill-creator is the active copy; a different built-in skill-creator also ships inside the OpenClaw package, so use this workspace version when creating or updating skills. It models skills as modular packages with a required SKILL.md plus optional scripts, references, and assets, and enforces progressive disclosure so metadata, body, and bundled resources load at increasing depth. The documented workflow covers understanding use cases through concrete examples, planning reusable resources, initializing, editing, packaging with automatic validation, and iterating on real usage.

Capabilities

- `init_skill.py` scaffolds a skill directory with templates and example resources
- `package_skill.py` validates frontmatter, naming, and structure, then builds a `.skill` package
- Writing guidance: concise bodies and degrees of freedom matched to task fragility
- Progressive disclosure patterns for references split by domain or by variant
- Frontmatter rules: only name and description, with all triggers in the description
- Design references for multi-step workflows and output format patterns

Usage

- `scripts/init_skill.py <skill-name> --path <output-directory>` — scaffold a new skill
- `scripts/package_skill.py <path/to/skill-folder>` — validate and package the skill
- `scripts/package_skill.py <path/to/skill-folder> ./dist` — package with output directory

When to use

Load this skill when the user wants to create a new skill or update an existing one that extends agent capabilities with specialized knowledge, workflows, or tool integrations.

Requirements & notes

The bundled helper scripts run under Python. Keep SKILL.md bodies under 500 lines and avoid extraneous files like README.md or CHANGELOG.md inside the skill folder. Added scripts should be tested by actually running them before packaging.

skill-vetter

Location	/home/work/.openclaw/skills/skill-vetter/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Security-first vetting protocol for third-party skills: source checks, code review, red flags, and risk rating.

Overview

skill-vetter defines a mandatory security review to run before installing any skill from ClawdHub, GitHub, or another source. It walks four steps: source check, full code review against a red-flag list, permission scope evaluation, and risk classification, then produces a standardized vetting report with metrics, permissions, risk level, and a verdict. Its guiding rule is that no skill is worth compromising security: when in doubt, do not install, and escalate high-risk decisions to the human.

Capabilities

- Four-step vetting protocol: source, code review, permission scope, risk class

- Red-flag list covering exfiltration, credential access, obfuscation, and eval abuse
- Risk classification from LOW to EXTREME with matching install actions
- Standardized skill vetting report format with verdict and notes
- Quick GitHub vetting commands for repo stats and file listings
- Trust hierarchy ranking official, popular, known, and unknown sources

Usage

- `curl -s "https://api.github.com/repos/OWNER/REPO" | jq '{stars: .stargazers_count, forks: .forks_count, updated: .updated_at}'` — check repository stats
- `curl -s "https://api.github.com/repos/OWNER/REPO/contents/skills/SKILL_NAME" | jq '.[].name'` — list skill files
- `curl -s "https://raw.githubusercontent.com/OWNER/REPO/main/skills/SKILL_NAME/SKILL.md"` — fetch SKILL.md for review

When to use

Load this skill before installing any skill from ClawdHub, GitHub, or shared sources, or whenever unknown skill code needs evaluation.

Requirements & notes

Requires `curl` and `jq` for the quick vetting commands. Skills that request credentials always require human approval, HIGH-risk skills require human approval, and EXTREME-risk skills must not be installed at all. Document vetting results for future reference.

skillhub-preference

Location	/home/work/.openclaw/skills/skillhub-preference/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Policy guidance preferring skillhub for skill search, install, and update, with clawhub as the documented fallback source.

Overview

Skillhub Preference is a small policy skill that governs how skill discovery, installation, and upgrades are sourced. It instructs the agent to try skillhub first for search, install, and update operations, since it is the preferred registry for speed and compliance on CN networks. When skillhub is unavailable, rate-limited, or returns no match, the workflow falls back to clawhub. The policy also demands summarizing the source, version, and notable risk signals before any installation, reporting raw command output for searches, and never claiming exclusivity, because both registries remain allowed.

Capabilities

- Prefer skillhub for search, install, and update operations
- Fall back to clawhub on unavailability, rate limits, or no match
- Summarize source, version, and risk signals before installation
- Report raw command output for every search request
- Keep both registries permitted without exclusivity claims

Usage

- `skillhub search <keywords>` — run first for search requests and report output
- `skillhub` — the preferred source for skill install and update operations
- `clawhub` — fallback source when skillhub fails or finds no match

When to use

Use whenever the task involves skill discovery, installation, or upgrades, including user questions about skills, plugins, or extending agent capabilities.

Requirements & notes

Requires access to the skillhub command-line tool first and to clawhub as backup; no API keys or local setup state are described in the skill. Treat this skill as policy guidance rather than a code tool: it defines sourcing order and disclosure duties only. Before installing anything, present the source, version, and notable risk signals to the user for confirmation.

teach-impeccable

Location	/home/work/.openclaw/skills/teach-impeccable/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	One-time setup that gathers project design context through codebase exploration and questions, then persists it to .impeccable.md.

Overview

teach-impeccable establishes persistent design guidelines for a project so all future design work has context. It first explores the codebase for READMEs and docs, package configuration, existing components, brand assets, and design tokens to discover what it can without asking. It then asks UX-focused questions only about what could not be inferred, covering users and purpose, brand personality, aesthetic preferences, and accessibility needs. Finally it synthesizes findings into a Design Context section written to .impeccable.md at the project root, with an option to mirror it elsewhere.

Capabilities

- Scans READMEs, configs, components, and brand assets for design clues
- Asks targeted questions on users, brand, aesthetics, and accessibility
- Synthesizes Design Context covering users, brand, aesthetics, principles
- Writes and updates the design context file in the project root
- Optionally appends context to the Copilot instructions file
- Summarizes key design principles that guide all future work

Usage

- .impeccable.md — project root file holding the Design Context section
- .github/copilot-instructions.md — optional secondary destination for context
- ## Design Context — the markdown section synthesized and persisted

When to use

Run once when starting design work on a project, or whenever frontend-design or a sibling design skill reports that no design context exists yet.

Requirements & notes

No special program requirements. The skill is user-invokable and depends on answers only the project creator can provide; audience, tone, and feeling cannot be inferred from code alone.

typeset

Location	/home/work/.openclaw/skills/typeset/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Improves typography by fixing font choices, hierarchy, sizing, weight consistency, and readability for intentional, polished text.

Overview

typeset assesses and improves typography that feels generic, inconsistent, or poorly structured. It audits font choices (flagging invisible defaults like Inter, Roboto, and Arial), hierarchy clarity, sizing and scale consistency, readability, and cross-page consistency. Improvements follow a plan built on a modular type scale and clear weight strategy, then executed through font selection and pairing, hierarchy construction, readability fixes like ch-based measure and tuned line-height, and detail refinements such as tabular numerals and letter-spacing. The goal is clarity and intentionality, not fanciness.

Capabilities

- Audits font choices against brand personality and family count limits
- Builds modular type scales with ratios like 1.25, 1.333, or 1.5
- Establishes hierarchy with five sizes from caption to heading
- Fixes readability with ch measures and context-tuned line-height
- Applies tabular-nums and intentional letter-spacing refinements

- Uses semantic token names like `--text-body` and `--text-heading`
- Verifies WCAG contrast and 200 percent zoom support

Usage

- `reference/typography.md` — typography reference for scales and pairing
- `clamp(min, preferred, max)` — fluid sizing across viewport ranges
- `font-display: swap` — avoid layout shift when loading web fonts
- `max-width: 65ch` — comfortable measure for body text
- `font-kerning: normal` — refine kerning and OpenType rendering
- `[TARGET=<value>]` — argument hint naming the typography target

When to use

Load when text looks generic, hierarchy is muddy, or typography needs to match brand personality and stay readable in long passages.

Requirements & notes

The frontend-design context protocol applies first, including teach-impeccable when no design context exists. Do not disable browser zoom with `user-scalable=no`, and size text in `rem` rather than `px` to respect user settings.

Word / DOCX

Location	/home/work/.openclaw/skills/word-docx/SKILL.md
Category	Custom workspace skill
Status	Active in skill catalog
Summary	Create, inspect, and edit Word DOCX files with reliable styles, numbering, tracked changes, and layout fidelity.

Overview

Word / DOCX treats `.docx` files as OOXML packages rather than plain text, because structure matters as much as visible content. It separates three distinct jobs: structured extraction for reading, style-driven generation for new files, and OOXML-aware editing of fragile existing documents. The rules cover styles and direct formatting, numbering definitions, section-level page layout, tracked changes, comments, fields, and bookmarks, plus round-trip compatibility checks across Word, LibreOffice, and Google Docs before delivery.

Capabilities

- Style-preserving editing that extends the document's existing style system
- Correct handling of numbering definitions, restarts, and indentation
- Section-aware layout: margins, headers, footers, page size, and breaks
- Precise tracked-change, comment, and field edits with stable anchors
- Trap catalog for copy-paste styles, broken relationships, and stale fields
- Round-trip compatibility verification guidance across editors

Usage

- `clawhub install <slug>` — install a related skill after user confirmation
- `clawhub star word-docx` — provide feedback if the skill was useful
- `clawhub sync` — stay updated with skill releases

When to use

Load this skill when the task is about Word or `.docx` files, especially when tracked changes, comments, fields, tables, templates, or page layout constraints require round-trip stability.

Requirements & notes

Runs on Linux, macOS, and Windows. Legacy `.doc` inputs usually need conversion first, and `.docm` files are macro-bearing and higher risk. For review workflows prefer minimal replacements over wholesale paragraph rewrites, and set page size explicitly because A4 and Letter defaults change pagination.

Part 2 - Plugin Skills (Active in Skill Catalog)

10 skills, alphabetical.

dingtalk-channel-rules

Location	/home/work/.openclaw/plugin-skills/dingtalk-channel-rules/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Defines DingTalk conversation output style and routes all DingTalk business operations through the dws CLI, not the connector.

Overview

This always-active skill governs how the agent writes and routes output in DingTalk conversations. It sets a concise, conversational, low-ceremony writing style, Markdown message conventions, and AI Card usage for streaming long replies. Its core routing principle is that the connector handles only the message channel while every DingTalk business feature goes through the dws CLI. For multi-bot setups it mandates credential isolation so messages never go out under the wrong bot identity.

Capabilities

- Enforces concise, conversational replies without ritual summary paragraphs
- Formats Markdown messages within the 2000-character single-message limit
- Streams long content through AI Cards with Markdown rendering
- Addresses replies to the current chat, a specific user, or a group
- Routes docs, calendar, AI tables, todos, reports, drive, and DING to dws
- Isolates multi-bot credentials with the `--client-id` parameter

Usage

- `dws chat message send-by-bot --client-id dingXXXXXXXX --robot-code <code> --users "userId"` — sends a bot message with the right credentials
- `dws doc` — creates, appends, searches, and lists DingTalk documents
- `dws calendar` — manages schedules and meetings
- `dws aitable` — creates tables and reads or writes row data
- `dws todo` — creates personal and group todos with due dates
- `dws ding` — sends strong-reminder DING messages to users or groups

When to use

Always active in DingTalk sessions. Applies whenever replying in DingTalk or when the user requests any DingTalk business feature such as documents, schedules, tables, todos, reports, files, or DING reminders.

Requirements & notes

A configured DingTalk bot is required. In multi-bot scenarios, extract `clientId` from the `[DingTalk Bot Context]` marker in each message and pass it via `--client-id`; otherwise `dws` may send messages from the wrong bot. Messaging, media, and conversation context are handled automatically by the connector.

dingtalk-troubleshoot

Location	/home/work/.openclaw/plugin-skills/dingtalk-troubleshoot/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Troubleshoots DingTalk connector failures: dws CLI installation, OAuth login, token expiry, permissions, bot connectivity, and recoverable errors.

Overview

This skill is a diagnostic guide for the DingTalk connector. It activates when `dws` commands fail, authorization behaves abnormally, or the connection drops. It provides an FAQ-style mapping from known `stderr` signals to fixes, covering missing CLI, missing login, expired tokens, and insufficient permissions. It also defines the Recovery loop for recoverable errors signalled by a `RECOVERY_EVENT_ID`, and a generic retry procedure that escalates unrecognized errors to the user instead of guessing workarounds.

Capabilities

- Diagnoses "command not found: dws" installation failures
- Resolves OAuth login prompts and expired-token errors
- Identifies permission-denied and HTTP 403 problems
- Troubleshoots bots that stay offline after device-auth scanning
- Executes the Recovery loop for RECOVERY_EVENT_ID errors
- Retries once with `--verbose` and reports unknown failures verbatim

Usage

- `npm i -g dingtalk-workspace-cli` — installs the dws CLI globally
- `dws --version` — verifies installation, version 1.0.6 or later
- `dws auth login` — completes OAuth sign-in via QR code scan
- `dws recovery execute --event-id <event_id> --format json` — fetches the recovery plan
- `dws recovery finalize --event-id <event_id>` — closes the recovery loop

When to use

Load when a dws command fails, authorization errors appear, the DingTalk connection breaks, or a bot does not come online after scanning.

Requirements & notes

Requires the dws CLI 1.0.6 or newer and network access to DingTalk. Permission errors need an organization admin to grant the missing API scopes. Unrecognized errors must be reported to the user in full; never invent substitute solutions.

dws-cli

Location	/home/work/.openclaw/plugin-skills/dws-cli/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Manages DingTalk products such as AI tables, calendar, contacts, chats, todos, approvals, attendance, reports, and DING via the dws CLI.

Overview

This full-product skill has the agent generate dws CLI commands that openclaw executes in the terminal, covering eleven DingTalk product areas from AI tables to the workbench. It requires dws 1.0.6 or newer plus completed OAuth authorization. An intent-routing decision tree maps user requests to the right product, and a four-step core flow classifies intent, resolves ambiguity by asking questions, maps to a product, and reads the product reference before implementation. Dangerous or irreversible operations require explicit user confirmation before the `--yes` flag is added.

Capabilities

- AI table base, table, field, and record CRUD plus template search
- Calendar events, participants, meeting rooms, and free/busy queries
- Contact and department directory lookups
- Group chat, bot messaging, Webhooks, and member management
- Todo, approval, attendance, report log, and DING message operations
- Structured error handling and Recovery loop execution

Usage

- `dws --version` — verifies CLI version 1.0.6 or newer
- `dws auth status` — confirms the current login state
- `dws auth login` — runs OAuth QR code authorization
- `--format json` — required on every command for parseable output
- `--yes` — confirms dangerous operations after explicit user approval

When to use

Load when users request DingTalk operations: table data, schedules, meetings, contacts, group chats, bot notifications, todos, approvals, attendance, reports, or DING reminders.

Requirements & notes

The dws CLI must be installed and authorized. The connector injects DWS_CHANNEL, DWS_CLIENT_ID, and DWS_CLIENT_SECRET automatically. Never fabricate IDs or field names, never use curl or HTTP APIs directly, and keep batch operations at 30 records or fewer per call.

feishu-doc

Location	/home/work/.openclaw/plugin-skills/feishu-doc/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Reads, writes, creates, and restructures Feishu documents and Docx tables through the single feishu_doc tool.

Overview

The feishu_doc tool covers all document operations through one action parameter, including block management and native table creation for Docx. Document tokens are extracted from Feishu URLs, for example the last path segment of a /docx/ link. Writing replaces the whole document with Markdown content, while appending adds to the end; Markdown tables are not supported, so tables must be built with the table actions. Document creation should always pass owner_open_id from the inbound sender_id so the requesting user automatically receives full_access, and the feishu_wiki tool depends on this one for page content.

Capabilities

- Reads documents with title, plain text, and block statistics
- Replaces or appends Markdown content in documents
- Creates documents, optionally inside a folder
- Lists, gets, updates, and deletes individual blocks
- Creates Docx tables and writes table cell values
- Uploads images and file attachments from URL or local path

Usage

- feishu_doc with {"action": "read", "doc_token": "ABC123def"} — reads text and block statistics
- feishu_doc with {"action": "write", "doc_token": "...", "content": "..."} — replaces the document with Markdown
- feishu_doc with {"action": "list_blocks", "doc_token": "..."} — returns full structured block data
- feishu_doc with {"action": "create_table_with_values", "doc_token": "..."} — creates a filled table in one step
- feishu_doc with {"action": "upload_image", "doc_token": "...", "url": "..."} — inserts an image

When to use

Activate when the user mentions Feishu docs, cloud docs, or docx links, or when wiki page content must be read or edited.

Requirements & notes

Requires the Feishu channel with the doc tool enabled (default true) and scopes docx:document, docx:document:readonly, docx:document.block:convert, drive:drive. Markdown tables are unsupported, and small images should be scaled to at least 800px wide before upload.

feishu-drive

Location	/home/work/.openclaw/plugin-skills/feishu-drive/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Manages Feishu cloud storage: list folders, get file info, create folders, move and delete files via feishu_drive.

Overview

The feishu_drive tool performs cloud storage operations in Feishu drive spaces. Folder tokens are extracted from drive URLs, and listings return each file's token, name, type, URL, and timestamps. Eight file types are supported: doc, docx, sheet, bitable, folder, file, mindnote, and shortcut. A key limitation is that bots authenticate with tenant_access_token and have no personal root folder, so create_folder without a folder_token fails and the bot can only reach files that have been explicitly shared with it.

Capabilities

- Lists root or specific folder contents with metadata
- Gets file info for any supported Feishu file type
- Creates folders at root or inside parent folders
- Moves files between folders
- Deletes files and folders
- Handles doc, docx, sheet, bitable, folder, file, mindnote, shortcut types

Usage

- `feishu_drive` with `{"action": "list"}` — lists root directory contents
- `feishu_drive` with `{"action": "list", "folder_token": "fldcnXXX"}` — lists a specific folder
- `feishu_drive` with `{"action": "info", "file_token": "ABC123", "type": "docx"}` — returns file details
- `feishu_drive` with `{"action": "create_folder", "name": "New Folder", "folder_token": "fldcnXXX"}` — creates a subfolder
- `feishu_drive` with `{"action": "move", "file_token": "ABC123", "type": "docx", "folder_token": "fldcnXXX"}` — moves a file

When to use

Activate when the user mentions Feishu cloud space, folders, or drive file management such as moving, deleting, or organizing files.

Requirements & notes

Requires scope `drive:drive` for create, move, and delete, or `drive:drive:readonly` for listing and info. Bots have no root folder: a user must first create a folder manually, share it with the bot, and then the bot can create subfolders inside it.

feishu-perm

Location	/home/work/.openclaw/plugin-skills/feishu-perm/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Manages Feishu document and file permissions by listing, adding, and removing collaborators with view, edit, or full_access rights.

Overview

The `feishu_perm` tool manages sharing and collaborator permissions for Feishu files and documents. It supports three actions: listing current collaborators, adding a collaborator with a chosen permission level, and removing a collaborator. Members can be addressed by email, openid, userid, unionid, openchat group, or department ID, and target objects include old-format docs, docx, sheets, bitables, folders, uploaded files, wiki nodes, and mindnotes. Because permission management is sensitive, the tool is disabled by default in the Feishu channel configuration and must be enabled explicitly.

Capabilities

- Lists collaborators with member type, ID, and permission level
- Adds collaborators by email, user ID, group chat, or department
- Removes collaborators from documents, folders, or wiki nodes
- Applies view, edit, or full_access permission levels
- Supports doc, docx, sheet, bitable, folder, file, wiki, mindnote targets

Usage

- `feishu_perm` with `{"action": "list", "token": "ABC123", "type": "docx"}` — lists current collaborators
- `feishu_perm` with `{"action": "add", "token": "...", "member_type": "email", "member_id": "user@example.com", "perm": "edit"}` — shares a file
- `feishu_perm` with `{"action": "remove", "token": "...", "member_type": "email", "member_id": "user@example.com"}` — revokes access

When to use

Activate when the user mentions sharing, permissions, or collaborators on Feishu documents, files, folders, or wiki pages.

Requirements & notes

Requires the `drive:permission` scope. The tool defaults to `perm: false` (disabled) because permission management is sensitive; enable it explicitly with `perm: true` under the Feishu channel tools configuration before use.

feishu-wiki

Location	/home/work/.openclaw/plugin-skills/feishu-wiki/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Navigates Feishu knowledge bases: list spaces and nodes, get node details, create, move, and rename wiki pages.

Overview

The `feishu_wiki` tool handles knowledge base navigation in Feishu. It lists accessible wiki spaces, enumerates node trees, returns node details including the `obj_token` needed for content editing, and can create, move, and rename nodes of types like `docx`, `sheet`, `bitable`, `mindnote`, `file`, `doc`, and `slides`. Wiki `space_id` values are opaque strings that must always stay quoted in tool calls, since passing long numeric-looking IDs as numbers can corrupt them through JavaScript precision limits. Editing wiki pages is a two-tool workflow: get the node to find its `obj_token`, then use `feishu_doc` to read or write content.

Capabilities

- Lists all accessible wiki knowledge spaces
- Lists nodes with optional parent node filtering
- Gets node details including `obj_token` and `obj_type`
- Creates nodes of type `docx`, `sheet`, `bitable`, `mindnote`, `file`, `doc`, `slides`
- Moves nodes within a space or across spaces
- Renames wiki nodes

Usage

- `feishu_wiki` with `{"action": "spaces"}` — lists accessible knowledge spaces
- `feishu_wiki` with `{"action": "nodes", "space_id": "7xxx", "parent_node_token": "wikcnXXX"}` — lists child nodes
- `feishu_wiki` with `{"action": "get", "token": "ABC123def"}` — returns node details with `obj_token`
- `feishu_wiki` with `{"action": "create", "space_id": "7xxx", "title": "New Page"}` — creates a wiki page
- `feishu_doc` with `{"action": "read", "doc_token": "obj_token"}` — reads the wiki page content

When to use

Activate when the user mentions Feishu knowledge bases, wiki spaces, wiki links, or asks to create or reorganize wiki pages.

Requirements & notes

Requires the `wiki:wiki` or `wiki:wiki:readonly` scope and the `feishu_doc` tool enabled, since wiki pages are documents and their content is read and written through `feishu_doc`. Always keep `space_id` values quoted strings.

qqbot-channel

Location	/home/work/.openclaw/plugin-skills/qqbot-channel/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Proxies QQ Open Platform HTTP API calls via <code>qqbot_channel_api</code> with automatic token auth for guilds, channels, members, forums, announcements, and schedules.

Overview

`qqbot_channel_api` is an HTTP proxy tool for the QQ Open Platform that fills in the authentication token automatically. The caller only supplies the HTTP method, API path, request body, and query parameters; the base URL is `https://api.sgroup.qq.com`. Coverage spans guilds, subchannels, members, announcements, forum threads, and channel schedules, with full parameter references in `references/api_references.md`. The skill also defines multi-step workflows for channel discovery, forum posting with comments, and paginated member management.

Capabilities

- Lists guilds and inspects guild API permissions
- Creates, modifies, and deletes subchannels

- Lists members, role members, and online member counts
- Creates and deletes announcements and recommended channels
- Publishes forum threads and comments (private-domain bots only)
- Creates, modifies, and deletes channel schedules

Usage

- qqbot_channel_api with {"method": "GET", "path": "/users/@me/guilds"} — lists the bot's guilds
- qqbot_channel_api with {"method": "GET", "path": "/guilds/{guild_id}/channels"} — lists subchannels
- qqbot_channel_api with {"method": "POST", "path": "/guilds/{guild_id}/channels"} — creates a subchannel
- qqbot_channel_api with {"method": "PUT", "path": "/channels/{channel_id}/threads"} — publishes a forum post
- qqbot_channel_api with {"method": "POST", "path": "/channels/{channel_id}/schedules"} — creates a schedule

When to use

Load when managing QQ channels: listing guilds or members, creating subchannels, publishing forum posts, managing announcements, or working with channel schedules.

Requirements & notes

Requires the channels.qqbot config with valid AppID and ClientSecret. Query parameter values must be strings, schedule timestamps are millisecond strings, deletes are irreversible, and forum operations require private-domain bot mode. Schedule calls are rate limited to 10 per admin per day and 100 per channel per day, and image URLs like avatars must use Markdown image syntax.

qqbot-media

Location	/home/work/.openclaw/plugin-skills/qqbot-media/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Sends and receives QQ rich media (images, voice, video, files) using qqmedia tags with the type inferred from file extensions.

Overview

This skill adds rich media support to QQBot conversations through <qqmedia> tags that wrap a path or URL. The media type is routed automatically from the file extension: image formats become images, audio formats become voice, video containers become video, everything else becomes a file, and extension-less URLs default to images. Incoming images are downloaded locally with their paths shown in the attachment context so they can be echoed back, and incoming voice is transcribed first when speech-to-text is available. Outgoing local files must live under the OpenClaw media directory because platform security only allows uploads from trusted media roots.

Capabilities

- Sends images, voice, video, and file attachments
- Routes media type automatically from the file extension
- Sends multiple attachments using multiple tags
- Re-sends received images directly from local paths
- Transcribes received voice when STT capability exists
- Enforces per-type upload size limits

Usage

- <qqmedia>path or URL</qqmedia> — sends one media attachment
- <qqmedia>/absolute/path/image.png</qqmedia> — sends a local image by absolute path
- <qqmedia>https://example.com/clip.mp4</qqmedia> — sends a remote video
- <qqmedia>/absolute/path/audio.silk</qqmedia> — sends a voice message

When to use

Load whenever the user sends media in a QQ conversation or asks to send an image, voice message, video, or file attachment.

Requirements & notes

Requires the `channels.qqbot` config. Paths must be absolute and local outbound files must sit under `~/.openclaw/media/qqbot/` or a subdirectory, since other locations are blocked by upload policy. Size caps: images 30MB, video 100MB, files 100MB, voice 20MB. Respect session capability notes and do not repeat text already spoken in voice.

qqbot-remind

Location	/home/work/.openclaw/plugin-skills/qqbot-remind/SKILL.md
Category	Plugin skill
Status	Active in skill catalog
Summary	Creates, lists, and cancels QQ reminders via <code>qqbot_remind</code> , with a <code>cron-tool</code> fallback for one-time and recurring schedules.

Overview

This skill registers real scheduled reminders for QQ conversations, and its mandatory rule is that any delayed or timed request must trigger a tool call rather than a verbal promise, since the agent has no memory or background thread. The primary path uses the `qqbot_remind` tool, which creates Gateway cron jobs directly, after which the separate cron tool must not be called. A documented fallback builds cron jobs manually when only the cron tool is available, with `payload.kind` fixed to `agentTurn` and announce delivery on the `qqbot` channel, because `systemEvent` never reaches the user as a QQ message.

Capabilities

- Adds one-shot reminders with relative times like `5m` or `1h30m`
- Adds recurring reminders via cron expressions with `timezone`
- Lists all registered reminders with job IDs
- Cancels reminders by job ID
- Supports direct-message and group delivery targets
- Clarify requests when the reminder time is missing or vague

Usage

- `qqbot_remind` with `{"action": "add", "content": "drink water", "time": "5m"}` — schedules a one-time reminder
- `qqbot_remind` with `{"action": "add", "content": "punch in", "time": "0 8 * * *"}` — schedules a daily reminder
- `qqbot_remind` with `{"action": "list"}` — shows all current reminders
- `qqbot_remind` with `{"action": "remove", "jobId": "xxx"}` — cancels a reminder

When to use

Load for any QQ request involving reminders, alarms, or scheduled tasks, including phrases like "in X minutes", "every day at X", or "wake me".

Requirements & notes

Requires the `channels.qqbot` config. Pure relative times can be scheduled directly; ambiguous or missing times require follow-up questions. In the cron fallback, recurring reminders must set `"tz": "Asia/Shanghai"` and one-shot jobs use absolute millisecond timestamps in `schedule.atMs`.

Part 3 - OpenClaw Built-in Skills (Active in Skill Catalog)

12 skills, alphabetical.

canvas

Location	/usr/lib/node_modules/openclaw/skills/canvas/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Presents HTML on connected node canvases with navigation, JavaScript evaluation, screenshots, and canvas host URL debugging.

Overview

The canvas skill shows HTML on connected Mac, iOS, and Android nodes. A canvas host serves files from `plugins.entries.canvas.config.host.root` over canvas routes on the Gateway HTTP port (default 18789), the node bridge sends canvas URLs to node apps, and the apps render them in a WebView. The reachable host name follows `gateway.bind`: loopback for local-only, LAN IP for LAN, or the Tailscale host for a tailnet, with auto picking the best route. Paired nodes normally receive node-scoped `pluginSurfaceUrls.canvas` capability URLs, which should be preferred when available.

Capabilities

- Presents hosted HTML pages on node canvases
- Hides the canvas or navigates to new URLs
- Evaluates JavaScript in the current canvas
- Captures canvas screenshots as proof
- Serves static HTML, CSS, and JS with optional live reload
- Debugs unreachable, blank, or stale canvas URLs

Usage

- `present` — shows the canvas with an optional URL
- `hide` — hides the canvas
- `navigate` — opens a new URL in the canvas
- `eval` — runs JavaScript in the current canvas
- `snapshot` — captures a screenshot of the canvas
- `/__openclaw__/canvas/<file>.html` — canvas route mapping to a file under the host root

When to use

Load when showing HTML, dashboards, games, or visual output on connected node screens, or when debugging canvas host URLs and blank pages.

Requirements & notes

The Canvas plugin host must be enabled with a configured root, and a paired node running an OpenClaw app is required. Localhost URLs only work for a node on the same machine; LAN and tailnet nodes need matching `gateway.bind` routes, and live reload requires `liveReload` plus files written under the root.

diagram-maker

Location	/usr/lib/node_modules/openclaw/skills/diagram-maker/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Creates SVG/HTML and Excalidraw diagrams for concepts, architecture, flows, and whiteboards as standalone visual artifacts.

Overview

Diagram Maker produces diagrams as visual artifacts instead of prose descriptions. It supports three output modes: `clean-svg` for educational concepts, physical systems, and processes; `architecture-svg` for software, cloud, and infrastructure topologies; and `excalidraw` for editable hand-drawn whiteboards. The workflow extracts nodes, groups, labels, and directed relationships, then picks a layout such as left-to-right, top-down, hub-spoke, swimlanes, layered stack, or sequence before generating the file and verifying its syntax when feasible. Routing is use-case driven: collaborative editing goes to Excalidraw, polished standalone browser output to SVG/HTML, and science or product

concepts to clean SVG.

Capabilities

- Generates standalone SVG/HTML diagram files with inline CSS and inline SVG
- Produces architecture SVGs covering services, databases, queues, and trust zones
- Emits editable Excalidraw JSON whiteboards with bound text labels
- Chooses output mode by use case: concept, architecture, or collaborative editing
- Enforces style rules such as semantic colors and connectors drawn behind nodes

Usage

- `references/svg-template.md` — SVG wrapper template; replace the SVG placeholder
- `references/excalidraw-patterns.md` — exact Excalidraw element snippets
- `./diagram.html` — default output path for SVG/HTML diagrams
- `./diagram.excalidraw` — default output path for Excalidraw diagrams

When to use

Use when the user requests a diagram, flowchart, architecture sketch, concept map, sequence diagram, lifecycle illustration, or an editable collaborative whiteboard.

Requirements & notes

SVG/HTML output must be a single self-contained file with no external fonts, JavaScript, images, gradients, glows, or remote assets, and text needs 24px padding inside boxes. Excalidraw files require the keys `type`, `version`, `source`, `elements`, and `appState`, with bound text directly after its container, minimum labeled shapes of 120x60, 16px body text, roughness 1, and `fontFamily` 1. Prefer 5 to 9 main elements per diagram and add a legend only when symbols are not obvious.

meme-maker

Location	<code>/usr/lib/node_modules/openclaw/skills/meme-maker/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Searches meme templates, suggests formats, and generates local SVG/PNG or Imgflip-hosted image memes.

Overview

Meme Maker creates meme drafts from a curated template registry without bundling copyrighted template images. Its Node script searches template names, aliases, tags, and use cases, ranks templates for a described joke, and renders captioned memes either locally as SVG or PNG or hosted through Imgflip. Local mode downloads template images from their source URL with a browser-like user agent, caches them under the user cache directory, and embeds them into the SVG output.

Capabilities

- Searches curated meme templates by name, alias, tag, or use case
- Suggests and ranks templates for a described topic
- Renders local SVG memes with embedded template images
- Renders local PNG memes using sharp when available
- Produces hosted memes via the Imgflip `caption_image` API
- Fetches current Imgflip top templates for research

Usage

- `{baseDir}/scripts/meme.mjs search "bad choice"` — search templates
- `{baseDir}/scripts/meme.mjs suggest "slow python image scripts"` — suggest a template
- `{baseDir}/scripts/meme.mjs render drake --text "before" --text "after" --out /tmp/meme.svg` — render local SVG
- `{baseDir}/scripts/meme.mjs render drake --text "before" --text "after" --out /tmp/meme.png` — render local PNG
- `{baseDir}/scripts/meme.mjs render drake --service imgflip --text "before" --text "after"` — render hosted on Imgflip
- `{baseDir}/scripts/meme.mjs refresh --limit N --json` — fetch current top templates

When to use

Use when the user wants a meme image, help choosing a meme format for a joke, or a captioned meme generated as a local file or a hosted Imgflip URL.

Requirements & notes

Requires Node.js; local PNG output additionally uses sharp when available, and Imgflip mode needs IMGFLIP_USER and IMGFLIP_PASS unless --username and --password are supplied. Do not ship template image files with the skill, never use shared or hardcoded Imgflip credentials, and keep Know Your Meme lookups out of the render hot path. The curated 20-template registry lives in references/templates.json.

node-connect

Location	/usr/lib/node_modules/openclaw/skills/node-connect/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Diagnoses OpenClaw node pairing failures by verifying the network route, gateway configuration, and device authentication before fixing pairing.

Overview

Node Connect is a troubleshooting skill for OpenClaw Android, iOS, and macOS node pairing. It helps when nodes fail to pair, refuse to connect, or report authentication errors. The method is to identify the one real network route from node to gateway (same machine, same LAN, Tailscale tailnet, or public URL), verify that OpenClaw is actually advertising that route in its setup code, and only then fix pairing or authentication. The skill provides canonical config checks, a root-cause map for common error messages, and fast heuristics.

Capabilities

- Diagnose pairing, route, auth, and connection failures step by step
- Inspect gateway bind, Tailscale, remote URL, and auth configuration values
- Generate and read the setup code payload with `openclaw qr --json`
- Map app error text to root causes like loopback binding or expired tokens
- Approve pending device pairing requests with exact device commands

Usage

- `openclaw qr --json` — preferred check; reports `gatewayUrl` and `urlSource`
- `openclaw config get gateway.bind` — shows which network interface the gateway binds
- `openclaw devices list` — lists pending pairing requests awaiting approval
- `openclaw devices approve <requestId>` — approves a specific pending device pairing
- `tailscale status --json` — verifies Tailscale tailnet membership when relevant

When to use

Use when a phone or desktop node cannot pair with the gateway, when setup codes produce errors like `pairing required`, `bootstrap token invalid or expired`, or `unauthorized`, or when the QR code advertises the wrong address for the intended network.

Requirements & notes

Requires the OpenClaw CLI and a node app on Android, iOS, or macOS. Tailscale checks need `tailscale` installed and the gateway host actually on the tailnet. Always ask for the exact app error text and the intended route before guessing fixes.

node-inspect-debugger

Location	/usr/lib/node_modules/openclaw/skills/node-inspect-debugger/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Debugs Node.js with <code>node inspect</code> , the Chrome DevTools Protocol, breakpoints, heap snapshots, and CPU profiles.

Overview

Node Inspect Debugger covers Node.js debugging that needs inspector access: hidden locals, async hangs, flaky tests, child processes, startup races, memory growth, and CPU hot paths. It defaults to the built-in `node inspect`

REPL and escalates to the Chrome DevTools Protocol (CDP) only when scripted breakpoints, automated state capture, heap snapshots, or CPU profiles are required. The skill also documents OpenClaw-specific paths for debugging the CLI and its test runner.

Capabilities

- Pause on entry or attach to running processes with `node inspect`
- Set line, function, and conditional breakpoints from the debugger REPL
- Script breakpoints and state capture over the Chrome DevTools Protocol
- Capture CPU profiles and heap snapshots for performance analysis
- Debug TypeScript sources with source maps and `tsx` import loading

Usage

- `node inspect path/to/script.js` — launches paused on entry for step debugging
- `node --inspect-brk --import tsx path/to/script.ts` — debug TypeScript before code runs
- `kill -SIGUSR1 <pid>` then `node inspect -p <pid>` — attach to a running process
- `curl -s http://127.0.0.1:9229/json/list | jq` — list inspector targets and ports
- `OPENCLAW_VITEST_MAX_WORKERS=1 node --inspect-brk scripts/run-vitest.mjs <file>` — debug one test file

When to use

Use when a Node.js program hangs, mutates state unexpectedly, fails only in certain test runs, leaks memory, or shows CPU hot paths that ordinary logging cannot explain.

Requirements & notes

Requires the `node` binary. Programmatic CDP needs `chrome-remote-interface`, installed outside the project repo unless the project already depends on it. Prefer `127.0.0.1` inspector binds; the default port is 9229, so give parallel targets unique ports, and remember `--inspect` does not pause execution.

notion

Location	<code>/usr/lib/node_modules/openclaw/skills/notion/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Manages Notion pages, data sources, files, comments, and Workers via the official <code>ntn</code> CLI with a <code>curl</code> fallback.

Overview

The `notion` skill wraps the official Notion CLI (`ntn`) for creating and editing pages as Markdown, querying data sources, uploading files, and calling the raw Notion REST API with typed input syntax. It prefers `ntn` because the CLI sets the `Authorization` and `Notion-Version` headers automatically, falling back to plain `curl` only when the CLI is unavailable or a raw request is clearer. The skill also covers Notion Workers automation and current API version notes.

Capabilities

- Create, read, update, and trash Notion pages with Markdown content
- Resolve database IDs and query data sources with sort and filter expressions
- Upload files and external URLs as Notion file objects
- Send raw API calls with typed body fields, query parameters, and headers
- Deploy and inspect Notion Workers with `run logs`

Usage

- `ntn login` — authenticates the CLI for interactive or headless use
- `ntn pages create --parent page:<page-id> --content '# Title\n\nBody'` — creates a Markdown page
- `ntn datasources query <data-source-id> --filter '{"property":"Done","checkbox":{"equals":true}}'` — filtered data source query
- `ntn api v1/search query=roadmap page_size:=10` — raw API search call
- `ntn files create < image.png` — uploads a file from stdin

When to use

Use when the user wants to read, create, or edit Notion pages or databases, query Notion content, upload files to Notion, or automate Notion with Workers.

Requirements & notes

Requires the `ntn` binary (installed via `npm install -g ntn`) or `curl`, plus a Notion integration token in `NOTION_API_TOKEN` or CLI login. Workers may require a Business or Enterprise plan with workspace enablement. Current API version is 2026-03-11; use `in_trash` rather than `archived`.

python-debugpy

Location	/usr/lib/node_modules/openclaw/skills/python-debugpy/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Debugs Python with <code>pdb</code> , <code>breakpoint()</code> , post-mortem inspection, and <code>debugpy</code> remote attach to running processes.

Overview

`python-debugpy` covers interactive Python debugging across four tools: the `breakpoint()` builtin for quick source-level stops, `python3 -m pdb` for launch-from-start debugging, `pdb` with `continue` for stopping at unhandled exceptions, and `debugpy` for remote or headless attach to running processes. The skill provides post-mortem recipes, the essential `pdb` commands for flow, stack, and values, and safety rules for debugger ports and cleanup before commits.

Capabilities

- Launch scripts under `pdb` or stop at unhandled exceptions
- Attach to running PIDs and headless services with `debugpy`
- Inspect stack frames, locals, and expressions interactively
- Set conditional breakpoints and mutate state mid-execution
- Perform post-mortem inspection after an exception

Usage

- `python3 -m pdb path/to/script.py arg1` — debugs a script from the start
- `python3 -m pdb -c continue path/to/script.py` — stops at the first unhandled exception
- `python3 -m debugpy --listen 127.0.0.1:5678 --wait-for-client path/to/script.py` — waits for a DAP client
- `python3 -m debugpy --listen 127.0.0.1:5678 --pid <pid>` — attaches to a running process
- `rg -n 'breakpoint\(|pdb\.set_trace|debugpy\.'` --type py — checks for leftover debugger calls

When to use

Use when Python code has hidden locals, confusing state mutation, failing tests, subprocesses, long-running services, or needs remote or headless debugging attach.

Requirements & notes

Requires `python3`; `debugpy` must be installed in the active environment. Disable parallel test workers under `pdb`, bind debug servers to `127.0.0.1`, and treat `--pid attach` as injection that needs explicit approval on production targets. If PID attach fails on Linux, check `ptrace` and container privileges first.

spike

Location	/usr/lib/node_modules/openclaw/skills/spike/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Run throwaway prototypes to validate feasibility, compare approaches, and report an evidence-backed verdict before real builds.

Overview

Spike is a disciplined prototyping workflow for answering concrete feasibility questions before committing to production work. It loops through five stages: state the question, research credible approaches, build the smallest runnable artifact, stress one edge case, and deliver a verdict of `VALIDATED`, `PARTIAL`, or `INVALIDATED`. Multi-spike ideas are split into two to five independent questions with the riskiest one tested first.

Capabilities

- Build minimal runnable artifacts: CLI, tiny HTML, one endpoint, or focused test
- Compare approaches with equal inputs and measured dimensions
- Stress-test one edge case or failure mode per spike
- Deliver evidence-based verdicts citing exact commands and measurements

- Check external dependency health before recommending adoption
- Split large ideas into ordered independent feasibility questions

Usage

- `.tmp/openclaw-spikes/<slug>` — default workspace for spike artifacts
- `spikes/<NNN-slug>/` with `README.md` — optional repo-local spike layout
- `## Verdict: VALIDATED | PARTIAL | INVALIDATED` — required verdict header format
- `Question: ... Evidence: ... What worked: ... Recommendation: ...` — verdict body fields

When to use

Load when the user says "spike this", "quick prototype", "is this possible", "compare A/B", or "before we build" and wants feasibility evidence rather than production code.

Requirements & notes

No special tooling is required beyond a runnable environment for the prototype. Do not use this skill when docs or code reading can answer the question, avoid package sprawl and production cleanup, and never merge spike code into production without a normal rewrite. When external dependencies are evaluated, check recent releases or commits, docs, license, and install friction.

taskflow

Location	<code>/usr/lib/node_modules/openclaw/skills/taskflow/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Coordinate multi-step detached work as one durable TaskFlow job with owner context, persisted state, waits, and linked child tasks.

Overview

TaskFlow manages jobs that must outlive a single prompt or detached run while keeping one owner session, one return context, and one place to inspect or resume work. It owns flow identity, the owner session and requester origin, `currentStep`, `stateJson`, `waitJson`, linked child tasks, lifecycle states such as finish, fail, cancel, waiting, and blocked, plus revision tracking for conflict-safe mutations. Branching and business logic stay in the authoring layer, not the runtime.

Capabilities

- Create managed flows and run linked child tasks under one owner
- Persist small state bags in `stateJson` between steps
- Model external waits with `waitJson` and resume cleanly
- Track revisions so every mutation is conflict-checked
- Cancel flows with or without cancelling active linked child tasks
- Summarize child-task health with `getTaskSummary(flowId)`

Usage

- `api.runtime.tasks.flow.fromToolContext(ctx)` — bind a flow from trusted tool context
- `taskFlow.createManaged({...})` — create a managed flow with goal and initial step
- `taskFlow.runTask({...})` — link and start a detached child task
- `taskFlow.setWaiting({...})` — pause pending a person or external system
- `taskFlow.resume({...})` — continue after the wait resolves
- `taskFlow.finish({...})` — close out the flow successfully

When to use

Load for multi-step background work with one owner, jobs waiting on detached ACP or subagent tasks, or work needing persisted state that survives restarts and revision conflicts cleanly.

Requirements & notes

Available to plugin and tool code via the `api.runtime.tasks.flow` entrypoint (`api.runtime.taskFlow` remains an alias); bind with `fromToolContext` or `bindSession`. Carry forward `flow.revision` after every successful mutation since all mutators are revision-checked. Managed flows suit code-owned orchestration; one-task mirrored flows are created by core runtime for detached ACP or subagent work.

taskflow-inbox-triage

Location	/usr/lib/node_modules/openclaw/skills/taskflow-inbox-triage/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Example TaskFlow pattern for inbox triage with intent routing, Slack reply waits, and deferred end-of-day summaries.

Overview

This skill is a concrete, worked example of using TaskFlow for inbox triage without turning the core runtime into a domain-specific language. One owner flow classifies new inbox items, routes business threads to Slack and waits for replies, notifies the owner immediately about personal items, and buckets everything else for an end-of-day summary. It documents the intended `stateJson` and `waitJson` shapes alongside minimal runtime calls.

Capabilities

- Triage a batch of inbox items under one managed flow
- Route items by intent: business, personal, or deferred
- Persist routing state in simple `stateJson` buckets
- Block on Slack thread replies with structured `waitJson`
- Resume the flow when classification or human input completes
- Finish once the batch has been fully routed

Usage

- `taskFlow.createManaged({...})` — open one flow per inbox batch
- `taskFlow.runTask({...})` — run the classifier child task
- `taskFlow.setWaiting({...})` — wait on a Slack reply via `waitJson`
- `taskFlow.resume({...})` — continue routing after the reply lands
- `skills/taskflow/examples/inbox-triage.lobster` — related runnable example

When to use

Load when building inbox triage, message routing, or reply-waiting automations on top of TaskFlow, or when a reference pattern for owner-session flows is needed.

Requirements & notes

Requires the TaskFlow runtime API (`api.runtime.tasks.flow`) and a detached-task runtime such as ACP for child sessions. The suggested `stateJson` keeps three buckets: `businessThreads`, `personalItems`, and `eodSummary`; a Slack wait uses `kind: "reply"` with a `threadKey`. See `skills/taskflow/examples/inbox-triage.lobster` for the full example.

video-frames

Location	/usr/lib/node_modules/openclaw/skills/video-frames/SKILL.md
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Extract single frames or quick inspection thumbnails from videos at chosen timestamps using <code>ffmpeg</code> .

Overview

Video Frames is a thin `ffmpeg` wrapper for extracting a single frame from a video or producing a quick thumbnail for inspection. A bundled `frame.sh` script takes a video path, an optional timestamp, and an output path, so the agent can answer "what is happening around here?" questions visually. The skill favors JPG for quick sharing and PNG for crisp UI frames.

Capabilities

- Extract the first frame of a video to an image file
- Extract a frame at a specific timestamp with `--time`
- Produce quick thumbnails for visual inspection
- Choose JPG for quick sharing or PNG for crisp UI frames
- Wrap `ffmpeg` frame extraction in a simple helper script

Usage

- `{baseDir}/scripts/frame.sh /path/to/video.mp4 --out /tmp/frame.jpg` — extract the first frame

- `{baseDir}/scripts/frame.sh /path/to/video.mp4 --time 00:00:10 --out /tmp/frame-10s.jpg` — extract a frame at a timestamp
- `ffmpeg` — the underlying extractor the skill depends on

When to use

Load when the user wants a frame, still, or thumbnail extracted from a video at a given time or from the start.

Requirements & notes

Requires `ffmpeg`, installable via Homebrew. The bundled helper script lives at `scripts/frame.sh` under the skill base directory. Prefer `--time` for "what is happening around here?" questions; use a `.jpg` for quick sharing and a `.png` for crisp UI frames.

weather

Location	<code>/usr/lib/node_modules/openclaw/skills/weather/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Active in skill catalog
Summary	Current weather, rain and temperature checks, and forecasts from <code>wtrr.in</code> via <code>curl</code> for locations and travel planning.

Overview

This skill fetches weather information from the public `wtrr.in` service with plain `curl` calls. It supports compact one-line conditions, single-day views, full forecasts, custom format strings, and JSON output, with locations given as city, region, airport code, or coordinates. The skill defines clear boundaries, pointing users to official services for severe alerts and to archives for historical climate data.

Capabilities

- Get compact current conditions for any location
- View current-only, one-day, or full forecast panels
- Build custom format strings for location, condition, and temperature
- Report feels-like temperature, wind, humidity, and precipitation
- Fetch structured JSON weather output
- Check destinations for travel planning

Usage

- `curl "wtrr.in/London?format=3"` — compact current condition line
- `curl "wtrr.in/London?0"` — current weather only
- `curl "wtrr.in/London"` — full multi-day forecast
- `curl "wtrr.in/London?1"` — one-day view
- `curl "wtrr.in/London?format=%l: +%c+%t, +feels+%f, +rain+%p, +wind+%w"` — custom format string
- `curl "wtrr.in/London?format=j1"` — JSON output

When to use

Load when the user asks about current weather, rain or temperature checks, forecasts, or weather considerations for travel planning.

Requirements & notes

Requires only `curl` and a location as city, region, airport code, or coordinates. For severe alerts, aviation or marine weather, and official decisions, use official local weather services; for historical climate or weather use an archive or API rather than `wtrr.in`, and for hyper-local microclimates prefer local sensors.

Part 4 - Additional Installed Skills (Not in Active Catalog)

46 skills, alphabetical.

1password

Location	/usr/lib/node_modules/openclaw/skills/1password/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Sets up and uses the 1Password CLI (op) for sign-in, desktop app integration, and secret injection via tmux sessions.

Overview

This skill covers installing and operating the 1Password CLI following the official get-started flow rather than guessing commands. Sign-in depends on the 1Password desktop app integration being enabled and the app unlocked. A hard requirement is that every `op` command runs inside a fresh tmux session with a unique socket and session name, because the shell tool allocates a fresh TTY per command and would otherwise hit re-prompts and failures. Access is verified with `op whoami` before any secret is read, and multiple accounts are handled with `--account` or `OP_ACCOUNT`.

Capabilities

- Verifies CLI installation and version
- Signs in via desktop app authorization inside tmux
- Reads vault items after identity verification
- Injects secrets with `op run` and `op inject` without writing to disk
- Handles multiple 1Password accounts per invocation
- Diagnoses sign-in failures and re-authorizes on demand

Usage

- `op --version` — verifies the CLI is installed
- `op signin` — authorizes via the 1Password desktop app prompt
- `op whoami` — confirms signed-in identity before secret access
- `op vault list` — lists available vaults after sign-in
- `op run / op inject` — injects secrets into commands or files
- `op account add` — adds an account when app integration is unavailable

When to use

Load when the user needs 1Password sign-in, vault access, credential lookup, or secrets injected into commands, configs, or files.

Requirements & notes

Requires the `op` binary (brew formula `1password-cli`), the 1Password desktop app unlocked with integration enabled, and tmux. Never paste secrets into logs, chat, or code, and never run `op` outside tmux; stop and ask if tmux is unavailable.

apple-notes

Location	/usr/lib/node_modules/openclaw/skills/apple-notes/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Creates, views, edits, deletes, searches, moves, and exports Apple Notes from the terminal via the memo CLI on macOS.

Overview

The memo CLI (`memo notes`) manages Apple Notes directly from the terminal. It covers listing and fuzzy-searching notes, creating notes interactively or with a quick title, editing and deleting via interactive selection, moving notes between folders, and exporting to HTML or Markdown with Mistune handling Markdown processing. Installation is through a Homebrew tap or a pip install from a cloned repository. The tool is macOS-only and may prompt for Automation access to Notes.app.

Capabilities

- Lists all notes and filters them by folder

- Fuzzy-searches notes by query string
- Creates notes interactively or with a given title
- Edits and deletes notes via interactive selection
- Moves notes between folders
- Exports selected notes to HTML or Markdown

Usage

- `memo notes` — lists all notes
- `memo notes -f "Folder Name"` — filters notes by folder
- `memo notes -s "query"` — fuzzy-searches notes
- `memo notes -a "Note Title"` — quick-adds a note with a title
- `memo notes -e` — interactively edits a selected note
- `memo notes -ex` — exports the selected note to HTML or Markdown

When to use

Load when the user wants to manage Apple Notes on macOS: create, view, search, edit, delete, move, or export notes.

Requirements & notes

macOS only. Install with `brew tap antoniorodr/memo && brew install antoniorodr/memo/memo` or `pip install .` from a repo clone. Grant Automation access to Notes.app under System Settings if prompted. Notes containing images or attachments cannot be edited, and interactive prompts may require terminal access.

apple-reminders

Location	/usr/lib/node_modules/openclaw/skills/apple-reminders/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Lists, adds, edits, completes, and deletes Apple Reminders and reminder lists from the terminal using <code>remindctl</code> on macOS.

Overview

The `remindctl` CLI manages Apple Reminders directly from the terminal so tasks sync to iPhone and iPad. It covers date-based views (today, tomorrow, this week, overdue, specific dates), reminder list management, creating reminders with due dates, and completing or deleting by ID. The skill draws a firm line against misuse: OpenClaw-scheduled alerts, calendar events, and project task tracking belong to other tools, and ambiguous "remind me" requests require clarifying whether Apple Reminders or an in-chat alert is wanted.

Capabilities

- Views today's, tomorrow's, this week's, and overdue reminders
- Creates, shows, and deletes reminder lists
- Adds reminders with titles, target lists, and due dates
- Completes and deletes reminders by ID
- Outputs JSON, TSV, or count-only formats
- Accepts natural and ISO 8601 date formats

Usage

- `remindctl today` — shows today's reminders
- `remindctl add --title "Call mom" --list Personal --due tomorrow` — adds a dated reminder
- `remindctl list Projects --create` — creates a new reminders list
- `remindctl complete 1 2 3` — completes reminders by ID
- `remindctl delete 4A83 --force` — deletes a reminder by ID
- `remindctl today --json` — JSON output for scripting

When to use

Load when the user explicitly mentions the Reminders app or wants personal to-dos with due dates that sync to iOS devices.

Requirements & notes

macOS only. Install with `brew install steipete/tap/remindctl`, grant Reminders permission when prompted, and use `remindctl status` or `remindctl authorize` to check access. Clarify intent when "remind me" could mean an OpenClaw alert instead, which should use the cron tool.

bear-notes

Location	/usr/lib/node_modules/openclaw/skills/bear-notes/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Creates, reads, searches, and manages Bear notes on macOS through the grizzly CLI with token-based operations.

Overview

The grizzly CLI drives the Bear note app on macOS through x-callback-url, creating notes from piped input, opening notes by internal ID, appending text, listing tags, and browsing by tag. Some operations (add-text, tags, open-note --selected) need a Bear API token obtained from Bear's Help menu and stored at `~/.config/grizzly/token`. Configuration resolution order is CLI flags, then environment variables like `GRIZZLY_TOKEN_FILE`, then a local `.grizzly.toml`, then `~/.config/grizzly/config.toml`. Bear must be running for any command to work.

Capabilities

- Creates notes with titles and tags from piped input
- Opens and reads notes by internal Bear ID
- Appends text to existing notes
- Lists all Bear tags
- Browses and searches notes via tag opening
- Previews x-callback URLs without executing them

Usage

- `grizzly create --title "My Note" --tag work` — creates a note with content from stdin
- `grizzly open-note --id "NOTE_ID" --enable-callback --json` — reads a note by ID
- `grizzly add-text --id "NOTE_ID" --mode append --token-file ~/.config/grizzly/token` — appends text
- `grizzly tags --enable-callback --json --token-file ~/.config/grizzly/token` — lists all tags
- `grizzly open-tag --name "work" --enable-callback --json` — opens notes by tag

When to use

Load when the user wants to create, read, append to, search, or tag Bear notes on macOS.

Requirements & notes

Requires the Bear app installed and running plus the grizzly binary from `go install github.com/tylerwince/grizzly/cmd/grizzly@latest`. A Bear API token saved to `~/.config/grizzly/token` is mandatory for add-text, tags, and open-note --selected operations.

blogwatcher

Location	/usr/lib/node_modules/openclaw/skills/blogwatcher/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Monitors blogs and RSS/Atom feeds for new articles, tracks read state, and manages the watch list via the blogwatcher CLI.

Overview

The blogwatcher CLI tracks blog and RSS/Atom feed updates from the terminal. Users add tracked blogs by name and URL, scan all feeds for new articles, list discovered articles, and manage read state individually or in bulk. Scan output reports each source's feed type and counts of found and new articles, making it easy to see what changed since the last check. The tool is installed with the Go toolchain and each subcommand supports `--help` for flags and options.

Capabilities

- Adds and removes tracked blogs by name and URL
- Scans all tracked feeds for new RSS or Atom articles
- Lists discovered articles across tracked blogs
- Marks individual or all articles as read

- Reports per-source scan statistics and new article totals

Usage

- `blogwatcher add "My Blog" https://example.com` — adds a blog to track
- `blogwatcher blogs` — lists tracked blogs
- `blogwatcher scan` — scans feeds for new articles
- `blogwatcher articles` — lists found articles
- `blogwatcher read 1` — marks article 1 as read
- `blogwatcher read-all` — marks all articles read

When to use

Load when the user wants to follow blogs or RSS/Atom feeds, check for new posts, review article lists, or manage read status.

Requirements & notes

Requires the `blogwatcher` binary, installed with `go install`

`github.com/Hyaxia/blogwatcher/cmd/blogwatcher@latest`. Use `blogwatcher <command> --help` to discover available flags and options for each subcommand.

blucli

Location	<code>/usr/lib/node_modules/openclaw/skills/blucli/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Controls Bluesound and NAD audio players via the blu CLI: device discovery, playback, grouping, volume, and TuneIn radio.

Overview

The blu CLI (`blucli`) controls BluOS-based Bluesound and NAD players on the local network. It covers device discovery, status reporting, playback control, volume adjustment, multi-room grouping, and TuneIn search and playback. Target selection follows a fixed priority: the `--device` flag with an ID, name, or alias, then the `BLU_DEVICE` environment variable, then the configured default. The skill advises confirming the target device before changing playback and preferring `--json` output for scripted use.

Capabilities

- Discovers players on the local network
- Reports per-device playback status
- Controls play, pause, and stop
- Sets volume levels
- Manages multi-room player groups
- Searches and plays TuneIn stations

Usage

- `blu devices` — lists available players to pick a target
- `blu --device <id> status` — shows the target player's status
- `blu play` — starts playback (also `blu pause`, `blu stop`)
- `blu volume set 15` — sets the volume to a specific level
- `blu group status` — shows grouping state (also `blu group add`, `blu group remove`)
- `blu tunein search "query"` — searches TuneIn (also `blu tunein play "query"`)

When to use

Load when the user wants to control Bluesound or NAD players: playback, volume, multi-room grouping, or TuneIn radio.

Requirements & notes

Requires the `blu` binary from `go install github.com/steipete/blucli/cmd/blu@latest` and reachable players on the network. Always confirm the target device before changing playback state.

camsnap

Location	/usr/lib/node_modules/openclaw/skills/camsnap/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Captures snapshots, video clips, and motion events from configured RTSP/ONVIF cameras via the camsnap CLI.

Overview

The camsnap CLI grabs snapshots, clips, and motion events from configured cameras. Cameras are registered with a name, host, and credentials and stored in `~/.config/camsnap/config.yaml`, while discovery and doctor commands help find devices and diagnose the setup. Motion watching supports a configurable threshold and an action to run on detection, and clips are recorded with a duration parameter. The skill recommends a short test capture before committing to longer recordings.

Capabilities

- Adds named cameras with host and credentials
- Discovers cameras and prints detailed device info
- Takes single-frame snapshots to image files
- Records timed video clips to files
- Watches for motion with threshold-triggered actions
- Diagnoses configuration with doctor probes

Usage

- `camsnap add --name kitchen --host 192.168.0.10 --user user --pass pass` — registers a camera
- `camsnap discover --info` — discovers cameras with details
- `camsnap snap kitchen --out shot.jpg` — captures a snapshot to a file
- `camsnap clip kitchen --dur 5s --out clip.mp4` — records a short clip
- `camsnap watch kitchen --threshold 0.2 --action '...'` — runs motion watching
- `camsnap doctor --probe` — probes and diagnoses the setup

When to use

Load when the user wants camera snapshots, video clips, or motion monitoring from RTSP/ONVIF cameras.

Requirements & notes

Requires the camsnap binary (`brew formula steipete/tap/camsnap`) and `ffmpeg` on `PATH`. Camera definitions live in `~/.config/camsnap/config.yaml`. Prefer a short test capture before longer clips.

clawhub

Location	/usr/lib/node_modules/openclaw/skills/clawhub/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Searches, installs, updates, and publishes agent skills through the ClawHub CLI and its public skill registry.

Overview

This skill covers the ClawHub CLI for managing agent skills against the ClawHub registry at <https://clawhub.com>. It supports searching the registry, installing skills with optional pinned versions, updating via hash-based local file matching, listing installed skills, and publishing skill directories with slug, name, version, and changelog metadata. The registry and workdir can be overridden with `CLAWHUB_REGISTRY`, `--registry`, `--workdir`, `--dir`, or `CLAWHUB_WORKDIR`, and the default install directory is `./skills`. Note that in this environment a custom `clawhub-cli` skill is the active catalog entry rather than this built-in skill.

Capabilities

- Searches the registry for skills by keyword
- Installs skills by slug with optional pinned versions
- Updates one or all skills via hash-based matching
- Force-updates silently without interactive prompts
- Lists installed skills
- Publishes skill directories with changelogs

Usage

- `npm i -g clawhub` — installs the ClawHub CLI
- `clawhub login / clawhub whoami` — authenticates for publishing
- `clawhub search "postgres backups"` — searches the registry
- `clawhub install my-skill` — installs a skill by slug
- `clawhub update --all` — updates every installed skill
- `clawhub publish ./my-skill --slug my-skill --name "My Skill" --version 1.2.0 --changelog "Fixes + docs"` — publishes a skill

When to use

Load when installing, syncing, or publishing skills through the ClawHub registry. In this environment the custom `clawhub-cli` skill is the active one; this built-in skill is installed but inactive.

Requirements & notes

Requires the `clawhub` binary from the npm package `clawhub`. Publishing requires prior `clawhub login`. The `update` command hashes local files to resolve the matching version and upgrades to latest unless `--version` is set.

coding-agent

Location	<code>/usr/lib/node_modules/openclaw/skills/coding-agent/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Delegates large coding tasks to Codex, Claude Code, OpenCode, or Pi as background workers with explicit completion notifications.

Overview

This skill delegates background feature builds, PR reviews, large refactors, and issue-to-PR loops to external coding agents running as background workers; not for simple edits, read-only lookup, or OpenClaw state directories. Workers are always launched with `background: true`; Codex, Pi, and OpenCode need `pty: true` while Claude Code runs without a PTY. Every worker prompt carries a notification block with a real message route, and workers report completion through `openclaw message send`. Long issue-to-PR work uses a GitHub issue as the durable spec and an isolated checkout, never `~/Projects/openclaw`.

Capabilities

- Launches Codex, Claude Code, OpenCode, or Pi as background workers
- Runs long issue-to-PR loops with GitHub issues as specs
- Appends real notification routes to every worker prompt
- Monitors workers through process list, poll, log, and kill
- Creates scratch git repos for standalone Codex builds
- Reports worker start, workdir, and sessionId to the user

Usage

- `codex exec - <"$PROMPT"` — launches Codex from a prompt file with pty and background
- `claude --permission-mode bypassPermissions --print <"$PROMPT"` — launches Claude Code in background
- `opencode run <"$PROMPT"` — launches OpenCode with pty and background
- `pi -p "$(cat "$PROMPT")"` — launches Pi with pty and background
- `openclaw message send --channel <channel> --target '<target>' --message '<brief result>'` — worker completion report

When to use

Load for background feature builds, PR reviews, large refactors, or issue-to-PR loops. Do not load for simple edits, read-only code lookup, or work inside OpenClaw state directories.

Requirements & notes

Requires at least one of `claude`, `codex`, `opencode`, or `pi`, plus `skills.entries.coding-agent.enabled` in config. Capture a real notification route before spawning, write prompts to a temp file first, and never run workers or checkout branches in `~/Projects/openclaw`.

discord

Location	/usr/lib/node_modules/openclaw/skills/discord/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Runs Discord message-tool operations: send, read, edit, delete, react, poll, pin, thread, search, presence, and media.

Overview

This skill documents how to perform Discord operations through the message tool with channel set to discord; there is no separate Discord tool. It covers outbound text and media messages, Components v2 rich UI, reactions, polls, pinning, thread creation, guild-wide search, and presence updates. The skill emphasizes explicit identifiers (guildId, channelId, messageId, userId), multi-account support via accountId, and strict respect for configured action permission gates.

Capabilities

- Sends text and media messages to channels or users, optionally silent
- Edits and deletes messages identified by channel and message IDs
- Adds reactions, creates polls, and pins messages
- Creates threads from messages and searches guild content
- Renders rich UI with Components v2 and sets bot presence

Usage

- { "action": "send", "channel": "discord", "to": "channel:123", "message": "hello", "silent": true } — send a silent message
- { "action": "read", "channel": "discord", "to": "channel:123", "limit": 20 } — read recent channel messages
- { "action": "edit", "channel": "discord", "channelId": "123", "messageId": "456", "message": "fixed typo" } — edit a message
- { "action": "poll", "channel": "discord", "to": "channel:123", "pollQuestion": "Lunch?", "polloption": ["Pizza", "Sushi"] } — create a poll
- { "action": "thread-create", "channel": "discord", "channelId": "123", "messageId": "456", "threadName": "bug triage" } — start a thread
- { "action": "set-presence", "channel": "discord", "activityType": "playing", "activityName": "OpenClaw", "status": "online" } — set presence (often gated)

When to use

Use when the task involves Discord: sending, reading, editing, or deleting messages, reacting, polling, pinning, threading, searching guilds, attaching media, or updating bot presence.

Requirements & notes

Requires the channels.discord.token configuration; all actions must respect the channels.discord.actions.* permission gates, with presence often gated separately. Multi-account setups must pass accountId, send targets use the "channel:<id>" or "user:<id>" form, and user mentions use the <@USER_ID> form. Avoid Markdown tables in outbound messages, and do not mix v2 components with legacy embeds.

eightctl

Location	/usr/lib/node_modules/openclaw/skills/eightctl/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Controls Eight Sleep pods via the eightctl CLI: status, power, temperature, alarms, schedules, audio, and base.

Overview

eightctl is a command-line interface for controlling Eight Sleep smart mattress pods from the shell. It reports pod status, switches the pod on and off, and sets the target temperature for the night. It also manages alarms, sleep schedules, audio playback, and the adjustable base position. The skill notes the underlying API is unofficial and rate-limited, so repeated logins should be avoided.

Capabilities

- Reports current pod status and powers the pod on or off

- Sets target temperature values such as 20 degrees
- Lists, creates, and dismisses alarms
- Lists, creates, and updates sleep schedules
- Inspects audio state and controls playback and base angle

Usage

- `eightctl status` — show current pod status
- `eightctl on|off` — power the pod on or off
- `eightctl temp 20` — set the target temperature
- `eightctl alarm list|create|dismiss` — manage alarms
- `eightctl schedule list|create|update` — manage sleep schedules
- `eightctl audio state|play|pause` — control audio; `eightctl base info|angle` — base settings

When to use

Use when the user wants to check or change their Eight Sleep pod, including temperature, power state, alarms, sleep schedules, audio playback, or the adjustable base.

Requirements & notes

Requires the `eightctl` binary, installable through the Go module `github.com/steipete/eightctl/cmd/eightctl@latest`, and authentication via the config file `~/.config/eightctl/config.yaml` or the environment variables `EIGHTCTL_EMAIL` and `EIGHTCTL_PASSWORD`. The API is unofficial and rate-limited, so avoid repeated logins. Always confirm with the user before changing temperature or alarms.

gemin

Location	<code>/usr/lib/node_modules/openclaw/skills/gemini/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Runs Gemini CLI headless one-shot prompts, summaries, generation, and skills, hooks, MCP, or Gemma routing management.

Overview

This skill drives the Gemini CLI in non-interactive, one-shot mode for prompts, summaries, and generation tasks. Positional text starts interactive mode, so the skill consistently uses the `-p/--prompt` flag to stay headless, and `stdin` content appends to the prompt for piping files in. It also covers model selection, structured JSON output, and managing Gemini extensions, skills, hooks, and MCP server configuration through dedicated subcommands.

Capabilities

- Executes one-shot prompts in headless Gemini CLI mode
- Selects specific models for a prompt with the model flag
- Requests structured JSON output from prompts
- Pipes file content into prompts through `stdin`
- Manages extensions, skills, hooks, and MCP configuration

Usage

- `gemini -p "Answer this question..."` — run a one-shot prompt
- `gemini -m <model> -p "Prompt..."` — prompt with a chosen model
- `gemini -p "Return JSON" --output-format json` — request JSON output
- `cat notes.md | gemini -p "Summarize"` — append `stdin` content to the prompt
- `gemini --list-extensions` — list installed extensions
- `gemini mcp <command>` — manage MCP servers (also `gemini extensions`, `gemini skills`, `gemini hooks`)

When to use

Use when a task calls for a Gemini model invocation, a one-shot answer or summary, structured generation, or management of Gemini CLI extensions, skills, hooks, or MCP servers.

Requirements & notes

Requires the `gemini` binary, installable via the brew formula `gemini-cli`. If authentication is required, run `gemini` once interactively and complete the login flow before using headless mode. Avoid the `--yolo` flag for safety, since it disables

confirmations.

gh-issues

Location	/usr/lib/node_modules/openclaw/skills/gh-issues/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Automates issue-to-PR work: fetch GitHub issues, spawn background fix agents, open PRs, and process review comments.

Overview

gh-issues is a user-invocable skill for issue-to-PR automation built on the gh CLI, falling back to gh api only when a high-level command lacks a needed field. It resolves the target repository, fetches and filters issues, prevents duplicate work with claim files and branch checks, then spawns up to eight background workers that implement minimal fixes and open PRs. A watch loop re-fetches issues and review comments on an interval, and a reviews-only mode patches existing PRs from actionable review feedback.

Capabilities

- Fetches issues filtered by label, milestone, assignee, and state
- Prevents duplicate work via claim files and existing-branch checks
- Spawns up to 8 background workers building fix/issue-n branches
- Opens PRs with summary, verification, and issue references
- Handles actionable PR review comments in reviews-only mode
- Supports fork-based pushes, watch loops, and unattended cron runs

Usage

- `gh issue list --repo "$SOURCE_REPO" --state open --limit 10 --json number,title,labels,url,body,assignees,milestone` — fetch candidate issues
- `gh pr list --repo "$SOURCE_REPO" --search "$SOURCE_REPO#<n>" --state open --json number,url,title,headRefName` — detect existing PRs
- `git remote get-url origin` — infer the repository when not specified
- `gh auth status` — verify GitHub authentication
- `gh pr view <n> --repo "$SOURCE_REPO" --json url,headRefName,comments,reviews` — fetch PR review threads

When to use

Use when the user wants GitHub issues fixed automatically into pull requests, or actionable review comments on fix/issue-* PRs processed, including watch and scheduled cron operation.

Requirements & notes

Requires the git and gh binaries plus a GH_TOKEN, which can fall back to the gh-issues skill apiKey in the OpenClaw config; if authentication is unavailable, the skill stops and asks. Worker branches follow fix/issue-n conventions with conventional commit messages, claims expire after two hours, and execution stops on a dirty worktree unless the user confirms otherwise.

gifgrep

Location	/usr/lib/node_modules/openclaw/skills/gifgrep/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Searches GIF providers with CLI or TUI, downloads results, and extracts stills or contact sheets from clips.

Overview

gifgrep searches GIF providers such as Tenor and Giphy through a search, preview, download, and extract workflow designed for fast review and sharing. It can browse results in an interactive terminal UI or emit pipe-friendly URLs and JSON for scripting. Downloaded GIF clips can be reduced to a single still frame at a chosen timestamp or to a multi-frame PNG contact sheet suitable for documentation, pull requests, and chat.

Capabilities

- Searches Tenor and Giphy with selectable sources and API keys

- Browses results in an interactive TUI with still-frame previews
- Downloads GIFs to the Downloads folder and reveals them in Finder
- Extracts still frames at a timestamp from GIF clips
- Builds PNG contact sheets of sampled frames with tunable grids
- Prints JSON result arrays or single pipe-friendly fields

Usage

- `gifgrep cats --max 5` — search with a result limit
- `gifgrep tui "office handshake"` — interactive TUI search
- `gifgrep still ./clip.gif --at 1.5s -o still.png` — extract a still frame
- `gifgrep sheet ./clip.gif --frames 9 --cols 3 -o sheet.png` — build a contact sheet
- `gifgrep search --json cats | jq '[0].url'` — JSON output for scripting
- `gifgrep cats --download --max 1 --format url` — download results

When to use

Use when the user wants to find, preview, download, or convert animated GIFs, or extract still images and frame sheets from GIF clips for review or documentation.

Requirements & notes

Requires the `gifgrep` binary, installable via `brew formula steipete/tap/gifgrep` or the Go module `github.com/steipete/gifgrep/cmd/gifgrep@latest`. `GIPHY_API_KEY` is required for the Giphy source, `TENOR_API_KEY` is optional since a demo key is used otherwise, and `--thumbs` previews work only in Kitty or Ghostty terminals. Before using a GIF URL, verify it is actually animated and record attribution, license, and source.

github (built-in)

Location	<code>/usr/lib/node_modules/openclaw/skills/github/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	GitHub CLI reference for issues, PRs, CI and check logs, comments, reviews, releases, repos, and API queries.

Overview

This built-in skill is a comprehensive `gh` CLI reference covering pull requests, issues, CI runs, and direct API queries, while `git` stays reserved for local commits, branches, and pushes. It documents authentication checks and notes that the gateway HOME can differ from the operator HOME, in which case `GH_CONFIG_DIR` must be set for the gateway service. Deep code reviews are delegated to code-reading tools rather than the CLI, and URLs can be passed to `gh` commands directly.

Capabilities

- Lists, views, diffs, creates, and merges pull requests
- Manages issues: list, view, create, comment, and close
- Inspects workflow runs, check results, and failed logs
- Reruns failed CI jobs and reads run conclusions
- Queries the GitHub REST API with `jq` filters and caching
- Safely submits bodies containing code via body files

Usage

- `gh auth status` — check authentication (also `gh auth login`)
- `gh pr create --repo owner/repo --title "feat: title" --body-file /tmp/pr.md` — open a pull request
- `gh pr checks 55 --repo owner/repo` — view PR check results
- `gh issue list --repo owner/repo --state open --json number,title,labels,url` — list issues
- `gh run view <run-id> --repo owner/repo --log-failed` — show failed CI logs
- `gh api repos/owner/repo/pulls/55 --jq '.title, .state, .user.login'` — raw API query

When to use

Use when working with GitHub pull requests, issues, CI runs and checks, comments, reviews, releases, repositories, or custom REST API queries through `gh`.

Requirements & notes

Requires the gh binary, installable via brew formula gh or apt package gh. Note that the active github skill in this environment is a smaller custom skill at ~/.openclaw/skills/github/SKILL.md; this entry documents the fuller built-in upstream reference that is installed but not active. Use --body-file for comments or bodies containing backticks, shell snippets, environment names, or user text.

gog

Location	/usr/lib/node_modules/openclaw/skills/gog/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Google Workspace CLI for Gmail, Calendar, Drive, Contacts, Sheets, and Docs with one-time OAuth setup.

Overview

gog is a command-line tool covering Google Workspace services: Gmail, Calendar, Drive, Contacts, Sheets, and Docs. One-time setup loads OAuth client credentials and adds accounts with selected services, after which each service exposes search, read, and write subcommands. The skill documents email formatting conventions, calendar event color IDs, and Sheets value updates through JSON payloads. Confirm with the user before sending mail or creating events.

Capabilities

- Searches Gmail threads or individual messages with query syntax
- Sends plain, HTML, file-based, or stdin email and manages drafts
- Lists, creates, and updates calendar events with color IDs
- Searches Drive files and lists contacts
- Reads, updates, appends, and clears Sheets ranges
- Exports or prints Docs content as plain text

Usage

- gog auth add you@gmail.com --services gmail,calendar,drive,contacts,docs,sheets — add an account
- gog gmail search 'newer_than:7d' --max 10 — search mail threads
- gog gmail send --to a@b.com --subject "Hi" --body-file ./message.txt — send email from a file
- gog calendar create <calendarId> --summary "Title" --from <iso> --to <iso> — create an event
- gog sheets update <sheetId> "Tab!A1:B2" --values-json '["A","B"],["1","2"]' --input USER_ENTERED — write sheet values
- gog docs export <docId> --format txt --out /tmp/doc.txt — export a document

When to use

Use when the user needs to read or send Gmail, manage Calendar events, search Drive, look up Contacts, or read and edit Google Sheets and Docs.

Requirements & notes

Requires the gog binary, installable via brew from steipete/tap/gogcli, plus one-time OAuth setup with gog auth credentials pointing at a client_secret.json file. Set GOG_ACCOUNT to avoid repeating the account flag, prefer --json with --no-input for scripting, and use --body-file or heredocs for multi-paragraph mail since --body does not unescape newlines. In-place Docs editing is not supported by gog.

goplaces

Location	/usr/lib/node_modules/openclaw/skills/goplaces/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Queries Google Places for text search, place details, resolution, and reviews with human or scriptable JSON output.

Overview

goplaces is a modern Google Places API (New) command-line interface that prints human-readable output by default and JSON for scripts. It supports text search with open-now and minimum-rating filters, location biasing by coordinates and radius, and pagination through page tokens. It also resolves free-form place names to candidates and fetches place details including reviews, making it suitable both for interactive lookups and automated pipelines.

Capabilities

- Text-searches places with open-now, rating, limit, and type filters
- Biases search results by latitude, longitude, and radius in meters
- Paginates result sets with page tokens
- Resolves free-form location strings to place candidates
- Fetches place details with optional review content
- Emits scriptable JSON output on demand

Usage

- `goplaces search "coffee" --open-now --min-rating 4 --limit 5` — filtered text search
- `goplaces search "pizza" --lat 40.8 --lng -73.9 --radius-m 3000` — location-biased search
- `goplaces search "pizza" --page-token "NEXT_PAGE_TOKEN"` — paginate results
- `goplaces resolve "Soho, London" --limit 5` — resolve a place name
- `goplaces details <place_id> --reviews` — fetch details with reviews
- `goplaces search "sushi" --json` — machine-readable output

When to use

Use when the user wants to find places, venues, or restaurants, resolve ambiguous location names, or pull Google Places details and reviews programmatically.

Requirements & notes

Requires the `goplaces` binary, installable via `brew` from `steipete/tap/goplaces`, and a `GOOGLE_PLACES_API_KEY`; `GOOGLE_PLACES_BASE_URL` is optional for testing or proxying. Price levels range from 0 to 4 covering free through very expensive, the type filter sends only the first `--type` value because the API accepts one, and `--no-color` or `NO_COLOR` disables ANSI color.

healthcheck (built-in)

Location	<code>/usr/lib/node_modules/openclaw/skills/healthcheck/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Audits and hardens OpenClaw hosts with read-only security checks and staged, reversible hardening recommendations.

Overview

This host `healthcheck` skill assesses security risk on machines running OpenClaw, starting with read-only checks and then proposing staged hardening that will not break access. It first infers context such as OS, privilege level, access path, network exposure, gateway binding and auth, backup status, disk encryption, and automatic updates. It then asks the user to pick a risk posture of convenience, balanced, or strict, and reports findings with severity and evidence before a reversible hardening plan whose write actions run only after explicit approval.

Capabilities

- Runs cross-platform read-only security audit commands
- Audits OpenClaw gateway status, binding, and authentication
- Checks firewall, SSH, listening ports, and update settings per OS
- Verifies backup status and disk encryption state
- Produces severity-ranked findings and staged hardening plans
- Offers hardening items such as loopback binding and key-only SSH

Usage

- `openclaw security audit --deep` — run a deep OpenClaw security audit
- `openclaw gateway status --deep` — inspect gateway binding and status
- `openclaw doctor` — run environment diagnostics
- `lsof -nP -iTCP -sTCP:LISTEN` — list listening TCP ports on macOS
- `ss -ltnup || ss -ltnp` — list listening sockets on Linux
- `fdsetup status` — check FileVault disk encryption on macOS

When to use

Use when the user wants a security review or hardening of a host running OpenClaw, or an audit of network exposure, updates, backups, SSH, firewall, or gateway configuration.

Requirements & notes

Requires appropriate privileges for the platform checks and one-time permission before running read-only commands; state-changing actions always need confirmation first. Note that the active skill named healthcheck in this environment is a different custom water and sleep tracker at `~/.openclaw/skills/healthcheck/SKILL.md`. Never print secrets, never change SSH or firewall settings until the access path is confirmed, and never claim OpenClaw manages the OS firewall, SSH, or updates.

himalaya

Location	<code>/usr/lib/node_modules/openclaw/skills/himalaya/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	IMAP/SMTP mail from the shell with himalaya: list, read, search, compose, reply, forward, copy, move, delete.

Overview

himalaya is a CLI for IMAP/SMTP email that handles listing folders, searching envelopes, reading messages, and full message composition directly from the shell. Composition and attachments use MML syntax documented in a bundled message-composition reference, while account, auth, and backend setup live in a separate configuration reference. Organization commands cover copying, moving, deleting, and flagging messages across folders, with multi-account support.

Capabilities

- Lists folders and message envelopes with search filters
- Reads individual messages by identifier
- Composes, replies to, and forwards messages using MML
- Copies, moves, and deletes messages between folders
- Adds and removes flags such as seen
- Handles multiple accounts through an account flag

Usage

- `himalaya account configure` — run first-time account setup
- `himalaya folder list` — list mail folders
- `himalaya envelope list from alice@example.com subject invoice` — search envelopes
- `himalaya message read <id>` — read a message
- `himalaya message reply <id>` — reply to a message (also `himalaya message forward <id>`)
- `himalaya message move <id> <folder>` — move a message (also copy and delete)

When to use

Use when the user wants to read, search, send, or organize IMAP/SMTP email from the shell instead of a GUI mail client.

Requirements & notes

Requires the himalaya binary, installable via brew formula `himalaya`, and a configuration file at `~/.config/himalaya/config.toml` created with `himalaya account configure`. Prefer password managers or keyrings for credentials and never paste secrets into chat or logs. Confirm before sending, deleting, or moving many messages, use the account flag when multiple accounts exist, and read `references/configuration.md` and `references/message-composition.md` before advanced use.

imgsg

Location	<code>/usr/lib/node_modules/openclaw/skills/imgsg/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	iMessage and SMS CLI for listing chats, reading history, watching messages, and sending via Messages.app.

Overview

imsg is a macOS command-line tool for reading and sending iMessage and SMS through Messages.app. It lists chats, reads conversation history with optional attachment details, watches chats for new incoming messages, and sends text or file attachments to phone numbers or Apple IDs. The skill clearly defines scope: Telegram, Signal, WhatsApp, Discord, and Slack belong to their own channels, group chat management is unsupported, and bulk messaging always requires user confirmation first.

Capabilities

- Lists recent Messages.app chats as JSON
- Reads chat history with optional attachment information
- Watches a chat for new incoming messages
- Sends text and file attachments to numbers or Apple IDs
- Forces iMessage, SMS, or automatic service selection

Usage

- `imsg chats --limit 10 --json` — list recent chats
- `imsg history --chat-id 1 --limit 20 --json` — view chat history
- `imsg history --chat-id 1 --limit 20 --attachments --json` — history with attachments
- `imsg watch --chat-id 1 --attachments` — watch for new messages
- `imsg send --to "+14155551212" --text "Hello!"` — send a text message
- `imsg send --to "+14155551212" --text "Check this out" --file /path/to/image.jpg` — send with attachment

When to use

Use when the user explicitly asks to send an iMessage or SMS, read iMessage conversation history, or check recent Messages.app chats and recipients.

Requirements & notes

Requires macOS with Messages.app signed in, Full Disk Access for the terminal, and Automation permission for Messages.app when sending; the binary installs via brew from steipete/tap/imsg. The service flag can force imessage, sms, or auto selection. Always confirm recipient and message content before sending, never message unknown numbers without explicit approval, verify attachment paths exist, and rate-limit sends.

mcporter

Location	/usr/lib/node_modules/openclaw/skills/mcporter/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Lists, configures, authenticates, calls, and inspects MCP servers and tools over HTTP or stdio with mcporter.

Overview

mcporter is a CLI for working directly with MCP servers outside a chat client. It lists servers and their tool schemas, invokes tools using selector, function, full-URL, or stdio syntax, and passes JSON payloads for structured arguments. Beyond calls, it manages OAuth authentication and multi-server configuration, runs a connection daemon, and generates CLI wrappers plus TypeScript clients and types from server definitions.

Capabilities

- Lists MCP servers and their tool schemas
- Calls tools by selector, function syntax, URL, or stdio
- Passes structured JSON argument payloads to tools
- Manages OAuth authentication and server configuration
- Runs a background daemon for MCP connections
- Generates CLI wrappers and TypeScript clients or types

Usage

- `mcporter list` — list configured MCP servers
- `mcporter list <server> --schema` — show a server's tool schema
- `mcporter call linear.list_issues team=ENG limit:5` — call a tool with selector syntax
- `mcporter call --stdio "bun run ./server.ts" scrape url=https://example.com` — call a stdio server

- `mcporter auth <server | url> [--reset]` — OAuth authentication for a server
- `mcporter generate-cli --server <name>` — generate a CLI for a server

When to use

Use when the user needs to inspect, debug, authenticate, or invoke MCP servers and their tools, or to generate reusable CLI or TypeScript clients from them.

Requirements & notes

Requires the `mcporter` binary, installable as the node package `mcporter`. Configuration defaults to `./config/mcporter.json` and can be overridden with the `--config` flag; prefer `--output json` for machine-readable results. Further commands include `mcporter config list`, `get`, `add`, `remove`, `import`, `login`, and `logout`, `mcporter daemon start`, `status`, `stop`, and `restart`, and `mcporter inspect-cli` and `emit-ts` for generated code inspection.

model-usage

Location	/usr/lib/node_modules/openclaw/skills/model-usage/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Summarizes CodexBar local cost logs by model for Codex or Claude with current-model or full breakdowns.

Overview

Model usage reads CodexBar's local cost logs and summarizes spend per model for the Codex or Claude providers. Current-model mode takes the most recent daily row with model breakdowns and picks the highest-cost model, falling back to the last entry in `modelsUsed` when breakdowns are missing. The bundled Python summarizer accepts live CodexBar CLI output, exported JSON files, or `stdin`, so aggregation works anywhere Python runs even though live CodexBar CLI invocation is documented for macOS only.

Capabilities

- Summarizes per-model costs for the Codex or Claude provider
- Reports current-model usage from the latest daily row
- Produces full all-models cost breakdowns
- Reads live CLI JSON, exported files, or `stdin` input
- Outputs plain text or pretty-printed JSON
- Overrides automatic model detection with a model flag

Usage

- `python {baseDir}/scripts/model_usage.py --provider codex --mode current` — current model summary
- `python {baseDir}/scripts/model_usage.py --provider codex --mode all` — all-models summary
- `python {baseDir}/scripts/model_usage.py --provider claude --mode all --format json --pretty` — JSON output
- `codexbar cost --provider codex --format json > /tmp/cost.json` — export cost JSON
- `python {baseDir}/scripts/model_usage.py --input /tmp/cost.json --mode all` — summarize from a file

When to use

Use when the user asks what they are spending per AI model, which model dominates recent Codex or Claude costs, or wants a breakdown of CodexBar cost logs.

Requirements & notes

Live CodexBar CLI usage requires the `codexbar` binary on macOS, installable as `brew cask steipete/tap/codexbar`; on Linux and other platforms use the `input` flag with exported CodexBar JSON instead. Values are cost-only per model because CodexBar output does not split tokens by model. See `references/codexbar-cli.md` for CLI flags and cost JSON fields.

nano-pdf

Location	/usr/lib/node_modules/openclaw/skills/nano-pdf/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Applies natural-language edits to a specific page of a PDF using the nano-pdf CLI.

Overview

nano-pdf is a small command-line tool that edits a chosen page of a PDF according to a natural-language instruction. Each invocation targets one file, one page number, and one instruction string, such as changing a title or fixing a typo in a subtitle. This makes quick textual corrections on slide decks and documents possible without opening a full PDF editor.

Capabilities

- Edits a single PDF page from a natural-language instruction
- Rewrites text content such as titles and subtitles
- Fixes typos directly in existing PDF pages
- Produces an edited PDF for review and distribution

Usage

- `nano-pdf edit <file> <page> "<instruction>"` — general command form
- `nano-pdf edit deck.pdf 1 "Change the title to 'Q3 Results' and fix the typo in the subtitle"` — edit a slide page

When to use

Use when the user wants a quick text-level edit on one page of a PDF, such as retitling slides, correcting typos, or small wording fixes.

Requirements & notes

Requires the nano-pdf binary, installable via uv package nano-pdf. Page numbers may be interpreted as 0-based or 1-based depending on the tool version or configuration, so if the result looks off by one, retry with the other index. Always sanity-check the output PDF before sending it out.

obsidian

Location	<code>/usr/lib/node_modules/openclaw/skills/obsidian/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Works with Obsidian vaults through the official obsidian CLI: notes, tasks, links, properties, and plugins.

Overview

The obsidian skill uses the official Obsidian command line interface to read, search, create, and edit notes in an Obsidian vault, plus manage tasks, daily notes, properties, backlinks, and plugins. Because vault files are plain Markdown, direct file edits remain acceptable when they are safer or faster, while CLI commands handle Obsidian-aware operations like moves and property updates. The CLI connects to the running Obsidian desktop app.

Capabilities

- Open, read, search, create, append, and prepend vault notes
- Move and delete notes with Obsidian-aware resolution
- Manage daily notes and task checkboxes
- Read and set note properties and inspect backlinks and tags
- Reload plugins, view dev errors, and evaluate app scripts

Usage

- `obsidian search query="TODO" matches` — searches the vault for matching text
- `obsidian create path="Inbox/Idea.md" content="# Idea"` — creates a note with content
- `obsidian daily:append content="- [ ] Review inbox"` — appends a task to today's note
- `obsidian property:set file=Note name=status value=done` — sets a note property
- `obsidian eval "app.vault.getFiles().length"` — evaluates JavaScript against the app

When to use

Use when the user asks to find, write, reorganize, or annotate notes in an Obsidian vault, manage daily notes and tasks, or inspect links and properties.

Requirements & notes

Requires Obsidian 1.12.7 or newer with the command line interface enabled in Settings, the obsidian binary on PATH, and the Obsidian app running. macOS registration installs `/usr/local/bin/obsidian`; Linux copies to `~/.local/bin/obsidian`. Do not rely on third-party obsidian-cli unless the user explicitly asks.

openai-whisper

Location	/usr/lib/node_modules/openclaw/skills/openai-whisper/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Transcribes audio locally with the OpenAI Whisper command line tool, requiring no API key.

Overview

openai-whisper runs OpenAI's Whisper speech-to-text model fully locally through the whisper command line tool. It converts audio files into text, subtitles, or translations without sending data to any cloud service and without requiring an API key. Model weights download automatically on first use, and the skill documents the minimal flags for choosing models, tasks, and output formats.

Capabilities

- Transcribe audio files to text and subtitle output formats
- Translate speech into English text with the translate task
- Choose model sizes that trade speed against accuracy
- Write outputs to a chosen directory in formats like txt and srt

Usage

- `whisper /path/audio.mp3 --model medium --output_format txt --output_dir .` — transcribe an audio file to text
- `whisper /path/audio.m4a --task translate --output_format srt` — translate speech to English subtitles

When to use

Use when the user wants offline speech-to-text transcription or translation of audio files and prefers keeping the audio private on the local machine.

Requirements & notes

Requires the whisper binary (installable via the Homebrew formula `openai-whisper`). Models download to `~/.cache/whisper` on first run. The `--model` flag defaults to `turbo` on this install; use smaller models for speed and larger models for accuracy.

openai-whisper-api

Location	/usr/lib/node_modules/openclaw/skills/openai-whisper-api/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Transcribes audio through the OpenAI Audio Transcriptions API with models like gpt-4o-transcribe, diarize, and whisper-1.

Overview

openai-whisper-api sends audio to OpenAI's hosted `/v1/audio/transcriptions` endpoint through a bundled `transcribe.sh` script. It supports modern transcription models including `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, a speaker-diarizing variant, and the classic `whisper-1`, with options for language hints, vocabulary prompts, and JSON output. `OPENAI_BASE_URL` can point requests at an OpenAI-compatible proxy or local gateway instead of the hosted API.

Capabilities

- Transcribe audio files to plain text or JSON transcripts
- Produce speaker labels with the diarize model and auto chunking
- Bias transcription with language hints and vocabulary prompts
- Route requests to OpenAI-compatible proxies via base URL override

Usage

- `{baseDir}/scripts/transcribe.sh /path/to/audio.m4a` — transcribes to `<input>.txt` with the default model
- `{baseDir}/scripts/transcribe.sh /path/to/audio.ogg --model gpt-4o-mini-transcribe` — uses the mini model
- `{baseDir}/scripts/transcribe.sh /path/to/audio.ogg --model gpt-4o-transcribe-diarize --json` — speaker-labeled JSON output

- `{baseDir}/scripts/transcribe.sh /path/to/audio.m4a --prompt "Speaker names: Peter, Daniel" —`
vocabulary hint for names

When to use

Use when the user wants high-quality cloud speech-to-text for audio files and has an OpenAI API key, especially for speaker diarization or batch transcription.

Requirements & notes

Requires `curl` and `node` plus an `OPENAI_API_KEY` environment variable or a key configured in the active OpenClaw config file. Supported uploads include mp3, mp4, mpeg, mpga, m4a, wav, and webm, with a 25 MB limit on the hosted API. The diarize model sends `chunking_strategy=auto` and rejects `--prompt`.

openhue

Location	<code>/usr/lib/node_modules/openclaw/skills/openhue/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Controls Philips Hue lights, rooms, and scenes through the OpenHue CLI over a local Hue Bridge.

Overview

openhue is a command line tool for controlling Philips Hue smart lighting through a Hue Bridge on the local network. It turns lights and rooms on and off, adjusts brightness, color, and color temperature, and activates scenes with short preset commands. The skill defines clear boundaries: only Hue devices are supported, not other smart home brands, thermostats, TVs, or HomeKit scenes and Shortcuts.

Capabilities

- List all lights, rooms, and scenes registered on the Hue Bridge
- Toggle individual lights or whole rooms on and off
- Set brightness from 0 to 100 and color temperature from 153 to 500 mirek
- Apply named colors or hex RGB values to color-capable bulbs
- Activate scenes per room and run quick presets for common moods

Usage

- `openhue get light` — lists all lights registered on the bridge
- `openhue set light "Bedroom Lamp" --on --brightness 50` — turns on a light at half brightness
- `openhue set light "Bedroom Lamp" --on --temperature 300` — sets color temperature in mirek
- `openhue set room "Bedroom" --off` — turns off an entire room
- `openhue set scene "Relax" --room "Bedroom"` — activates a scene in a room

When to use

Use for requests like turning lights on or off, dimming rooms, setting scenes or movie mode, and adjusting Hue brightness, color, or color temperature.

Requirements & notes

Requires the openhue binary (Homebrew formula `openhue/cli/openhue-cli`) and a Hue Bridge on the local network. The first run requires pressing the button on the Hue bridge to pair. Colors only work on color-capable bulbs, and non-Hue smart devices are not supported.

oracle

Location	<code>/usr/lib/node_modules/openclaw/skills/oracle/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Runs one-shot second-model reviews of code and design with selected files via the Oracle CLI, using API or browser engines.

Overview

oracle packages a prompt together with a hand-picked file set for a single second-model review, debug, refactor, or design pass by a strong external model such as `gpt-5.5-pro`. Its output is advisory and must be verified against the code and tests. The workflow emphasizes previewing payload and token spend with dry runs first, choosing the browser engine for long Pro reasoning or the API engine for explicit API calls, and reattaching to stored sessions

instead of blindly re-running detached jobs.

Capabilities

- Preview prompts and token spend without spending tokens
- Attach files, directories, and globs with include and exclude patterns
- Run long browser-engine sessions with automatic attachment handling
- Reattach to stored sessions and render their results later
- Render prompts to the clipboard for manual pasting as a fallback

Usage

- `oracle --dry-run summary --files-report -p "<task>" --file "src/**"` — previews payload and token spend
- `oracle --engine browser --model gpt-5.5-pro -p "<task>" --file "src/**"` — long browser-engine run
- `oracle --render --copy -p "<task>" --file "src/**"` — copies the rendered prompt as fallback
- `oracle status --hours 72` — lists stored sessions for reattachment
- `oracle session <id> --render` — attaches to and renders a stored session

When to use

Use when the user wants an external second opinion on code, design, or debugging strategy, or a deep review that benefits from a large model reading selected files.

Requirements & notes

Requires the `oracle` binary (npm package `@steipete/oracle`; `npx -y` works as a fallback). The API engine needs `OPENAI_API_KEY` or Azure config; the browser engine supports GPT and Gemini only. Sessions store under `~/.oracle/sessions`, runs are one-shot, and secrets should never be attached.

ordercli

Location	/usr/lib/node_modules/openclaw/skills/ordercli/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Checks past food delivery orders and tracks active Foodora order status, with reorder and session import support.

Overview

`ordercli` is a command line tool for food delivery accounts, currently supporting Foodora with Deliveroo marked work in progress. It lists active orders with arrival and status information, browses order history, reorders past orders by adding them to the cart, and can log in through a browser flow or imported cookies to handle Cloudflare and bot protection. The skill stresses confirming with the user before any reorder or cart-changing action.

Capabilities

- List active orders with arrival times and watch live status
- Browse past order history with JSON detail output
- Reorder past orders into the cart with optional address selection
- Log in via browser flow or import Chrome cookies and sessions
- Refresh sessions with a client ID to avoid using passwords

Usage

- `ordercli foodora login --email you@example.com --password-stdin` — logs in with a password from stdin
- `ordercli foodora orders --watch` — watches active order status live
- `ordercli foodora history show <orderCode> --json` — shows a past order as JSON
- `ordercli foodora reorder <orderCode> --confirm` — confirms reordering a past order
- `ordercli foodora session chrome --url https://www.foodora.at/ --profile "Default"` — imports a Chrome session

When to use

Use when the user asks about recent or past food delivery orders, wants to track an active delivery, or wants to reorder a previous meal from Foodora.

Requirements & notes

Requires the `ordercli` binary (Homebrew tap `steipete/tap/ordercli` or Go install). Deliveroo support is incomplete and needs `DELIVEROO_BEARER_TOKEN`, optionally `DELIVEROO_COOKIE`. Use `--config /tmp/ordercli.json` for testing, and confirm before any reorder or cart-changing action.

peekaboo

Location	/usr/lib/node_modules/openclaw/skills/peekaboo/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Captures and automates the macOS UI with the Peekaboo CLI: screenshots, element targeting, input, apps, and windows.

Overview

Peekaboo is a full macOS UI automation CLI that captures and inspects screens, targets UI elements from annotated snapshots, drives keyboard and mouse input, and manages apps, windows, menus, the Dock, and Spaces. Commands share a snapshot cache and support `--json` output for scripting, and a `vision` command produces annotated UI maps with element IDs. The skill documents the reliable see-click-type flow, capture analysis, and permission requirements.

Capabilities

- Capture screenshots of screens, windows, and menu bar regions
- Produce annotated UI maps with element IDs for targeting
- Click, type, drag, scroll, swipe, and press keys on targets
- Manage apps, windows, dialogs, menus, Dock items, and Spaces
- Record motion-aware live captures and analyze images with prompts

Usage

- `peekaboo see --annotate --path /tmp/peekaboo-see.png` — annotated UI map with element IDs
- `peekaboo click --on B1` — clicks the element with snapshot ID B1
- `peekaboo type "Hello" --return` — types text and presses Return
- `peekaboo image --mode screen --screen-index 0 --retina --path /tmp/screen.png` — retina screen capture
- `peekaboo menu click --app Safari --item "New Window"` — clicks an application menu item

When to use

Use for macOS-only UI automation: clicking and typing into apps, taking and analyzing screenshots, driving menus and dialogs, or scripting window and app management.

Requirements & notes

Requires macOS with the peekaboo binary (Homebrew tap `steipete/tap/peekaboo`) plus Screen Recording and Accessibility permissions. In OpenClaw subprocesses use the default Bridge path and do not pass `--no-remote` unless the process has its own Screen Recording grant. On macOS 15+ a separate "bypass private window picker" prompt may appear.

sag

Location	/usr/lib/node_modules/openclaw/skills/sag/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Generates expressive ElevenLabs text-to-speech audio with local playback and a mac-style say command experience.

Overview

`sag` is a command line text-to-speech tool backed by ElevenLabs that speaks text aloud locally with a `say`-style interface. It supports voice selection, multiple ElevenLabs models ranging from expressive v3 to fast flash, normalization of numbers and URLs, and v3 audio tags that stage delivery effects such as whispering or laughter. The skill also documents chat voice replies that generate MP3 files for sending as media.

Capabilities

- Speak text aloud immediately with a chosen voice
- List voices and print model-specific prompting tips
- Control pronunciation with respelling, normalization, and language bias

- Stage delivery with v3 audio tags like whispers, pauses, and laughter
- Generate MP3 audio files for use as chat voice replies

Usage

- `sag "Hello there"` — speaks text with the default voice and model
- `sag speak -v "Roger" "Hello"` — speaks text in a named voice
- `sag voices` — lists available voices
- `sag -v Clawd -o /tmp/voice-reply.mp3 "Your message here"` — writes speech to an MP3 file

When to use

Use when the user wants spoken audio, a voice-acted narration or character line, or a voice reply file for chat, using ElevenLabs expressive synthesis.

Requirements & notes

Requires the `sag` binary (Homebrew tap `steipete/tap/sag`) and an ElevenLabs API key in `ELEVENLABS_API_KEY` or `SAG_API_KEY`. The default model is `eleven_v3`, where SSML `<break>` is unsupported and `[pause]` style tags are used instead; v2 and v2.5 models support SSML breaks. Confirm voice and speaker before long output.

self-improving-agent

Location	<code>/home/work/.openclaw/skills/self-improving-agent/SKILL.md</code>
Category	Custom workspace skill
Status	Installed, not in active catalog
Summary	Duplicate copy of the self-improvement skill; inactive, the active copy is self-improvement.

Overview

This directory holds a duplicate copy of the self-improvement skill and is not active in the skill catalog. The active copy is the one in the `self-improvement` directory, whose `SKILL.md` content is identical. Consult the self-improvement entry for full capabilities, logging formats, and workflow documentation.

Capabilities

- Mirrors the learning and error logging skill verbatim
- Same log files, hook scripts, and extraction helpers as the active copy
- Kept for inventory purposes only; not loaded from the skill catalog

Usage

- `clawdhub install self-improving-agent` — install command named in the shared source
- `git clone https://github.com/peterskoett/self-improving-agent.git`
`~/.openclaw/skills/self-improving-agent` — manual install path

When to use

Do not load this copy; use the self-improvement skill instead.

Requirements & notes

No additional requirements. The active copy in the `self-improvement` directory is the authoritative version.

session-logs

Location	<code>/usr/lib/node_modules/openclaw/skills/session-logs/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Searches and analyzes stored session JSONL transcripts with <code>jq</code> and <code>ripgrep</code> to recover historical conversation context.

Overview

`session-logs` provides techniques for searching complete conversation history stored in append-only JSONL session files under the agent state directory. Each message records type, timestamp, role, content blocks, and cost, so the skill offers `jq` recipes for listing sessions, extracting user messages, searching assistant replies, computing session and daily costs, and summarizing usage. `Ripgrep` covers cross-session phrase searches, and the `sessions.json` index maps chat providers to session IDs.

Capabilities

- List sessions by date and file size from JSONL timestamps
- Extract user or assistant text messages from any session
- Search keywords within a single session or across all sessions
- Compute session totals and daily cost summaries
- Summarize message counts, timing, and tool usage breakdowns

Usage

- `jq -r 'select(.message.role == "user") | .message.content[]? | select(.type == "text") | .text' <session>.jsonl` — extracts user messages
- `jq -s '[.[] | .message.usage.cost.total // 0] | add' <session>.jsonl` — totals session cost
- `rg -l "phrase" "$SESSION_DIR"/*.jsonl` — finds sessions containing a phrase
- `jq -r '.message.content[]? | select(.type == "toolCall") | .name' <session>.jsonl | sort | uniq -c | sort -rn` — tool usage counts

When to use

Use when the user references older or parent conversations, asks what was said before, or wants cost or usage analysis of past sessions not covered by memory files.

Requirements & notes

Requires `jq` and `rg` (ripgrep). Session files live in `$OPENCLAW_STATE_DIR/agents/<agentId>/sessions/` (default `~/.openclaw/agents/<agentId>/sessions/`) and can be several MB each, so sample large files with `head` and `tail`. Deleted sessions carry a `.deleted.<timestamp>` suffix.

sherpa-onnx-tts

Location	<code>/usr/lib/node_modules/openclaw/skills/sherpa-onnx-tts/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Produces offline text-to-speech audio with the sherpa-onnx CLI and local voice models, with no cloud dependency.

Overview

sherpa-onnx-tts performs fully local text-to-speech using the sherpa-onnx offline CLI through a bundled wrapper script. Setup downloads a platform runtime and a voice model (the default Piper en_US lessac high voice) into the OpenClaw tools directory, then registers their paths as skill environment variables in the OpenClaw config file. After setup, the wrapper renders text to WAV audio entirely offline, and alternate sherpa-onnx voices can be swapped in.

Capabilities

- Synthesize spoken WAV audio entirely offline
- Choose alternate voices from the sherpa-onnx tts-models releases
- Select a specific model file when a directory holds several onnx files
- Override token and data paths for custom model layouts
- Run on macOS, Linux, and Windows through Node

Usage

- `{baseDir}/bin/sherpa-onnx-tts -o ./tts.wav "Hello from local TTS."` — renders text to a WAV file
- `export PATH="{baseDir}/bin:$PATH"` — adds the wrapper to PATH for direct use
- `node {baseDir}\bin\sherpa-onnx-tts -o tts.wav "Hello from local TTS."` — Windows invocation

When to use

Use when the user wants private, offline text-to-speech without cloud services or API keys, or a lightweight local voice for generated text.

Requirements & notes

Requires `SHERPA_ONNX_RUNTIME_DIR` and `SHERPA_ONNX_MODEL_DIR` environment variables pointing to the downloaded runtime and model directories, configured in the active OpenClaw config file. Works on macOS, Linux, and Windows; set `SHERPA_ONNX_MODEL_FILE` or pass `--model-file` when the model directory contains multiple onnx files.

skill-creator (built-in)

Location	/usr/lib/node_modules/openclaw/skills/skill-creator/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Authoring rules and validation helpers for creating, editing, auditing, and restructuring AgentSkills and SKILL.md files.

Overview

This built-in skill documents how to create, edit, audit, tidy, validate, or restructure AgentSkills and SKILL.md files. Skills are compact triggerable workflows: metadata is always visible, the body loads only after the trigger fires, and references, scripts, and assets load only as needed. The skill prescribes hard authoring rules, a standard directory shape, an edit workflow, and frontmatter validation commands so skills stay lean and reliable.

Capabilities

- Create new AgentSkills with lean SKILL.md files and organized resource folders
- Audit and tidy existing skills by removing generic advice and keeping brittle syntax
- Restructure long skills by moving examples into references/ and helpers into scripts/
- Validate YAML frontmatter after every edit with bundled or inline tooling
- Condense frontmatter descriptions into quoted noun-phrase triggers
- Replace tables with bullets and relax prose while preserving safety rules

Usage

- `python skills/skill-creator/scripts/quick_validate.py skills/<name>` — conservatively validate one skill folder
- `python - <<'PY' ... yaml.safe_load(fm)` — inline snippet validating frontmatter across all SKILL.md files
- `skills/skill-creator/scripts/quick_validate.py` — bundled validator; repo-local frontmatter may allow extra keys

When to use

Load when the user wants to create a new skill, edit or restructure a SKILL.md, audit a skill's layout, or validate skill frontmatter.

Requirements & notes

The validation snippets require Python with a YAML parser such as PyYAML. Note that the active skill-creator in this environment is a different, larger custom skill at `~/openclaw/skills/skill-creator/SKILL.md`; this built-in copy is installed but is not the one active in the skill catalog.

slack

Location	/usr/lib/node_modules/openclaw/skills/slack/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Slack tool guidance for sending, reading, editing, and deleting messages, plus reactions, pins, emoji lists, and member info.

Overview

This skill teaches the agent how to drive the `slack` tool for day-to-day Slack operations inside OpenClaw. It covers sending and reading messages, editing and deleting posts, adding emoji reactions, pinning and unpinning, listing pins and reactions, listing workspace emoji, and looking up member information. The guidance emphasizes reusing `channelId` values and Slack timestamp message IDs already present in context rather than fuzzy channel names.

Capabilities

- Send messages to channels or users with `to` targets like `channel:<id>`
- Read recent messages from a channel with a configurable limit
- Edit and delete existing messages identified by Slack timestamps
- Add emoji reactions and list reactions on a message
- Pin or unpin messages and list pins for a channel
- Look up member information and the workspace emoji list

Usage

- { "action": "sendMessage", "to": "channel:C123", "content": "Hello" } — post a message
- { "action": "readMessages", "channelId": "C123", "limit": 20 } — read recent channel messages
- { "action": "react", "channelId": "C123", "messageId": "1712023032.1234", "emoji": ":white_check_mark:" } — react to a message
- { "action": "editMessage", "channelId": "C123", "messageId": "1712023032.1234", "content": "Updated text" } — edit a message
- { "action": "deleteMessage", "channelId": "C123", "messageId": "1712023032.1234" } — delete a message
- { "action": "listPins", "channelId": "C123" } — list pinned messages

When to use

Load whenever the user asks to post, read, edit, delete, react to, pin, or inspect Slack messages, pins, reactions, emoji, or members.

Requirements & notes

Requires Slack channel configuration (channels.slack) in the OpenClaw config. Confirm destructive deletes when the context is unclear, keep outbound messages short, avoid Markdown tables in messages, and prefer exact thread or message IDs over channel-name guesses.

songsee

Location	/usr/lib/node_modules/openclaw/skills/songsee/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Generate spectrograms and audio feature-panel visualizations from audio files or streams using the songsee CLI.

Overview

songsee is a CLI that renders spectrograms and multi-panel audio feature visualizations from WAV, MP3, or streamed audio. It supports a single spectrogram panel and grids of views such as mel, chroma, HPSS, self-similarity, loudness, tempo, MFCC, and flux. Output styling, pixel size, FFT window and hop settings, frequency range, and time slices are all controllable from the command line.

Capabilities

- Render a spectrogram image from an audio file or standard input
- Build multi-panel feature grids combining up to nine visualization types
- Extract time slices with start offset and duration controls
- Style output with classic, magma, inferno, viridis, or gray palettes
- Tune FFT window and hop plus minimum and maximum frequency
- Export JPG or PNG at chosen output dimensions

Usage

- songsee track.mp3 — default spectrogram render
- songsee track.mp3 --viz spectrogram,mel,chroma,hpss,selfsim,loudness,tempogram,mfcc,flux — multi-panel feature grid
- songsee track.mp3 --start 12.5 --duration 8 -o slice.jpg — time-sliced output image
- cat track.mp3 | songsee - --format png -o out.png — render from standard input as PNG
- --style, --width, --height, --window, --hop, --min-freq, --max-freq — styling and FFT options

When to use

Load when the user asks for a spectrogram, audio visualization, or acoustic feature panel from a track or recording.

Requirements & notes

Requires the songsee binary, installable via Homebrew from steipete/tap/songsee. WAV and MP3 decode natively; other formats rely on ffmpeg when it is available. Passing multiple --viz values renders a grid.

sonoscli

Location	/usr/lib/node_modules/openclaw/skills/sonoscli/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Control Sonos speakers for discovery, status, playback, volume, grouping, favorites, queues, and Spotify search on the local network.

Overview

sonoscli provides the sonos command for controlling Sonos speakers on the local network. It covers speaker discovery, per-room status, playback control, volume, multi-room grouping, favorites, queues, and Spotify search through SMAPI. The skill also includes detailed troubleshooting for common SSDP discovery failures in sandboxed or restricted network environments, including macOS local-network permission issues.

Capabilities

- Discover Sonos speakers and read named room status
- Play, pause, and stop media on named speakers
- Set speaker volume with numeric levels
- Manage groups: status, join, unjoin, party, and solo modes
- List and open favorites, and list, play, or clear queues
- Search Spotify tracks via the SMAPI integration

Usage

- `sonos discover` — find speakers on the local network
- `sonos status --name "Kitchen"` — show status for a named speaker
- `sonos play|pause|stop --name "Kitchen"` — playback control
- `sonos volume set 15 --name "Kitchen"` — set the volume level
- `sonos group status|join|unjoin|party|solo` — grouping operations
- `sonos smapi search --service "Spotify" --category tracks "query"` — Spotify search

When to use

Load when the user wants to control, inspect, group, or play music on Sonos speakers, or troubleshoot Sonos discovery errors like no route to host.

Requirements & notes

Requires the sonos binary, installable via `go install github.com/steipete/sonoscli/cmd/sonos@latest`. If SSDP fails, pass `--ip <speaker-ip>`; on macOS enable Settings -> Privacy & Security -> Local Network for the gateway's host parent process (node, Terminal, or Visual Studio Code). Optional Spotify Web API search needs `SPOTIFY_CLIENT_ID/SECRET`; a `bind: operation not permitted` error indicates a sandbox without network access.

spotify-player

Location	/usr/lib/node_modules/openclaw/skills/spotify-player/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Control Spotify playback and search from the terminal using spogo (preferred) or spotify_player as the fallback CLI.

Overview

This skill drives terminal Spotify playback and search through the spogo CLI, falling back to `spotify_player` when needed. It covers authentication via browser cookie import, track search, playback control, device switching, and status checks. The `spotify_player` fallback adds Spotify Connect integration, track liking, and a full-screen TUI with discoverable keyboard shortcuts.

Capabilities

- Authenticate to Spotify by importing browser cookies with spogo
- Search tracks and control play, pause, next, and previous playback
- List and select playback devices by name or ID
- Check current playback status
- Connect devices and like tracks via the `spotify_player` fallback
- Configure Spotify Connect with a user `client_id`

Usage

- `spogo auth import --browser chrome` — import cookies for authentication
- `spogo search track "query"` — search Spotify tracks
- `spogo play|pause|next|prev` — playback control
- `spogo device list`, `spogo device set "<name|id>"` — device management
- `spogo status` — show playback status
- `spotify_player search "query"` — fallback search command

When to use

Load when the user wants to play, pause, search, or manage Spotify playback or devices from the terminal.

Requirements & notes

Requires a Spotify Premium account and either `spogo` or `spotify_player` installed (both via Homebrew; `spogo` from the `steipete/tap`). `spotify_player` stores config in `~/.config/spotify-player` (for example `app.toml`); Spotify Connect integration requires setting a user `client_id` there. TUI shortcuts are available via `?` inside the app.

summarize

Location	<code>/usr/lib/node_modules/openclaw/skills/summarize/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Summarize or transcribe URLs, YouTube videos, podcasts, articles, transcripts, PDFs, and local files with the <code>summarize</code> CLI.

Overview

`summarize` is a fast CLI for summarizing web URLs, local files, and YouTube links, with best-effort transcript extraction from videos and no `yt-dlp` required. It works across providers including OpenAI, Anthropic, xAI, and Google using the API key for the chosen one; the default model is `auto` and config can pick the provider and model. Optional Firecrawl and Apify integrations handle blocked sites and YouTube fallback extraction.

Capabilities

- Summarize web pages, articles, and URLs with controllable output length
- Summarize local files including PDFs
- Extract best-effort YouTube and video transcripts without `yt-dlp`
- Print raw extracted content with no LLM summarization
- Emit machine-readable JSON output
- Fall back to Firecrawl extraction or Apify for stubborn sources

Usage

- `summarize "https://example.com"` — summarize a web URL
- `summarize "/path/to/file.pdf"` — summarize a local PDF
- `summarize "https://youtu.be/dQw4w9WgXcQ" --youtube auto` — summarize a YouTube video
- `summarize "https://youtu.be/dQw4w9WgXcQ" --youtube auto --extract` — print transcript extraction only
- `--length short|medium|long|x1|xx1|<chars>` — control summary length
- `--json` — machine-readable output

When to use

Load when the user says "use `summarize.sh`", "what's this link or video about?", "summarize this URL or article", or "transcribe this YouTube or video".

Requirements & notes

Requires the `summarize` binary (Homebrew: `steipete/tap/summarize`) and an API key such as `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `XAI_API_KEY`, or `GEMINI_API_KEY` (with Google aliases). Optional `FIRECRAWL_API_KEY` helps with blocked sites and `APIFY_API_TOKEN` enables the YouTube fallback; config lives at `~/.summarize/config.json`. If a transcript is huge, return a tight summary first and ask which section to expand.

things-mac

Location	/usr/lib/node_modules/openclaw/skills/things-mac/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Add, update, list, search, or inspect Things 3 todos, inbox, today, projects, areas, and tags on macOS.

Overview

things-mac wraps the things CLI (things3-cli) to read the local Things 3 database and manage todos through the Things URL scheme. Read commands cover inbox, today, upcoming, search, projects, areas, and tags; write commands add and update todos with notes, schedules, deadlines, lists, headings, tags, and checklist items. Updates use an auth token and support safe dry-run previews that print the URL without opening Things.

Capabilities

- Read inbox, today, and upcoming views from the Things database
- Search todos and list projects, areas, and tags
- Add todos with notes, when-dates, deadlines, tags, and checklists
- Place todos into projects, areas, and project headings
- Update titles, notes (replace, append, prepend), lists, and tags
- Complete or cancel todos and preview writes with dry-run

Usage

- `things inbox --limit 50` — read the inbox
- `things search "query"` — search todos
- `things add "Title" --notes "..."` `--when today` `--deadline 2026-01-02` — create a todo
- `things add "Pack charger" --list "Travel" --heading "Before"` — add into a project heading
- `things update --id <UUID> --auth-token <TOKEN> --completed` — complete a todo
- `things --dry-run add "Title"` — preview the URL without opening Things

When to use

Load when the user wants to manage Things 3 todos on macOS: capture tasks, review the inbox or today list, reschedule items, or reorganize projects and tags.

Requirements & notes

macOS-only; install via `GOBIN=/opt/homebrew/bin go install github.com/ossianhempel/things3-cli/cmd/things@latest`. Grant Full Disk Access to the calling app if database reads fail (Terminal or OpenClaw.app), and set `THINGSDB` or `--db` for a non-default data folder. Writes need `THINGS_AUTH_TOKEN` or `--auth-token`; deleting todos is not supported, so use `--completed` or `--canceled`.

tmux

Location	/usr/lib/node_modules/openclaw/skills/tmux/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Control tmux sessions and panes for interactive CLIs: list, capture output, send keys, paste text, and monitor prompts.

Overview

This skill covers controlling existing interactive tmux sessions from scripts: listing sessions, windows, and panes, capturing pane output including full scrollback, and sending literal text or special keys. It includes prompt-check patterns for safely answering interactive CLI confirmations, plus helper scripts for session discovery and waiting on output text. It explicitly excludes one-shot shell commands and new non-interactive background jobs.

Capabilities

- List sessions, windows, and panes using `session:window.pane` targets
- Capture pane output to stdout, including full scrollback history
- Send literal text with Enter pressed separately to avoid paste surprises
- Send special keys like C-c, C-d, and Escape
- Create, rename, and kill sessions
- Monitor panes for prompts before approving any selection

Usage

- `tmux ls` — list tmux sessions
- `tmux capture-pane -t shared:0.0 -p` — print current pane content
- `tmux capture-pane -t shared:0.0 -p -S -` — capture full scrollback
- `tmux send-keys -t shared:0.0 -l -- "Please continue"` — send literal text
- `tmux send-keys -t shared:0.0 Enter` — press Enter separately
- `scripts/wait-for-text.sh` — helper that waits until pane output contains text

When to use

Load when working with existing interactive tmux sessions: driving interactive CLIs, reading pane output, or answering prompts in long-running terminal programs.

Requirements & notes

Requires tmux on macOS or Linux (installable via Homebrew). Use `-l --` for arbitrary text and split text from Enter sends. `capture-pane -p` prints to stdout for scripts, `-S -` captures full scrollback, and tmux sessions persist across SSH disconnects. `scripts/find-sessions.sh` discovers sessions.

trello

Location	<code>/usr/lib/node_modules/openclaw/skills/trello/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Manage Trello boards, lists, and cards with curl and jq against the Trello REST API.

Overview

This skill manages Trello boards, lists, and cards directly from OpenClaw by issuing curl requests to the Trello REST API and shaping responses with jq. It covers listing boards, lists, and cards, creating and moving cards, commenting, and archiving. Setup requires a Trello API key and token exported as environment variables, and the source documents rate limits and secret-handling cautions.

Capabilities

- List boards, lists within boards, and cards within lists
- Create cards with title and description in a target list
- Move cards between lists by updating idList
- Add comments to existing cards
- Archive cards by closing them
- Filter boards and cards with jq expressions

Usage

- `curl -s "https://api.trello.com/1/members/me/boards?key=$TRELLO_API_KEY&token=$TRELLO_TOKEN" | jq '.[0] | {name, id}'` — list boards
- `curl -s "https://api.trello.com/1/boards/{boardId}/lists?key=$TRELLO_API_KEY&token=$TRELLO_TOKEN" | jq '.[0] | {name, id}'` — list lists in a board
- `curl -s -X POST "https://api.trello.com/1/cards?key=$TRELLO_API_KEY&token=$TRELLO_TOKEN" -d "idList={listId}" -d "name=Card Title"` — create a card
- `curl -s -X PUT "https://api.trello.com/1/cards/{cardId}?key=$TRELLO_API_KEY&token=$TRELLO_TOKEN" -d "idList={newListId}"` — move a card
- `curl -s -X PUT "https://api.trello.com/1/cards/{cardId}?key=$TRELLO_API_KEY&token=$TRELLO_TOKEN" -d "closed=true"` — archive a card

When to use

Load when the user wants to view or manage Trello boards, lists, or cards: creating work items, moving cards, commenting, or archiving.

Requirements & notes

Requires jq plus environment variables `TRELLO_API_KEY` and `TRELLO_TOKEN` from `https://trello.com/app-key`; keep them secret since they grant full account access. Rate limits: 300 requests per 10 seconds per API key, 100 per 10 seconds per token, and 100 per 900 seconds for `/1/members` endpoints. Board, list, and card IDs come from Trello URLs or the list commands.

voice-call

Location	/usr/lib/node_modules/openclaw/skills/voice-call/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Start and manage outbound voice calls through the OpenClaw voice-call plugin with Twilio, Telnyx, Plivo, or mock providers.

Overview

The voice-call skill drives the OpenClaw voice-call plugin for starting and inspecting outbound phone calls. Both a CLI (`openclaw voicecall`) and an agent-facing `voice_call` tool are supported, with actions for initiating calls, continuing conversations, speaking new messages, ending calls, and checking status. Providers include Twilio, Telnyx, Plivo, and a mock provider for offline development.

Capabilities

- Start outbound voice calls with an initial spoken message
- Check call status by call ID
- Continue a call with follow-up messages and speak to the user
- End active calls on demand
- Run calls through Twilio, Telnyx, Plivo, or a mock provider
- Operate from either the CLI or the `voice_call` tool

Usage

- `openclaw voicecall call --to "+15555550123" --message "Hello from OpenClaw"` — place a call
- `openclaw voicecall status --call-id <id>` — inspect call status
- `initiate_call` (message, to?, mode?) — `voice_call` action to start a call
- `continue_call` (callId, message) — `voice_call` action to continue a call
- `speak_to_user` (callId, message) — speak into an active call
- `end_call` (callId) and `get_status` (callId) — end or inspect a call

When to use

Load when the user wants the agent to place, manage, or inspect outbound voice phone calls, or to speak to someone over an active call.

Requirements & notes

Requires the voice-call plugin enabled (`plugins.entries.voice-call.enabled`), with config under `plugins.entries.voice-call.config`. Twilio needs `twilio.accountSid/authToken` plus `fromNumber`; Telnyx needs `telnyx.apiKey/connectionId`; Plivo needs `plivo.authId/authToken`. The mock provider works without network access for development.

wacli

Location	/usr/lib/node_modules/openclaw/skills/wacli/SKILL.md
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Send third-party WhatsApp messages or sync and search WhatsApp history via wacli, reserved for messaging other people.

Overview

wacli is a WhatsApp CLI used only when the user explicitly asks to message someone else on WhatsApp or to sync and search WhatsApp history. It handles QR-based authentication, continuous sync, chat and message search, history backfill, and sending text, group messages, and files. It is deliberately excluded from normal user chats because OpenClaw already routes WhatsApp conversations automatically.

Capabilities

- Authenticate with QR login and run initial or continuous sync
- List chats by name or number and search messages with date ranges
- Backfill chat history from the phone on request
- Send direct text messages to phone numbers

- Send group messages and files with captions
- Diagnose setup problems with the doctor command

Usage

- `wacli auth` — QR login and initial sync
- `wacli sync --follow` — continuous background sync
- `wacli chats list --limit 20 --query "name or number"` — find chats
- `wacli messages search "invoice" --after 2025-01-01 --before 2025-12-31` — search history by date
- `wacli send text --to "+14155551212" --message "Hello! Are you free at 3pm?"` — send a text
- `wacli send file --to "+14155551212" --file /path/agenda.pdf --caption "Agenda"` — send a file

When to use

Load only when the user explicitly asks to message a third party on WhatsApp or to sync or search WhatsApp chat history.

Requirements & notes

Requires the `wacli` binary (Homebrew `steipete/tap/wacli` or `go install github.com/steipete/wacli/cmd/wacli@latest`); the store directory is `~/.wacli` with a `--store` override, and `--json` gives machine-readable output. Backfill needs the phone online and is best-effort. JIDs: direct chats look like `<number>@s.whatsapp.net`, groups like `<id>@g.us`. Always confirm recipient and message text before sending.

xurl

Location	<code>/usr/lib/node_modules/openclaw/skills/xurl/SKILL.md</code>
Category	OpenClaw built-in skill
Status	Installed, not in active catalog
Summary	Authenticated X work with xurl: posts, replies, reads, search, DMs, media upload, followers, auth status, and raw v2 API calls.

Overview

`xurl` is a CLI for working with the X API, offering shortcut commands that return JSON plus a raw mode that can call any v2 endpoint. Shortcuts cover posting, replying, quoting, deleting, reading, searching, timelines, mentions, likes, reposts, bookmarks, follows, blocks, mutes, and direct messages. It also handles media uploads, authentication and app selection, and prescribes strict secret-handling rules for agent sessions.

Capabilities

- Post, reply, quote, delete, and read posts by ID or full URL
- Search posts and read timelines and mentions with result counts
- Like, repost, bookmark, follow, block, and mute accounts
- Send and list direct messages
- Upload images and videos and attach them to posts
- Call any X API v2 endpoint in raw mode with custom payloads

Usage

- `xurl post "Hello world!"` — publish a post
- `xurl reply POST_ID "Nice."` — reply to a post
- `xurl search "query" -n 20` — search posts
- `xurl dm @handle "message"` — send a direct message
- `xurl media upload image.jpg` — upload media for attachment
- `xurl -X POST /2/tweets -d '{"text":"Hello world!"}'` — raw v2 API call

When to use

Load when the user wants to post, read, search, or manage content and relationships on X, upload media, or call the X v2 API directly.

Requirements & notes

Requires the `xurl` binary (Homebrew `xdevplatform/tap/xurl` or `npm @xdevplatform/xurl`) with valid auth, checked via `xurl auth status`. Never read, print, or inspect `~/.xurl`, never ask for pasted tokens, and never use `--verbose` in agent sessions since it can expose auth headers. On 401 or 403 check auth scope or app mismatch; on 429 back off.

